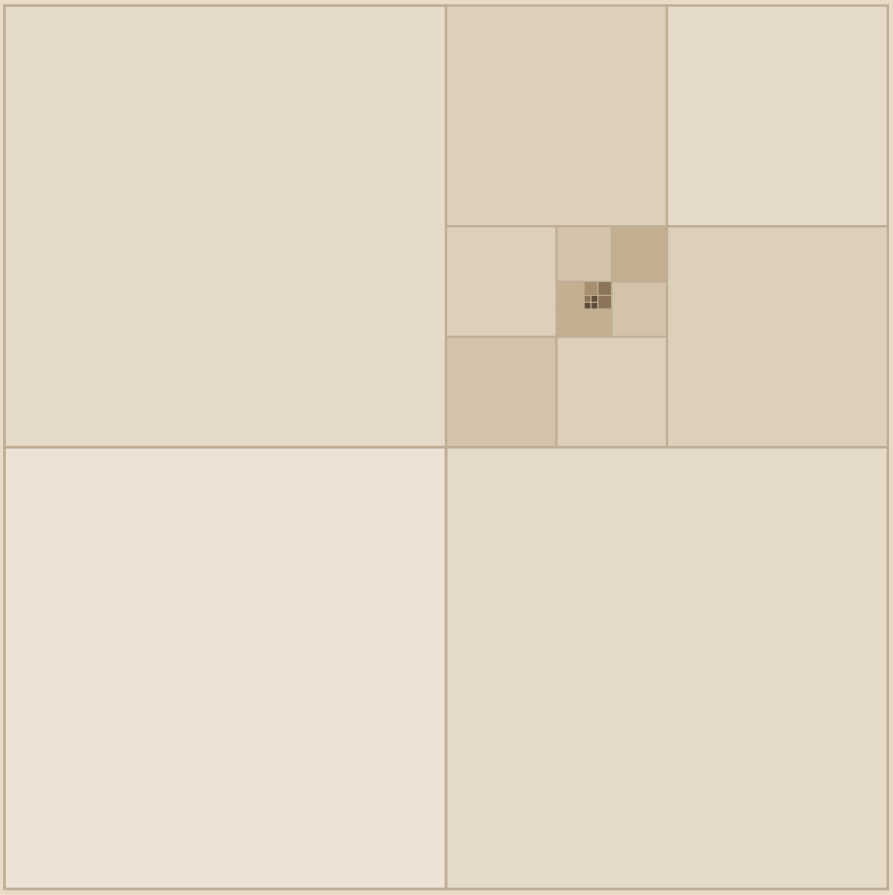
TESSERA · A STUDY GUIDE



z 12 · x 3339 · y 1788 Macau

Sixteen *Tiles*

The technology behind one map server, in short chapters

Pure Core, Effects at the Edge · C# and Its Types · Modular Monolith · Nginx · ASP.NET Core · .NET Concurrency
PostgreSQL · PostGIS · Web Application Security · Build, Test and Release · Performance · The Mathematics of the Limits
Mini-guides: the Python pipeline · the map in the browser · the scripts · the office machine
Tessera, phase one · October 2026

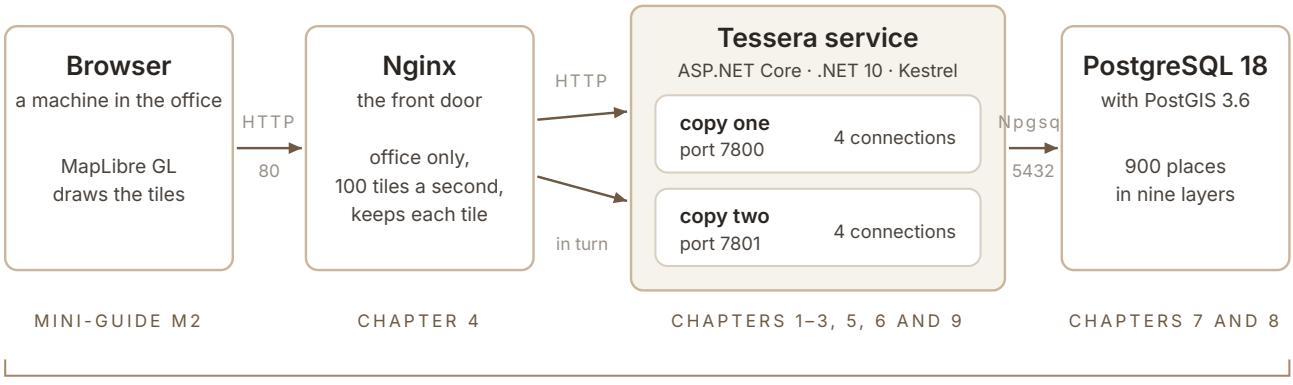
One job, one rule

One developer can build a map server that stays correct, safe and fast enough by giving each technology one job and one firm rule, sizing every limit from a few formulas, and measuring the whole. This guide shows how Tessera does it, one technology to a chapter.

Tessera is a map server. It sends the cafés, parks, museums and bus stops of Macau to browsers as ***vector tiles***: small squares of map that hold places as shapes, not pictures. It serves 900 places, in nine layers such as cafés and parks, to every machine in one office, on port 80. One developer builds it alone, in C# 14 on .NET 10. The work comes as 30 ***user stories***. A user story is a short statement of one thing a person needs, with the criteria that say when it is done; each has an ID, such as US-07 or S-01, which the guide cites where the code meets it. The delivery plan releases 16 of the stories, the map and the tiles beneath it, to the office on 7 December 2026. The repository, the plan's worked answer, already holds all 30, and this guide describes the repository as it stands.

A server that many people share carries risk: any machine may send it anything, and one mistake reaches every user at once. The guide's first four parts meet that risk. The design refuses mistakes before the program runs; each stop on a request's way, from the door to the database, keeps one rule; tests and measurements prove the service safe and fast; and a few formulas tie every number together. Part five covers the programs around the service.

EXHIBIT 0.1 One request for a tile, and the chapter that covers each part



Chapters 10 to 12: how the project tests, releases, measures and sizes every part
Mini-guides M1, M3 and M4: the pipeline that cleans the places, the scripts, and the office machine

Source: `nginx/default.conf.template`, `tessera.json`, `db/Dockerfile`, `scripts/lib/plan.ts`

As Exhibit 0.1 shows, a request for a ***tile***, one square of the map, crosses four programs. The browser asks Nginx. Nginx refuses any machine outside the office and passes the rest to the two copies of the service, in turn. The service asks the database, PostgreSQL, whose PostGIS extension cuts the places into the tile. Each program has one job, and parts one and two give each a chapter.

Each chapter follows one plan

Each chapter opens with its answer in bold, then says what the technology is and where it fits in Tessera. Next it shows how the technology works, with the project's own code trimmed to the lines that make the point. A *Foundations* box links the idea to what you met at school, and a *Try it* box gives you a small task. Each chapter ends with a common mistake, four questions with their answers, and one question to think

over. Most fill three or four pages; the mini-guides of part five fill two or three, with a Foundations box, a Try it box and three questions each.

The guide falls into five parts, each built on one idea, and the appendices:

PART	CHAPTERS	THE IDEA
One	01 to 03	The design refuses mistakes before the program runs: pure functions, C#'s types, and one program with firm walls inside.
Two	04 to 08	One request, from the door to the database: Nginx, ASP.NET Core, .NET's tasks and threads, PostgreSQL and PostGIS.
Three	09 to 11	Proof that the service is safe, tested and fast: security, the build and its tests, and performance.
Four	12	Every limit follows from a few formulas: the arithmetic that sizes the numbers earlier chapters name.
Five	M1 to M4	The parts around the service, one short guide each: the Python pipeline, the map in the browser, the scripts and the office machine.
Appendices	A, B	Tessera's own commands for a working day, the general Linux commands beside them, and a glossary.

Bold italics mark a term where the guide explains it; the glossary defines every such term, and the index finds its page. Words set in grey come from the Oxford list of common words, which the guide uses wherever it can.

You can read the guide without running the code. Read each sample as you would a worked example in mathematics: line by line, asking what goes in and what comes out.

Tessera at a glance

One tile's journey, from a click to the screen

`GET /food.cafes/12/3339/1788.pbf`

1. **Browser · MapLibre GL** — M2
   asks for one square of Macau, over HTTP on port 80
2. **Nginx 1.29 · the door** — CH 4, 9
   refuses machines outside the office; caps each at 100 a second
3. **Nginx's store of tiles** — CH 4, 11
   hands back a kept tile at once; else passes on to a copy, in turn
4. **ASP.NET Core on .NET 10 · two copies** — CH 5, 6
   port 7800 or 7801; await frees each thread while the database works
5. **TileServer.Answer · the pure core** — CH 1, 2, 3
   `catalogue → Find → Of` : list loaded, layer published, tile on the grid
6. **Database · the one effect** — CH 1, 6, 7
   one of 4 pooled connections; any failure comes back as a value
7. **PostgreSQL 18 · PostGIS 3.6** — CH 7, 8
   finds the shapes in the square, cuts them and packs one tile
8. **Back to the browser** — CH 4, 5
   200, a weak ETag and max-age=60; Nginx packs it with gzip and keeps it

FIVE NUMBERS OF THE BUILD

30	167	2	100	4
stories docs/user-stories	tests of the service in three projects	copies of the service	requests a second for each machine	connections a copy

AROUND THE SERVICE · THE MINI-GUIDES

- **M1 · Python** — cleans the places
- **M2 · TypeScript** — draws the map
- **M3 · zx scripts** — build, start, back up
- **M4 · the office** — one Windows machine

Contents

One job, one rule 2

PART ONE *The design refuses mistakes before the program runs*

01 **Pure Core, Effects at the Edge** 7
Pure functions decide; the database is the one effect, at the edge

02 **C# and Its Types** 12
The compiler checks every value, and the build stops at the first warning

03 **Modular Monolith** 17
One program, with walls that a test keeps standing

PART TWO *One request, from the door to the database*

04 **Nginx as a Reverse Proxy** 22
Every request enters by one guarded door

05 **ASP.NET Core** 28
Every request passes a line of checks, then one method answers it

06 **.NET Concurrency** 34
Many requests share a few cores, safely

07 **PostgreSQL** 40
A grant is the only way to publish

08 **PostGIS** 45
One statement turns shapes into a tile

PART THREE *Proof that the service is safe, tested and fast*

09 **Web Application Security** 49
Six attacks meet six defences, checked from outside

10 **Build, Test and Release** 55
Every change passes every check before it ships

11 **Performance** 61
Each stage caps its cost, and a query plan shows where the time goes

PART FOUR *Every limit follows from a few formulas*

12 **The Mathematics of the Limits** 67
One measured time sizes the pool, the queue and the doors

PART FIVE *The parts around the service, one short guide each*

M1 **The Python Pipeline** 75
Dirty places in, one clean SQL script out

M2 **The Map in the Browser** 79
A pure model decides; MapLibre draws

M3 **The Scripts** 83
Pure decisions in lib, commands at the edge

M4 **The Office Machine** 86
One Windows machine, four parts, two scheduled tasks

APPENDICES *Commands and words*

A **Everyday Commands** 89

B **Linux Commands for Every Day** 92

Glossary 96

Index 102

01
Pure Core, Effects at the Edge

Tessera follows one design rule: pure functions make every decision, failures travel as values, and the program's one dangerous step, the database, arrives from the edge as a function handed in. C# 14 carries the rule with records, nullable types, pattern matching and the `Result` type of a small library. Chapter 2 shows how C#'s types hold the code to it.

Pure functions decide; the edge acts

The style rests on three habits, which ADR 0004 sets for every line of the service. (An ADR, or ***architecture decision record***, is a short note of one design choice and its reasons.) First, data is ***immutable***: a value never changes once made, so to 'change' one you make a new one. Second, each decision is a function that takes values and returns a value, and an expected failure comes back as a value too. Third, a function reads only its parameters: the clock, the settings and the database reach it as values passed in, so the same inputs always give the same output.

A ***pure function*** behaves like a function in mathematics. Give $f(x) = 2x$ the number 3 and it returns 6, today, tomorrow and on every computer. Anything a function does besides returning its answer is a ***side effect***: reading a database, writing a log, reading the clock. A recipe card decides; only the cook lights the oven. Keep the cards many and the cooks few.

A browser asks Tessera for one square of map, a tile, and receives the places of Macau inside it. Every decision on the way is pure: whether the list of layers has loaded, whether the layer exists, whether the address lies on the grid, which SQL to send, and what to answer. One step acts on the world: running the query. Exhibit 1.1 follows the request through `TileServer.Answer` and gives the answer each step returns when it fails.

EXHIBIT 1.1 The path of one tile request through `TileServer.Answer`, and the answer each step gives when it fails

STEP	WHAT IT DECIDES	WHEN IT FAILS
`catalogue()`	Has the list of layers loaded yet?	503, try again
`layers.Find`	Is a layer of that exact name published?	404
`TileAddress.Of`	Does the address name a tile on the grid?	404
`Queries.Tile`	Build the SQL text and its parameters as a `Query`	cannot fail
`read`	**The one effect:** run the query in PostgreSQL	a `DatabaseError` value
`built.Match`	Wrap the bytes with their headers, or turn each kind of `DatabaseError` into an answer	404, 503 or 500

Source: `src/Tessera/Modules/Tessera/Features/Tiles.cs`

The project calls the design a ***functional core, imperative shell***: pure functions make every decision, and a thin shell carries out the effects. Five of the six steps are pure, so a test checks them in milliseconds.

> FOUNDATIONS Subroutines, parameters and return values; records; selection. A pure function is a subroutine that returns a value and has no side effects. C#'s `switch` is selection, like `if`, and a record is the same idea as a record in pseudocode: named fields grouped into one value.

Result makes a failure a value

A step that can fail returns a `Result<T, E>`: either a success holding its value, of type `T`, or a failure holding the reason, of type `E`. Tessera takes `Result` from a library, CSharpFunctionalExtensions, and writes none of its own, as ADR 0004 records. `ConnectionString.FromUrl` shows the shape, trimmed:

CSHARP

```csharp
public static Result<ConnectionString, string> FromUrl(string? url)
{
    if (string.IsNullOrEmpty(url))
        return Result.Failure<ConnectionString, string>("missing or empty");
    // … a URL that is not postgresql://user:password@host:port/database fails the same way
    return new ConnectionString(/* Npgsql's connection string, built from the URL */);
}
```

The last line returns a plain `ConnectionString`; C# turns it into a success, since the library tells the compiler how. Four methods do the rest of the work. `Map` applies a function that cannot fail to the value inside a success. `Bind` applies one that can, which returns a `Result` of its own. `MapError` rewrites the reason inside a failure. All three pass the other case on untouched, so steps joined this way stop at the first failure. `Match` ends the chain: it takes two functions, one for each case, and runs one of them.

The code passes functions as values. A ***lambda*** is a function without a name, written where it is needed: `why => $"{variable}: {why}"` takes a reason and puts the variable's name in front of it. A ***method group***, such as `Report`, names a method that already exists and passes it without calling it.

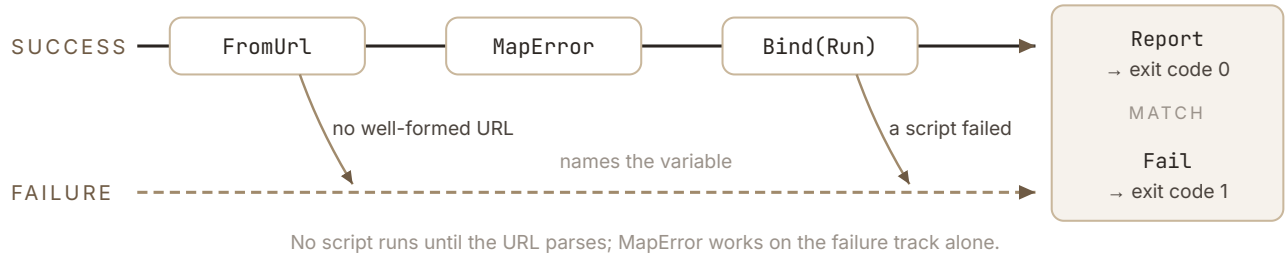
The seed chains its steps; the tile route returns early

The seed is the command `Tessera seed`, in `Program.cs`, which loads the places into the database. Its code reads as a railway with two tracks, as Exhibit 1.2 draws. A run stays on the success track while each step succeeds; the first failure switches it to the failure track, where every later step lets it pass. Programmers call this style ***railway-oriented programming***.

CSHARP

```csharp
return ConnectionString.FromUrl(configuration.Build()[variable])
    .MapError<ConnectionString, string, string>(why => $"{variable}: {why}")
    .Bind(connection => SeedScripts.Run(connection.Text, SeedScripts.Folder))
    .Match(Report, why => Fail([why]));
```

EXHIBIT 1.2 Two tracks: the first failure leaves the success track, and `Match` reads where the run ended



No script runs until the URL parses; MapError works on the failure track alone.

Source: `src/Tessera/Program.cs`, `Infrastructure/Sql/Connection.cs`, `Infrastructure/Seeding/SeedScripts.cs`

The tile route can refuse for three reasons. ADR 0004 judges that such a route reads better from top to bottom, with one `if` and one `return` for each refusal. Each check returns a value that may be missing, and `is not { } name` tests it and names it for the lines below:

CSHARP

```
if (catalogue() is not { } layers)
    return Problems.Unavailable("The list of layers is not ready yet.");
if (layers.Find(layer) is not { } found)
    return Problems.NotFound("No published layer has that name.");
if (TileAddress.Of(z, x, y, settings.MinZoom, settings.MaxZoom) is not { } tile)
    return Problems.NotFound("No tile has those coordinates.");

var built = await read(Queries.Tile(found, tile, settings), cancel).ConfigureAwait(false);
```

The rails are the same, laid as a ladder: no query runs until all three checks pass (US-07 test 1). Each refusal is a problem document from `Common/Api/Problems.cs`: a short report in JSON, the form every Tessera error takes.

The database is the one effect, and it arrives as a function

`TileServer` never names the database. Its constructor takes the effect as a parameter, a `TileReader`: a ***delegate***, C#'s name for the type of a function.

CSHARP

```
public delegate Task<Result<byte[]?, DatabaseError>> TileReader(
    Query query, CancellationToken cancel);
public sealed class TileServer(
    Func<LayerCatalogue?> catalogue, TileReader read, TileSettings settings, ILogger log)
```

When the program runs, `TileRoutes.MapTiles` hands it `database.Single<byte[]>`, a method group, so `read` runs the real query; a test hands it a lambda that returns any answer it likes. The log, a second effect, arrives the same way, as an `ILogger`, so a test can hand in one that records each line. Inside `Database`, one private method, `Run`, runs the work on a borrowed connection. Npgsql, the PostgreSQL client for .NET, signals a failure with an ***exception***, .NET's way to break off a method when something goes wrong. `Run` catches the exception and turns it into a value:

```csharp
// Database.Run, in part
try
{
    return Result.Success<T, DatabaseError>(
        await Within(_limit, limit => RunOnce(work, limit), cancel).ConfigureAwait(false));
}
catch (NpgsqlException error)
{
    return Result.Failure<T, DatabaseError>(Error(error));
}
// … and a query past the service's own clock becomes a value too (chapter 6)
```

`Run` takes the work as a function, so it decides when the work runs and can wrap it in `try`: `RunOnce` borrows the connection and runs the work, and `Within` holds it to the service's own time limit. `Database.Single` hands it the query and returns `Result<T?, DatabaseError>`: the tile's bytes, `null` for an empty tile, or the reason. `DatabaseError.Of`, a pure function, sorts the failure into one of four kinds (chapter 2). The seed's `SeedScripts.Run` catches Npgsql's exceptions for its own one command in the same way; both live in the Infrastructure, the one layer that may use Npgsql (chapter 3).

Records never change

A ***record*** groups named values into one, like a row of a database table. `Layer` describes one table the service may publish, in one line of parameters:

```csharp
public sealed record Layer(
    string Id, SqlName Table, SqlName Geometry, int Srid, string GeometryType,
    ImmutableArray<SqlName> Properties, ImmutableArray<SqlName> Order);
```

A positional record's properties take a value once, when the record is made, and never again. To 'change' one, a `with` expression makes a new record: `layer with { Srid = 3857 }` leaves `layer` as it was. The lists inside are `ImmutableArray`s, and the catalogue, `LayerCatalogue`, keeps every layer in an `ImmutableDictionary`: collections that offer no way to add, remove or replace an item. Many requests read the same list of layers at once, and none can spoil it for the rest. When `LiveCatalogue` rereads the list, it swaps in a whole new one in a single step (chapter 6).

Closed records list every case

`DatabaseError` names its four cases in one attribute, `[Closed]`, from the ExhaustiveMatching library: `MissingTable`, `MissingColumn`, `Unavailable` and `Other`, each a record that holds PostgreSQL's code and message. C# 14 cannot close a family of types itself, so the library's ***analyser***, a check that runs inside the compiler, does it. `TileServer.Answer` turns each case into an answer:

```csharp
error => error switch
{
    DatabaseError.MissingTable _ => Problems.NotFound("No published layer has that name."),
    DatabaseError.MissingColumn _ => Problems.Unavailable("The layer has changed. …"),
    DatabaseError.Unavailable _ => Problems.Unavailable("The database cannot be reached. …"),
    DatabaseError.Other _ => Fault(found, tile, error),
    _ => throw ExhaustiveMatch.Failed(error),
}
```

The last real case, `Other`, is the service's own fault. Its 500 names no cause, so a stranger learns nothing of the service's insides, and `Fault` writes the cause to the log instead, at error level: the layer, the tile, PostgreSQL's code and its message. `TileTests.Logs_the_cause_of_a_500` checks that line. The underscore, `_`, names a value the code never reads. Add a fifth kind of error to `[Closed]` and the analyser stops the build until this `switch` handles it; chapter 5 lists the answers.

Pure functions make the core easy to test

A pure function needs only its inputs, so a test calls it with values and checks the answer: no server, no database and no clock. `TileTests` checks addresses against the grid in rows such as `[InlineData(1, 2, 0)]`: at level 1 the grid has two columns, 0 and 1, so column 2 names no tile. Beside the rows, a ***property test***, `Gives_only_addresses_on_the_grid`, hands `TileAddress.Of` a hundred sets of made-up numbers from FsCheck, a library that invents inputs, and checks that every address it gives lies on the grid. FsCheck shrinks any failing input to the smallest that still fails. Chapter 9 meets two more property tests, which guard the SQL.

The same file tests the route itself without a database. `Answers_503_for_a_dropped_column` builds a `TileServer` whose reader is a lambda that fails with code 42703, and checks for 503.

> TRY IT In `tests/Tessera.UnitTests/TileTests.cs`, add `[InlineData(22, 4194304, 0)]` and `[InlineData(22, 0, 4194304)]` to `Refuses_a_tile_outside_the_grid`, then run `dotnet run --project tests/Tessera.UnitTests`. Why is 4,194,303 the last column at level 22?

> COMMON MISTAKE Hiding an effect inside a decision. A lambda inside `Map` that reads `DateTime.Now` gives different answers on different days, and its tests need a clock. Read the clock at the edge and pass the time in as a value.

Check your understanding

1. What does `ConnectionString.FromUrl` return when the URL is empty?
2. What is the difference between `Map` and `Bind`?
3. Why does `LayerCatalogue` keep its layers in an `ImmutableDictionary`?
4. When does the arm `_ => throw ExhaustiveMatch.Failed(error)` run?

Answers. 1. A failure holding the reason 'missing or empty'. 2. `Map` applies a function that cannot fail; `Bind` applies one that returns a `Result` of its own. Both pass a failure on unchanged. 3. Many requests read the same list at once, and none may change it under the others. 4. Never, while the arms above it handle all four cases, which the analyser checks.

> THINK IT OVER A test can hand `TileServer` any reader it likes, but only the integration tests meet a real PostgreSQL. What does a test with a stand-in reader prove about a tile, and what can it never prove?

02
C# and Its Types

C# is Tessera's language because its compiler checks every value against its type before the program runs, and the build stops at every warning, from a forgotten `null` to a missed case. Chapter 1 set out the design; this chapter shows how the types hold the code to it, and where they stop.

The compiler checks the whole program before it runs

Three properties let C# find mistakes before any user meets them:

- **Compiled.** A ***compiler*** translates the whole program before it runs and refuses a program with errors. `dotnet build`, the command that `npm run build` runs, compiles the service into `Tessera.dll`.
- **Statically typed.** Every variable has a type fixed in the source code; put text where a number belongs and the build fails.
- **High-level.** You write `if`, `switch` and named variables, not processor instructions. The .NET runtime, the program that carries out compiled C#, runs the result on Windows and Linux alike.

Tessera uses C# 14 on .NET 10, a release with long-term support, for the service and its tests: `Directory.Build.props` sets `LangVersion` 14 and `net10.0` for every project, and `global.json` fixes the tools at version 10.0.100. xUnit, a library for tests, runs them. Three other languages each do one job: SQL, kept in one file, `Queries.cs`, selects the places inside one tile; TypeScript draws the map in the browser and runs the scripts; Python cleans the places before they reach the database.

> FOUNDATIONS High-level languages and translators; data types (integer, real, Boolean, character and string) and casting; syntax errors and logic errors. C#'s `int`, `double`, `bool`, `char` and `string` match these types one for one. A compiler finds syntax errors and type errors; no compiler finds a logic error.

Every value has a type, and the compiler checks every use

A ***type*** states which values a variable may hold and what you may do with them. C# sorts its types into two families, as Exhibit 2.1 shows.

EXHIBIT 2.1 C#'s two families of types, with examples from Tessera

FAMILY	THE VARIABLE HOLDS	EXAMPLES IN TESSERA
Value types	The value itself	`int Z, X, Y` in `TileAddress`; `double` longitudes in `AreaSettings`; `bool ShowPreview`; `ImmutableArray<SqlName>`
Reference types	An address that leads to an object	`string`; arrays such as `byte[]`, a tile's bytes; records such as `TileAddress` and `Layer`

Source: `Domain/Tiles.cs`, `Domain/Settings.cs`, `Domain/Layers.cs`, `Features/Settings.cs`

Copy a ***value type*** and you copy the value. Copy a ***reference type*** and both variables lead to the same object. Sharing is safe only when the object never changes, which is why Tessera's records are immutable. A tile arrives from the database as a `byte[]`, an array that can change, so the record that sends it, `TileResult`, holds it as a `ReadOnlyMemory<byte>`: a view of the bytes through which no code can write.

The right number type stops a silent fault

An `int` holds 32 bits and stops at 2,147,483,647; a `long` holds 64 bits. At zoom level *z*, the number that says how far the map is zoomed in, the grid cuts the world into 2^z by 2^z tiles, and `TileAddress.Of` works out 2^z with a shift: `1 << z` moves the bit 1 left by *z* places.

CSHARP

```
public sealed record TileAddress
{
    public int Z { get; }
    public int X { get; }
    public int Y { get; }

    private TileAddress(int z, int x, int y) => (Z, X, Y) = (z, x, y);

    public static TileAddress? Of(int z, int x, int y, int minZoom = 0, int maxZoom = 22) =>
        z < minZoom || z > maxZoom || z > 30 || x < 0 || y < 0 || x >= 1 << z || y >= 1 << z
            ? null
            : new TileAddress(z, x, y);
}
```

The service serves levels 0 to 22, and `Of` refuses any level above 30 whatever the settings say, so `1 << 30`, 1,073,741,824, is the largest shift the code makes, and it fits an `int`. So Tessera can store *x* and *y* as `int`, the type PostGIS's `ST_TileEnvelope` takes. The order of the checks matters too: `||` stops at the first true part, so the shift never runs with a level above 30. Write `1 << 31` and the result ***overflows*** to −2,147,483,648; write `1L << 31`, with `L` for `long`, and you get 2,147,483,648. An overflow compiles, runs and gives the wrong answer: a ***logic error***, which no compiler reports.

The route guards the other end. Its pattern, `/{layer}/{z:int}/{x:int}/{y:int}.pbf`, asks ASP.NET Core to read each number as an `int` before the route may answer (chapter 5 calls this a route constraint). A part that cannot be read as an `int`, such as `x` or `9999999999` (larger than the largest `int`), matches no route, and the request ends in a 404 problem document before it reaches `TileServer`.

A ***cast*** converts a value from one type to another. `Queries.Tile` writes `(double)limits.Buffer / limits.Resolution`: without the cast, `256 / 4096` would be integer division, and give 0, not 0.0625.

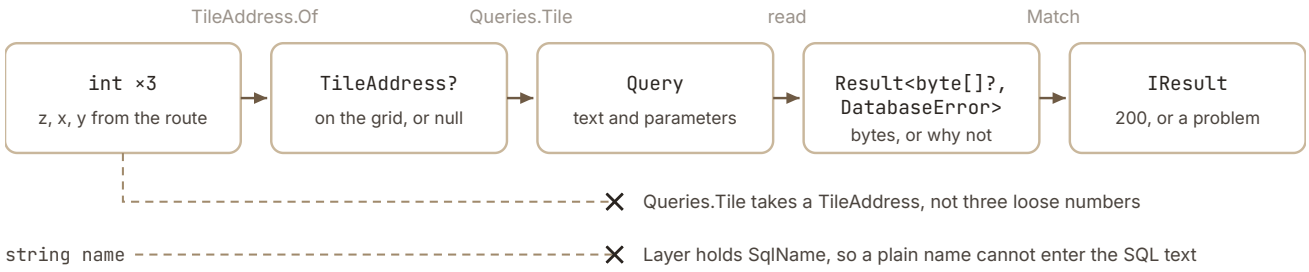
Records name the parts of a value, and compare by value

A record holds named properties, and C# writes `Equals`, `GetHashCode` and `ToString` for it. Two records with the same values count as equal, so `TileAddress.Of(5, 31, 31) == TileAddress.Of(5, 31, 31)` is true, though each call makes a new object. Tessera's domain records each name one idea. `TileAddress` is one tile's place in the grid; `Layer`, one table the service may publish; `LayerCatalogue`, every layer by id; `SqlName`, a quoted name; `TileSettings` and `AreaSettings`, the limits read from `tessera.json`. Named properties stop a caller from swapping *x* and *y* by mistake, which three loose `int` arguments would allow.

`TileAddress` and `SqlName` go one step further: each is a ***smart constructor***. Its own constructor is `private`, so only its static methods can make one, and they make only good values. `new TileAddress(31, -1, 7)` fails to compile outside the record, and its properties have `get` alone, so a

`with` expression cannot copy an address off the grid either. In the same way, only `SqlName.Quote` and `SqlName.Qualified` make a `SqlName`, doubling any quote inside, and a name reaches the text of a query only as a `SqlName`; chapter 9 shows the attack this stops. Exhibit 2.2 follows the types through one request.

EXHIBIT 2.2 Each step accepts only the type before it, so a raw value cannot reach the database; values travel beside the SQL as named parameters



Source: `Features/Tiles.cs`, `Infrastructure/Sql/Queries.cs`, `Domain/SqlName.cs`

Nullable reference types say a value may be missing

A function that may find nothing returns a ***nullable type***, written with a question mark, so its caller must say what happens when the value is missing. `Directory.Build.props` switches the check on, `<Nullable>enable</Nullable>`, for every project. The compiler then refuses code that would put `null` in a `Layer`; a `Layer?` may hold one, and the compiler refuses to use it until the code has checked it. `LayerCatalogue.Find` returns one: `public Layer? Find(string id) => Layers.GetValueOrDefault(id);`. `TileAddress.Of` returns a `TileAddress?`, and `LiveCatalogue.Current` a `LayerCatalogue?`, `null` until the first list loads. Chapter 1's `is not { } found` is the check: it tests for a value and names it in one step.

CSharpFunctionalExtensions also offers `Maybe<T>`, a box for a value that may be missing, and ADR 0004 weighs it and sets it aside. It repeats the compiler's own check, and needs converting wherever a library hands over a `null`, as Dapper, the library that reads rows into records (chapter 7), does for 'no row'. So in Tessera's code `T?` is the one way to say that a value is missing, and `Result<T, E>` the one way to say that a step failed, and why.

Closed records and pattern matching make every switch complete

`Result<T, E>` is a ***generic type***: it takes other types as parameters, in angle brackets, so one design serves every step. A `Result<ConnectionString, string>` cannot stand where a `Result<LayerCatalogue, DatabaseError>` belongs; the compiler refuses it.

A `switch` expression chooses a result by ***pattern matching***: each arm tests the value's shape, not one exact value. Tessera uses several kinds of pattern. `Program.cs` matches the command line as a list, `[]` for no arguments and `["seed"]` for one; `Database.Error` tests a type, `PostgresException refusal`, and a property, `{ InnerException: TimeoutException }`.

A `switch` over a value with endless cases needs a plain `_` arm. `DatabaseError.Of` sorts PostgreSQL's five-character code, a `string`, and no compiler can list every string:

```csharp
public static DatabaseError Of(string sqlState, string message) => sqlState switch
{
    PostgresErrorCodes.InsufficientPrivilege or PostgresErrorCodes.UndefinedTable
        => new MissingTable(sqlState, message),
    PostgresErrorCodes.UndefinedColumn => new MissingColumn(sqlState, message),
    PostgresErrorCodes.QueryCanceled or PostgresErrorCodes.AdminShutdown // … three more
        => new Unavailable(sqlState, message),
    _ when sqlState.StartsWith("08", StringComparison.Ordinal)
        => new Unavailable(sqlState, message),
    _ => new Other(sqlState, message),
};
```

Npgsql names each code, so `UndefinedTable` stands for `42P01`, and `or` joins two patterns in one arm. The `when` clause, a ***guard***, adds a condition to an arm: every code that starts `08` is a failed connection. The code sorts each failure once, where it starts. Every later `switch` works on the four closed kinds, ends with `_ => throw ExhaustiveMatch.Failed(error)`, and lets the ExhaustiveMatching analyser check each one.

The build turns every warning into an error

A few lines of `Directory.Build.props` make the compiler strict. `TreatWarningsAsErrors` makes every warning stop the build, so none piles up unread, and a forgotten `null` check fails at once. `AnalysisMode` `All` at `AnalysisLevel` `latest` switches on every rule .NET's own analysers know; Meziantou's analyser adds more, such as naming how two strings compare. `EnforceCodeStyleInBuild` runs the style rules in every build, among them IDE0130, which fails a file whose namespace strays from its folder (chapter 3). The root `.editorconfig` switches off eleven rules for every file, and two more for `TesseraHost.cs`, each with a comment that says why.

Where C#'s types stop

A type guards the code that goes through it, never the code that goes round it. The nullable check runs at compile time alone. A library can still hand over a `null` when the program runs, and the `!` operator tells the compiler to trust the code: the tests write `TileAddress.Of(15, 26719, 14311)!`, and if that address were off the grid, the test would fail with an exception, not a compiler error.

An `ImmutableArray` is a value type, so it has a default value that holds no array, and reading that value throws an exception. `Queries.Tile` checks `layer.Order.IsDefaultOrEmpty`, not `IsEmpty`, for that reason. `[Closed]` names the cases of `DatabaseError`, but a sixth record could inherit from it unnamed, and the analyser would not see it; `DatabaseErrorTests.Lists_every_kind_as_a_case` fails if the two lists disagree.

`Query`, the SQL text with its parameters, is a positional record, so its constructor is public: `new Query("DROP TABLE food.cafes")` compiles in any file that may use the Infrastructure. The rule that every SQL text lives in `Queries.cs` (S-01 c2) rests on review, and a test pins the tile query's text to an approved copy. The project puts its strongest guard, a private constructor, on `SqlName`, where a slip would let SQL in, and a cheaper one where the risk is smaller.

> TRY IT .NET 10 runs a single C# file as a program. In an empty folder outside the repository, write `shift.cs` holding `int z = 30;` and four lines that print `1 << z`, `1 << (z + 1)`, `1L << (z + 1)` and `1 << (z + 2)`, then run `dotnet run shift.cs`. Explain why the second is negative and the fourth is 1. (C# uses only the lowest five bits of an `int` shift's distance.) Then add `var bad = new TileAddress(31, -1, 7);` to a test in `TileTests`, run `dotnet build`, and read the error.

> COMMON MISTAKE Ending a `switch` over `DatabaseError` with `_ => Problems.Fault()`, or passing everything as a `string`. Both silence the compiler: the analyser checks only a `switch` whose last arm throws `ExhaustiveMatch.Failed`, so the first lets a new case slip through; the second lets a layer's name, a query and a visitor's text stand in for one another.

Check your understanding

1. Why can Tessera store *x* and *y* as `int`, not `long`?
2. What is the difference between a value type and a reference type?
3. Why does `DatabaseError.Of` end with `_ => new Other(…)`, while the `switch` in `TileServer.Answer` ends with a `throw`?
4. What does the compiler refuse once `Nullable` is enabled, and why does `Find` return `Layer?`?

Answers. 1. `Of` refuses any level above 30, so $2^{30} - 1$, the largest column or row, fits in 32 bits. 2. A value type holds the value itself; a reference type holds an address that leads to an object, which a copy then shares. 3. A `string` has endless values, so any unknown code is a real case, `Other`; `DatabaseError` has four, which the analyser knows, so the last arm should never run. 4. Any use of a possibly `null` value without a check; the catalogue may hold no layer of that name, and the type makes the caller decide what then.

> THINK IT OVER Each new type costs a few lines and a name to learn. `SqlName` and `TileAddress` earn private constructors; `Query` keeps a public one and relies on review. How would you decide which values deserve a type of their own, and which types deserve a guard?

03
Modular Monolith

Tessera is one program with firm walls inside: it ships as one release, yet each part can change without breaking another. Engineers call this arrangement a modular monolith. In Tessera, folders name the parts, the build makes every namespace match its folder, three layers divide the one module, and six ArchUnitNET rules fail the tests the moment code climbs a wall.

Architecture decides how a program's parts may talk

Every architecture answers one question: how do you split a program so that a person can understand and change one part without breaking another? A few hundred lines fit in one head; many thousands do not. Three styles give three answers.

A ***monolith*** is one program built and shipped as one unit. ***Microservices*** split the work into separate programs that talk over the network. A ***modular monolith*** keeps the single program but divides its inside into parts with fixed borders. A school shares one building and one fire alarm, yet a science teacher who needs paper asks the English office and leaves its store alone: the rule, more than the wall, keeps order.

One project holds Common and a module of three layers

The whole service is one C# project, `src/Tessera`, laid out as the modular-monolith template that ADR 0010 adopts. A ***module*** is a part of the program that owns one job and shows the rest of the program a small public face. In Tessera each module is a folder under `Modules`, and each folder is a ***namespace***: a named group of types, such as `Modules.Tessera.Domain`. `Tessera.csproj` leaves its root namespace empty, so a folder's path is its namespace, and rule IDE0130, at error level in `src/Tessera/.editorconfig`, fails the build when a file claims any other.

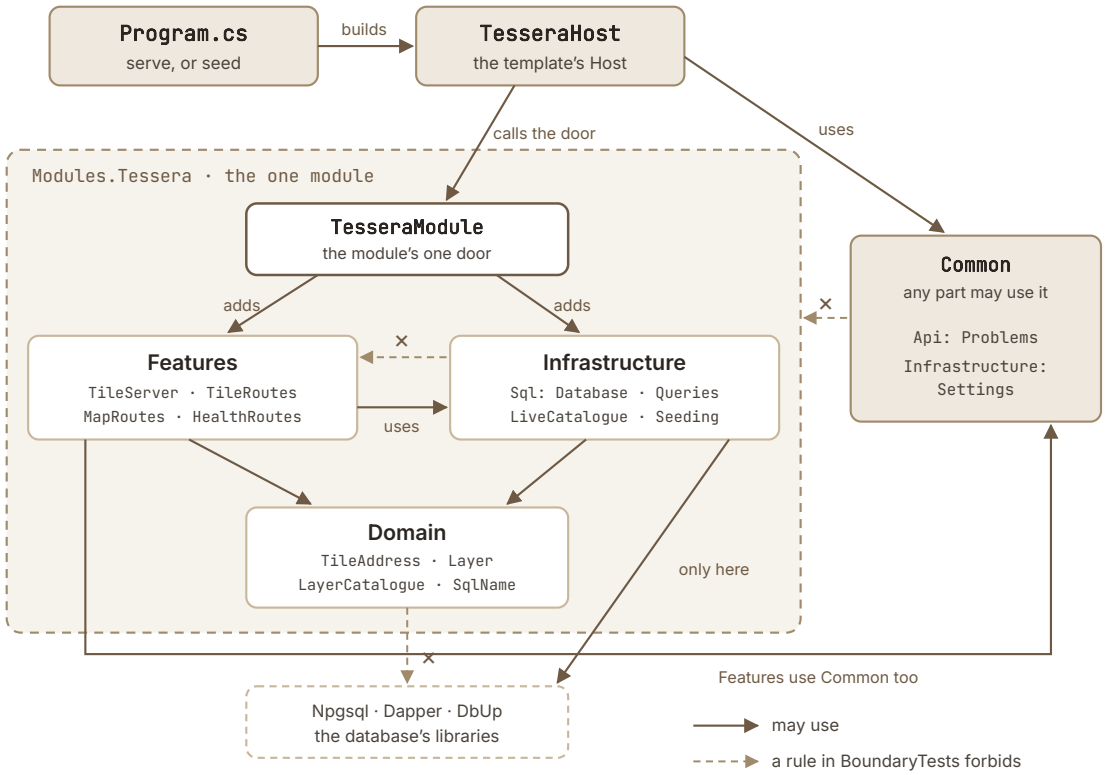
```text
src/Tessera/
  Program.cs              serve, or seed
  TesseraHost.cs          builds the web service from Common and each module's door
  Common/
    Api/Problems.cs       the shape of every error the service answers
    Api/Targets.cs        refuses an address that hides what it asks for
    Infrastructure/       Settings.cs: reads and checks tessera.json
  Modules/Tessera/
    TesseraModule.cs      the module's one door: AddTessera and MapTessera
    Domain/               records and pure functions
    Infrastructure/       the database, every SQL text, the seed
    Features/             the routes: tiles, the map page, /health
```

- `Common` holds what every module may use: `Problems`, the shape of every error; `Targets`, which refuses a disguised address before any route sees it; and `Settings`, which binds each section of `tessera.json` to its record and checks it at start-up. It serves every module and reaches none.
- `Modules.Tessera` holds the Tessera module itself: the list of layers, the tiles, the map page and the health checks. It is the one module so far.

Inside the module, three ***layers*** divide the code by the kind of work it does. A layer is a band of code that may call the bands beneath it, never the bands above. The lines between them follow ADR 0004's functional style: pure rules in the middle, effects at the edge.

- `Domain` holds records and pure functions: `TileAddress`, `Layer`, `LayerCatalogue`, `SqlName`, and the tile and area settings. It uses .NET alone.
- `Infrastructure` talks to the database: every SQL text (`Sql/Queries`), the one class that sends it (`Sql/Database`), the catalogue and its live copy, the two health checks, and the seed (`Seeding/SeedScripts`).
- `Features` answers the web: the tile route (`TileServer` and `TileRoutes`), the map page (`MapRoutes`) and `/health` (`HealthRoutes`).

EXHIBIT 3.1 Who may use whom: solid arrows are allowed, crossed arrows fail a test



Source: `src/Tessera`, the `using` lines of each file; `tests/Tessera.ArchitectureTests/BoundaryTests.cs`

As Exhibit 3.1 shows, every allowed arrow points one way, down or to the right. `Features` uses `Infrastructure`, `Domain` and `Common`; `Infrastructure` uses `Domain` and the database's libraries; the Domain uses no other part of Tessera. The tile query shows the layers at work: `Queries.Tile`, in the Infrastructure, takes a `Layer` and a `TileAddress` from the Domain, and `TileServer`, in `Features`, sends it through `Database`.

> FOUNDATIONS Decomposition and abstraction. Each module and each layer owns one smaller part of the problem, which engineers call high ***cohesion***. Each link between parts adds ***coupling***. So every type a developer adds to `Common`, which all modules may use, ties the modules closer. A module's public face is an abstraction: other parts see what it does, never how.

The module has one door, and the host builds the program from doors

ASP.NET Core's minimal APIs find nothing by searching: a route exists only when code adds it. So one class lists everything the module adds. `TesseraHost.Build`, the program's ***host***, builds the web service in two halves. Before `builder.Build()` it shapes every problem document with `Problems.Shape` and calls `builder.AddTessera()`; after it, it checks every setting, lays the steps every request passes, such as `UseExceptionHandler`, and calls `app.MapTessera()`.

Those two calls are the module's door. `TesseraModule` names every part the module serves: its settings, the pool of connections, the live list of layers, the two health checks, and the three groups of routes. Chapter 5 shows how ASP.NET Core builds each part and hands it to whatever needs it.

CSHARP

```csharp
// src/Tessera/Modules/Tessera/TesseraModule.cs
public static IHostApplicationBuilder AddTessera(this IHostApplicationBuilder builder)
{
    AddSettings(builder.Services);
    // … the database URL, one Database, one LiveCatalogue and its background loop
    builder.Services.AddHealthChecks()
        .AddCheck<DatabaseAnswers>("database")
        .AddCheck<CatalogueReady>("catalogue");
    return builder;
}

public static WebApplication MapTessera(this WebApplication app)
{
    var catalogue = app.Services.GetRequiredService<LiveCatalogue>();
    app.MapHealth();
    app.MapTiles(
        catalogue,
        app.Services.GetRequiredService<Database>(),
        app.Services.GetRequiredService<IOptions<TileSettings>>().Value);
    app.UseMap(/* … the area and map settings, and the folder of the built map */);
    return app;
}
```

A reader finds every part in one list. The seed, the run that fills the database with places, never opens the door: `Program.cs` matches `["seed"]`, reads one setting, the publisher's database URL, hands it to `SeedScripts.Run` and stops, so no web server starts. One cost, which ADR 0001 records, is that each story begins with a choice of module and layer; another is that a new route needs a line in the door.

> COMMON MISTAKE Writing a new route's `MapSomething` method in `Features` and never calling it from `MapTessera`. The build passes and every test of the method passes, yet the route answers 404, since nothing added it to the program. Add the call to the door, and test the route through real HTTP, as `ServiceTests` does.

Architecture tests fail the build when code breaks a wall

An ***architecture test*** checks a program's structure, not its behaviour, and Tessera runs six of them in `tests/Tessera.ArchitectureTests/BoundaryTests.cs`. One project builds one assembly, and C#'s `internal` hides a type only from other assemblies, so every type in `src/Tessera` can see every other: the compiler draws no wall between the folders. The tests close that gap. They use ArchUnitNET, a library that reads the

built program and states rules about its types in C# that reads like English. Each failure names every type that broke the rule, such as `Modules.Tessera.Domain.Bad does depend on "Npgsql.NpgsqlConnection"`.

The tests load the program together with the libraries it uses, since a rule sees only the types it has loaded: without Npgsql loaded, the program's use of it would pass unseen. The first rule reads as a sentence: our types in the Domain should not depend on the module's workings, nor on the database's libraries, nor on ASP.NET Core.

CSHARP

```csharp
// Rule 1: the Domain holds records and pure functions, so it needs no database,
// no web, and no other layer of its module (ADR 0004).
[Fact]
public void Domain_stands_alone() =>
    OursIn("Modules.Tessera.Domain").Should().NotDependOnAny(Workings)
        .AndShould().NotDependOnAny(Database).AndShould().NotDependOnAny(Web)
        .Check(Program);
```

EXHIBIT 3.2 Six rules, six walls

TEST	WALL IT GUARDS	STATE
`Domain_stands_alone`	The Domain uses no database library, no ASP.NET Core, and no Infrastructure or Features	Checks today
`Only_Infrastructure_talks_to_the_database`	No type outside the Infrastructure uses Npgsql, Dapper or DbUp (S-01)	Checks today
`Infrastructure_knows_no_web`	The Infrastructure uses no ASP.NET Core and no Features	Checks today
`Common_never_reaches_a_module`	`Common` serves every module and uses none	Checks today
`Other_modules_reach_Tessera_through_PublicApi_alone`	Another module reaches Tessera through its `PublicApi` folder alone	Waits for a second module
`PublicApi_shows_no_workings`	The public face uses no Infrastructure or Features	Waits for `PublicApi`'s first type

Source: `tests/Tessera.ArchitectureTests/BoundaryTests.cs`*; ADR 0010, 'The rules, and what guards each'*

Exhibit 3.2 lists the six. The last two cover no type yet, and ArchUnitNET fails a rule that covers nothing, so each carries `WithoutRequiringPositiveResults()`: it passes now and bites the day a second module or a `PublicApi` type appears. `dotnet build` alone runs none of these tests; `npm test` and the GitHub build run them before the service's other suites, since they finish in seconds and a broken wall is a fault of design. ADR 0010 counts two more rules. The compiler guards rule 7, a namespace that matches its folder, through IDE0130, and the rules above trust it. Rule 8 rests on review: every error takes its shape from `Common/Api/Problems.cs`, and parts pass values, never shared state.

> TRY IT Break a wall on purpose. In `Domain/SqlName.cs`, add `using Npgsql;` at the top and, inside the record, `public static NpgsqlConnectionStringBuilder Builder(string text) => new(text);`. Run `dotnet build`: the compiler accepts it, since one project may use any library it references. Then run `dotnet run --project tests/Tessera.ArchitectureTests`. One change breaks two walls: which two tests fail, and which type does each name? Undo the change and run the tests again.

Two records explain the choice and when to undo it

ADR 0001 chose one program with walls, and ADR 0010 chose how to lay it out and guard it. Each architecture decision record states a decision, its facts, the options set aside and the cost. One person runs Tessera on one machine, so separate services would add releases and network calls for no gain. A project for each layer would let the compiler guard the borders, but the maintainer finds such project files hard to follow, and the borders they guard are few; six rules in one test project guard them more cheaply. The walls mark where the program would split, if the tiles one day need a machine of their own. A second module joins as a folder under `Modules`, and the rules already cover it. Should ArchUnitNET ever miss a breach, ADR 0010 says the layers become projects.

Check your understanding

1. What is the difference between a monolith and a modular monolith?
2. Why can C#'s `internal` not keep the Domain's types out of `Features`, and what closes the gap?
3. A type in `Domain` uses `NpgsqlConnection`. Which tests fail?
4. Why do the last two rules call `WithoutRequiringPositiveResults()`?

Answers. 1. Both run as one program; the modular one divides its inside into parts with fixed borders, and tests keep them. 2. The whole service is one assembly, and `internal` hides a type only from other assemblies; the six ArchUnitNET rules close the gap, trusting namespaces that IDE0130 keeps true to their folders. 3. `Domain_stands_alone` and `Only_Infrastructure_talks_to_the_database`. 4. No type falls under them yet, and ArchUnitNET would fail a rule that covers nothing; they wait for a second module and a `PublicApi`.

> THINK IT OVER Suppose a search module joins Tessera, and it needs the list of layers. Which folders would it add, what would Tessera place in `PublicApi`, which rule would start to bite, and what two calls would `TesseraHost` gain?

04

Nginx as a Reverse Proxy

Nginx guards Tessera's front door: it answers the office alone, caps what each machine may ask, shares the work between two copies of the service, keeps tiles, and still answers when the service is down. Nginx (said 'engine-x') is a free web server and proxy. Tessera runs release 1.29 in Docker, a tool that runs each program in a sealed box called a ***container***. Every request an office machine sends reaches Nginx first, so the tour of a request begins here.

A reverse proxy acts for the servers behind it

A ***proxy*** acts on someone else's behalf. A ***forward proxy*** acts for clients: a school's web filter gets pages for pupils and blocks some. A ***reverse proxy*** acts for servers: clients believe they talk to the server, yet they talk to the proxy, which passes each request inside. A reverse proxy often does four jobs: it guards, forwards, keeps answers, and answers for a server that is down. ADR 0008 hands Nginx all four, so the service keeps no code for them.

Nginx is the only door from the office

The office reaches Tessera on port 80, and Nginx alone listens there. The service runs in Ubuntu 24.04 inside WSL2, the Linux that Windows can run, on one Windows 11 machine. WSL2's mirrored networking shares the PC's network with Ubuntu (mini-guide M4), and `npm start` runs Nginx in Docker with `--network host`, so Nginx sees each office machine's own address. Behind Nginx, two copies of the service listen on 127.0.0.1, which no other machine can reach. Nginx speaks plain HTTP, with no encryption; chapter 9 names that among the known gaps.

> FOUNDATIONS IP addresses, ports, subnets and binary. A ***port*** number picks one program on a machine, as a flat number picks one door in a block of flats. An office network such as `192.168.10.0/24` is a ***subnet***: the `/24` says that the first 24 bits of every address in it are the same, so the network holds $2^8 = 256$ addresses. Nginx decides whether a visitor belongs to the office by comparing those 24 bits, the same masking you practise with binary AND.

Nginx reads its file in nested blocks

An Nginx file looks dense until you see its shape: a few blocks, each nested in the one before, and each answering one question. Tessera's whole file is `nginx/default.conf.template`, 188 lines, most of them comments.

EXHIBIT 4.1 The blocks of the file, and the question each one answers

BLOCK	THE QUESTION IT ANSWERS	IN TESSERA
Top level	What does every server share?	The tile store, `geo $office`, the visitor's key, the two limit zones and the check for `%2e`

`upstream`	Where may requests go?	`tessera`: the two copies of the service
`server`	Where does traffic arrive, and what applies to all of it?	Port 80, the office rule, the cap on requests at once, gzip, the timeouts
`location`	What happens to this kind of address?	Tiles (`.pbf`, `.mvt`) are rated and kept; everything else passes straight on
`location @name`	What does Nginx answer by itself?	`@forbidden` (403), `@busy` (429), `@unavailable` (503), `@unknown` (404); their kin, `location = /malformed` (400), is marked `internal`, so only Nginx may reach it

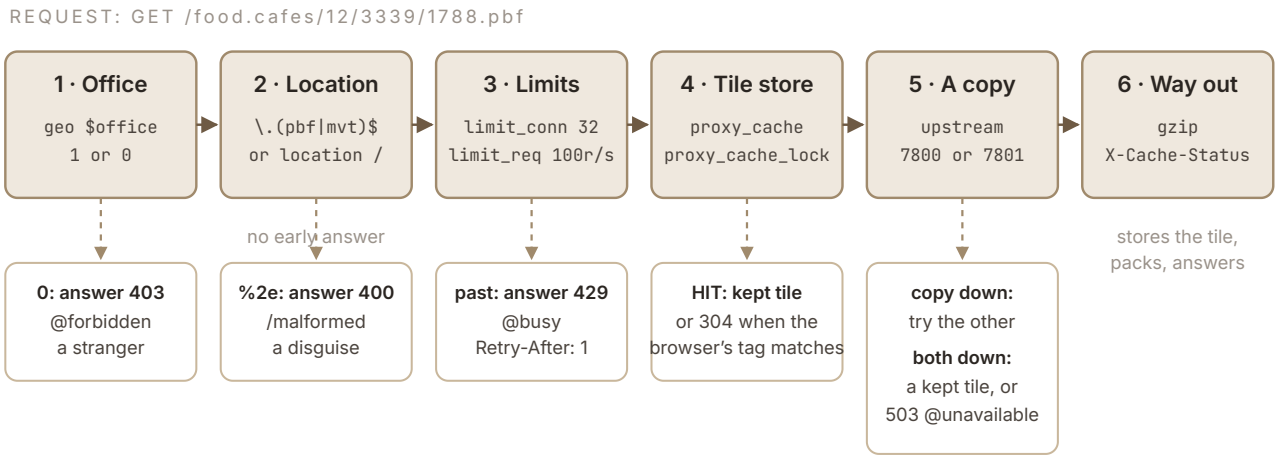
Source: `nginx/default.conf.template`

The file is a template. `npm start` mounts it into the Nginx image, which fills in each `${…}` as it starts, from settings passed in from outside: the port, the office's network and the two copies' ports. A change to the file needs no new image, only a restart of the container.

One request passes through Nginx in stages

Nginx handles each request in a fixed order of stages, and each stage may answer at once instead of passing the request on. Exhibit 4.2 follows one tile request from an office machine.

EXHIBIT 4.2 One tile request through Nginx, and where each stage may answer early



Source: `nginx/default.conf.template`*; the order of stages is Nginx's own*

1. Answer the office alone

The `geo` block turns the visitor's address into a flag, and the server block refuses every stranger before any other rule runs, so a stranger gets no kept tile either (S-07). The machine itself, 127.0.0.1, counts as the office, so `npm start`'s health check gets through. A range too wide would defeat the rule, so `npm start` refuses a prefix below 8, such as `0.0.0.0/0`, which lets in every address.

```nginx
geo $office {
    default 0;
    127.0.0.1 1;
    ${TESSERA_OFFICE_NETWORK} 1;
}
# … in the server block:
if ($office = 0) { return 403; }    # three lines in the file
error_page 403 = @forbidden;
```

The location stage refuses one more kind of request at once: an address in disguise (S-06). `%2e` is a dot written as its number, and no layer, page or file of the map holds one, so a `map` on `$request_uri`, the address exactly as the browser sent it, flags any `%2e` in any case, and both locations that pass requests on answer 400:

```nginx
map $request_uri $disguised {
    default 0;
    ~*%2e 1;
}
# … in the tile location and in location /:
if ($disguised) {
    return 400;
}
```

`error_page 400 /malformed` turns that 400, like every other, into a problem document (chapter 9). The check sits in the two locations, not in the server block, since `error_page` sends the 400 on to `/malformed` with the same `$request_uri`, where a check for the whole server would refuse it again.

2. Cap each machine, by its own address

Two limits, both counted for each address, keep one busy machine from slowing the rest (S-30). A ***rate limit*** caps requests over time; a connection limit caps how many requests one machine may have open at once.

```nginx
map $remote_addr $visitor {
    127.0.0.1 "";
    default $binary_remote_addr;
}
limit_req_zone $visitor zone=visitor:10m rate=100r/s;
limit_conn_zone $visitor zone=connections:10m;
# … in the server block:
limit_conn connections 32;
limit_conn_status 429;    # and limit_req_status 429
error_page 429 = @busy;
# … in the tile location:
limit_req zone=visitor burst=300 nodelay;
```

Each machine may send a burst of 300 tile requests, which `nodelay` serves at once, and 100 a second after that; chapter 12 works out the arithmetic. The numbers fit the map: one fresh view of nine layers asks for about a hundred tiles at once (ADR 0007). The limit follows the address, so a machine gains nothing by opening more connections. The `map` gives the machine itself, 127.0.0.1, an empty key, and Nginx counts nothing for an empty key. Every refusal ends in `@busy`, which answers 429 with `Retry-After: 1` and a problem document, the same JSON shape the service gives its own errors (chapter 5).

3. Share the work, and step round a copy that fails

The `upstream` block names the two copies, and Nginx hands them requests in turn (N-29). Nginx's free edition watches a copy only through the requests it passes on, a ***passive health check***: three failures within ten seconds (`max_fails=3 fail_timeout=10s`) rest the copy for ten seconds. Nginx waits 1 second for a copy to accept a connection and 20 seconds for its answer. With `proxy_next_upstream error timeout`, it tries the other copy when either wait runs out. `keepalive 16` keeps up to 16 idle connections to the copies open in each Nginx worker, so a tile skips the work of opening a new one.

Only `error` and `timeout` count as failures: a 503 from a database outage reaches both copies alike, so it never rests a healthy one. The read timeout, 20 seconds, outlasts the service's own worst case, 3 + 10 = 13 seconds (chapters 6 and 12), so the service answers its own 503 first; the other 7 seconds leave room for a busy machine.

4. Keep tiles, and serve them while the service is down

Nginx keeps each tile in a store on disk, for as long as the service's `Cache-Control` header allows: `public, max-age=60` by default. The store holds 64 MB, and a tile no one asks for in ten minutes leaves it.

NGINX

```
proxy_cache_path /var/cache/nginx/tiles levels=1:2 keys_zone=tiles:10m max_size=64m
                 inactive=10m use_temp_path=off;    # one line in the file
# … in the tile location:
proxy_cache tiles;
proxy_cache_lock on;
proxy_cache_lock_timeout 20s;
proxy_cache_use_stale error timeout updating http_500 http_502 http_503 http_504;
proxy_cache_background_update on;
proxy_ignore_client_abort on;
add_header X-Cache-Status $upstream_cache_status always;
```

`proxy_cache_lock` lets the first request for a new tile fetch it while the others wait, up to 20 seconds, so twenty people who ask at once cost one query (N-24; chapter 11 shows the gain). `proxy_cache_use_stale` serves a kept tile, even an expired one, when a copy fails or the database is down. `proxy_cache_background_update` then fetches the replacement behind it. `proxy_ignore_client_abort` lets a tile finish when the browser that asked for it leaves, since others may be waiting for it; chapter 6 shows what that means for the service. `X-Cache-Status` says where the tile came from: `HIT`, `MISS` or `STALE` (N-19).

With both copies down and no kept tile, `error_page 502 504 = @unavailable` turns the failure into a 503 problem document with `Retry-After: 5`, the same answer the service gives when its database is down (N-22, N-28).

Smaller answers, and no body when nothing changed

Two more jobs cut the bytes that cross the network. ***gzip*** packs tiles, problem documents and the map's scripts before they leave (N-20). `gzip_comp_level 1` picks the fastest level; the template's own comment says a tile shrinks nearly as much as at level 9, for a fraction of the work. `gzip_vary on` adds `Vary: Accept-Encoding`, so no cache along the way hands a packed copy to a client that cannot unpack it.

The second job needs no line of the template. The service sends each tile with an ***ETag***, a short tag that names its exact contents: `TileServer.Tag` takes the first 16 bytes of the tile's SHA-256 hash, a fingerprint computed from its bytes. The tag is weak (`W/"…"`) because the packed and the plain bytes are the same

tile. A browser that already holds the tile asks again with the tag in `If-None-Match`, a ***conditional request***. When the tag matches the tile Nginx keeps, Nginx answers ***304 Not Modified*** with no body, and the browser uses its own copy. ASP.NET Core answers the same way when a request reaches the service (ADR 0008).

Nginx and the service each own their rules

A rule written in two places will one day disagree with itself, so each rule has one owner. ADR 0007 keeps every limit in Nginx, and every library for retries and limits out of the service. ASP.NET Core has rate limiters of its own, but two copies of the service, each counting on its own, would let each visitor through twice. Nginx owns the office rule, both limits, the tile store, sharing the copies, gzip and malformed addresses. The service owns how long a tile may be kept, its `ETag`, `/health`, the time limits on each query, and its own problem documents, such as the 503 it sends when the database is down. One rule stands in both places on purpose: the service refuses a dot folder in disguise too, for a request that reaches a copy directly, past Nginx (chapter 9).

The edge checks prove each job

Every job in the template has a check that runs against a real Nginx. `npm run e2e` builds the image from `nginx/Dockerfile`, starts it with the same template and the office set to `127.0.0.4/30`, runs the browser tests through it, then calls `checkEdge` in `scripts/lib/edge.ts`. Each check plays an office machine or a stranger by sending from a chosen address on the machine itself. As Exhibit 4.3 shows, the checks also prove what Nginx must not do: after the flood, another office machine still gets its tile at once.

EXHIBIT 4.3 What `checkEdge` sends, and what it expects back

JOB	THE CHECK SENDS	IT EXPECTS
Kept tiles, 304 (N-19)	One tile twice, then its weak tag in `If-None-Match`	`X-Cache-Status: HIT` and the same bytes; then 304 with no body
gzip (N-20)	The tile and the map's main script, packed and plain	`Vary: Accept-Encoding`; the script under half its size
HEAD	The tile, asked with `HEAD`	200 with the headers alone
Malformed addresses (S-06)	Twelve raw addresses, such as `/x/%2e%2e/food.cafes/0/0/0.pbf` and `/food.cafes/15/1/2.pbf%00`; then `/malformed` by name	The exact status, 400 or 404, as a problem document that never repeats the layer's name; then 404
Office alone (S-07)	A tile from 127.0.0.2, then from 127.0.0.5	403 for the stranger, 200 for the office machine
Both limits (S-30)	600 tiles over eight connections; then 40 at once for a slow tile	At least 300 tiles, then 429 with `Retry-After: 1`, while another office machine still gets 200; at least 8 of the 40 refused
One query for a crowd (N-24)	20 requests at once for a tile that takes two seconds	All 200, within 1.8 to 4 seconds
Copies fail (N-29)	Six new tiles with one copy stopped; then two tiles with both stopped	All 200 and `/health` still `UP`; then the kept tile, and 503 for the other

Source: `scripts/lib/edge.ts`; `scripts/e2e.ts`

> TRY IT With `npm start` running, ask for one tile twice from the machine itself and read the headers: `curl -s -o /dev/null -D - http://127.0.0.1/food.cafes/12/3339/1788.pbf`. Note `X-Cache-Status` and `ETag` each time. Then send the tag back with `-H 'If-None-Match: …'` and read the status. Why does the machine itself never meet the rate limit, however fast you ask?

> COMMON MISTAKE Listing one copy in the `upstream` block and trusting `max_fails` to guard it. With a single server in the group, Nginx ignores both `max_fails` and `fail_timeout`: there is no other copy to send the request to. That is why `npm start` runs two copies of `Tessera.dll`, on ports 7800 and 7801, and starts again any copy that stops (ADR 0008).

Check your understanding

1. Name the five kinds of block in `nginx/default.conf.template`, and say what each one decides.
2. Why does a request from 127.0.0.1 meet neither limit?
3. Why does `proxy_next_upstream` list `error` and `timeout`, but not `http_503`?
4. Twenty people ask for one new tile at the same moment. How many queries reach the database, and which line makes it so?

Answers. 1. Top level: what every server shares; `upstream`: where requests may go; `server`: where traffic arrives and what applies to all of it; `location`: what happens to one kind of address; named `location @…`: what Nginx answers by itself. 2. The `map` gives it an empty key, and Nginx counts nothing for an empty key. 3. A database outage gives both copies the same 503, so counting it would rest healthy copies; `proxy_cache_use_stale` serves kept tiles instead. 4. One; `proxy_cache_lock on`.

> THINK IT OVER ADR 0007 keeps every limit in Nginx because one Nginx sees every request. Suppose the office one day ran a second Nginx on a second machine. What would happen to each visitor's limit of 100 tiles a second, and where would you keep the count then?

05
ASP.NET Core

ASP.NET Core passes every request through a short line of steps, then lets one method answer it with an honest status code. It is Tessera's ***framework***, the ready-made code that runs the program and calls Tessera's own code at set points: it starts the service, reads and checks its settings, runs the Kestrel web server and hands each request to the right part of the code.

A request travels over HTTP and through a line of steps

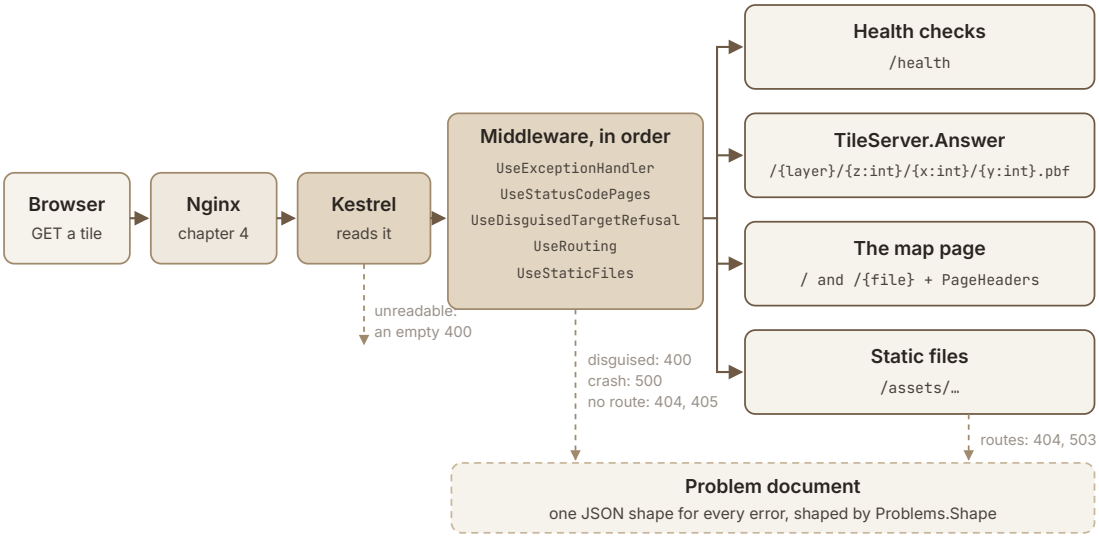
The web follows the ***client–server model***. A client, such as a browser, sends a request; a server sends back a response. Both speak ***HTTP***, the Hypertext Transfer Protocol. A request names a method and a path:

```text
GET /food.cafes/12/3339/1788.pbf HTTP/1.1
```

`GET` means 'send me this'. The path asks for the layer `food.cafes` at zoom 12, column 3339, row 1788: the square of Macau that holds the map's home point, this guide's example tile. The response opens with a ***status code***, such as 200 for success or 404 when nothing lives at that path.

Inside ASP.NET Core, the web server ***Kestrel*** accepts the connection and reads the request. The request then passes a line of ***middleware***: small steps, each of which may read it, change the response or answer at once. Last, it reaches an ***endpoint***, the one method chosen to answer it. Picture security at an airport: each desk checks one thing, and only a passenger who clears every desk reaches the gate. Tessera keeps its line short, because Nginx, in front, already refuses a stranger or a visitor who asks too often (chapter 4).

EXHIBIT 5.1 One request through ASP.NET Core's parts, and where each kind of answer comes from



Source: `TesseraHost.cs`, `TesseraModule.cs`, `Features/Tiles.cs`, `Features/Map.cs`, `Features/Health.cs`, `Common/Api/Problems.cs`

As Exhibit 5.1 shows, the line has five steps and four ends. Routing picks the endpoint early and runs it at the end of the line, so the two error steps wrap every request, whatever answers it. Before routing, `UseDisguisedTargetRefusal` reads the address as the browser sent it and answers 400 to one that hides a folder of dots as `%2e` (chapter 9). The static files step serves a file only when routing found no endpoint. Any error on the way becomes a problem document, described below; only a request Kestrel cannot read gets an empty 400, before any step sees it.

> FOUNDATIONS Client–server networks, protocols such as HTTP and TCP/IP, and web servers. A URL's path, like a subroutine's parameters, carries the details of what you want; a status code, like a return value, says how it went.

Program.cs starts the program, and TesseraHost wires its parts

The program has one entry point, `Program.cs`, and two commands, chosen by a `switch` on its arguments. With `seed`, it runs the seed scripts with DbUp and stops (chapter 7). With no arguments, it builds the service with `TesseraHost.Build` and runs it. Any other word prints the two commands and ends with exit code 2.

```csharp
return args switch
{
    [] => await Serve().ConfigureAwait(false),
    ["seed"] => Seed(),
    _ => Usage(),
};
```

`TesseraHost.Build` starts from `WebApplication.CreateBuilder`, ASP.NET Core's standard start-up, and adds the module with one call, `builder.AddTessera()`. Kestrel listens on the address in the `Urls` setting, `http://127.0.0.1:7800` in `tessera.json`, so only programs on the same machine, Nginx among them, can reach it; `npm start` gives the second copy 7801.

Dependency injection then builds each part the program needs and hands it to whatever asks for its type. The program registers each part as a ***service*** with a lifetime; Tessera's own parts are ***singletons***, built once for the whole program. `TesseraModule.AddTessera` lists them by hand:

EXHIBIT 5.2 What `AddTessera` registers, and what each holds

REGISTRATION	WHAT IT HOLDS
Five settings sections, and the database URL	`Database`, `Tiles`, `Map`, `Catalogue`, `Area`, each bound to its record; `DATABASE_URL`, parsed into a connection string
`AddSingleton(… Database.Open(…))`	One `Database`, so one pool of connections that every request shares (chapter 6)
`AddSingleton(… new LiveCatalogue(…))`	One list of layers, which every request reads
`AddHostedService(… LiveCatalogue)`	The same object again, run as a loop in the background that reloads the list
`AddHealthChecks()`	Two checks, `DatabaseAnswers` and `CatalogueReady`

Source: `Modules/Tessera/TesseraModule.cs`; `TesseraHost.Build` *adds* `AddProblemDetails`

The tile endpoint asks for nothing by type. `MapTiles` builds one `TileServer` and hands it two functions, one that gives the current list of layers and one that runs the tile query, and the tile's settings. A test hands it stand-ins in their place, such as a database on a port where nothing answers.

Three sources give the settings, and start-up checks them all

Tessera reads each setting from three places, each overriding the one before. Each settings record carries a default for every key. `tessera.json`, beside the program, overrides those defaults. Last come ***environment variables***: `Tiles__CacheSeconds=0` overrides `Tiles.CacheSeconds`, with `__` for each step down. `Settings.AddFile` adds the environment again after the file, so the environment always wins:

```csharp
configuration
    .AddJsonFile(Path.Combine(AppContext.BaseDirectory, "tessera.json"), optional: true)
    .AddEnvironmentVariables();
```

ASP.NET Core then copies each section into a record, and checks it, through the ***options pattern***: its way of handing settings to the code as typed records. Each limit is an attribute on the property; a rule that ties two values together lives in `Validate`:

```csharp
public sealed record TileSettings : IValidatableObject
{
    // … Resolution, Buffer, MaxFeatures
    [Range(0, 30)] public int MinZoom { get; init; }
    [Range(0, 30)] public int MaxZoom { get; init; } = 22;
    [Range(0, int.MaxValue)] public int CacheSeconds { get; init; } = 60;

    public IEnumerable<ValidationResult> Validate(ValidationContext validationContext) =>
        MaxZoom >= MinZoom ? [] : [new ValidationResult("MaxZoom must not be below MinZoom.")];
}
```

`Settings.Section` binds each record and checks it in three steps. First it reads each key of the section and sets aside any the record lacks, most often a misspelling such as `Tiles.CacheSecond`, and any value of the wrong kind, such as a word where a number belongs; the binder sees only the keys that read cleanly. A check of its own, `KeyCheck`, then names each key it set aside, with its section. `ValidateDataAnnotations` runs the attributes and `Validate`. `ValidateOnStart` runs every check before the service answers a single request; `TesseraHost.Build` calls `Settings.Check` at once, after `builder.Build()`. On any fault the program prints each problem on a line of its own and ends with exit code 1, so no request ever meets a bad value. A misspelt section name, though, passes unseen: its keys bind to nothing, and the section keeps its defaults. `SettingsTests` proves each of these, from `Refuses_a_key_it_does_not_know` to `Leaves_a_misspelt_section_at_its_defaults`.

```text
$ Tiles__CacheSecond=5 Tiles__CacheSeconds=-1 Tiles__MaxZoom=soon dotnet Tessera.dll
DataAnnotation validation failed for 'TileSettings' members: 'CacheSeconds' with the
  error: 'The field CacheSeconds must be between 0 and 2147483647.'.
Tiles: 'CacheSecond' is not a setting; Tiles holds Resolution, Buffer, MaxFeatures,
  MinZoom, MaxZoom, CacheSeconds.
Tiles: MaxZoom must be a whole number, not 'soon'.
DataAnnotation validation failed for 'DatabaseConnection' members: '' with the error:
  'DATABASE_URL: missing or empty'.
```

All four problems arrive in one run, as US-00 asks, the misspelt key among them. .NET's messages for a value out of its limits name the record; the program's own, for a key or a value it cannot read, name the section. .NET's binder could refuse an unknown key itself, but it stops at the first, so one slip would hide every other. `Names_a_misspelt_key_with_every_other_problem`, in both `SettingsTests` and `StartTests`, proves that it hides none.

Endpoints map a path to a method

Tessera uses ***minimal APIs***: each endpoint is one line that joins a path to a method, with no controller class around it. `TileRoutes.MapTiles` adds both tile endings, `.pbf` and `.mvt`, for `GET` and `HEAD`:

```csharp
string[] read = [HttpMethods.Get, HttpMethods.Head];
routes.MapMethods("/{layer}/{z:int}/{x:int}/{y:int}.pbf", read, server.Answer);
routes.MapMethods("/{layer}/{z:int}/{x:int}/{y:int}.mvt", read, server.Answer);
```

The `:int` on each number is a ***route constraint***: `/food.cafes/a/1/2.pbf` matches no route and answers 404. ASP.NET Core fills each parameter of `TileServer.Answer` from the part of the path with the same name, and fills its `CancellationToken` with a signal that fires when the client gives up the request (chapter 6). The method then checks in order: the list of layers has loaded, the layer is published, and `TileAddress.Of` places the address on the grid. Only then does a query run. ASP.NET Core answers 304, 'Not Modified', when a browser's tag still matches the tile's.

The map takes two more paths. `MapRoutes.UseMap` answers `/` and one page for each of the nine layers, such as `/food.cafes.html`; any other name, `/index.html` included, answers 404. The two page endpoints share a group with an ***endpoint filter***, `PageHeaders`, a step that runs around those endpoints and no others: it adds the browser's rules for the page, such as its Content Security Policy, and `Cache-Control: no-cache`. The map's scripts and styles come from the static files step, under `/assets/`, from the `wwwroot` folder beside the program, with `public, max-age=31536000, immutable`: a browser may keep each file for a year, since a new build gives each file a new name. Set `Map.ShowPreview` to false, and the service serves neither the page nor its files.

Status codes tell the truth about every request

Every answer carries the code that matches what happened, and every error has one shape.

EXHIBIT 5.3 The status codes Tessera sends, and when

CODE	MEANING	WHEN TESSERA SENDS IT
200	OK	A tile, even an empty one; the map page; `/health` when UP
304	Not Modified	The browser's tag matches the tile's
400	Bad Request	An address that hides a dot folder as `%2e`, or holds a raw control character; Kestrel's own empty 400 for one it cannot read, such as `%00`. Through Nginx, Nginx refuses such an address first (chapter 9)
404	Not Found	An unknown or withdrawn layer; a tile off the grid; an unknown page or path
405	Method Not Allowed	`POST`, `PUT` or `DELETE` on a tile, a page or `/health`, with `Allow: GET, HEAD`

500	Internal Server Error	The service's own fault; the reply gives no detail
503	Service Unavailable	The list of layers has not loaded; the database cannot answer; a layer changed under a query; `/health` when not UP

Source: `Features/Tiles.cs`, `Common/Api/Problems.cs`, `ServiceTests.cs`; *403 and 429 come from Nginx (chapter 4)*

Every error is a ***problem document***: a short JSON report in the shape that RFC 9457, an internet standard, sets, with four fields at most and nothing from inside the server. An address no route knows gets this one, with no `detail`, as `ServiceTests` checks:

```json
{"type":"about:blank","title":"Not Found","status":404}
```

Three parts share the work. `AddProblemDetails` lets ASP.NET Core write a problem document for any error, and hands each one to `Problems.Shape`, which sets the type to `about:blank`, removes the trace id, which would tell a stranger of the service's insides, adds `Cache-Control: no-store`, and adds `Retry-After: 5` to every 503 (N-22). `UseExceptionHandler` catches any crash and answers 500 with no detail; the log keeps the cause. `UseStatusCodePages` fills any empty error, such as the 404 for a path no route knows or a 405, with a problem document. The routes themselves answer through `Problems.NotFound`, `Problems.Unavailable` and `Problems.Fault`, which may add a fourth field, `detail`, such as 'No published layer has that name.'

Health checks say when the service can work

A ***health check*** is an address a watcher asks to learn whether the service can do its job. Tessera registers two, `DatabaseAnswers` and `CatalogueReady`, at `/health`:

```csharp
await database.Ping(cancellationToken).ConfigureAwait(false)
    ? HealthCheckResult.Healthy() : HealthCheckResult.Unhealthy()
catalogue.Current is null ? HealthCheckResult.Degraded() : HealthCheckResult.Healthy()
```

The worse of the two decides. `HealthRoutes.MapHealth` writes the answer in the words of Spring Boot's Actuator: `{"status":"UP"}` comes with 200; `OUT_OF_SERVICE`, before the first list of layers loads, and `DOWN`, when the database fails its ping, both come with 503. Every answer carries `Cache-Control: no-store`, so no cache keeps a stale 'UP'.

> TRY IT Start the built program with a misspelt key and a value out of range: `Tiles__CacheSecond=5 Tiles__MaxZoom=31 dotnet src/Tessera/bin/Release/net10.0/Tessera.dll`; read the messages, one for each problem, and the exit code that `echo $?` prints. Then restart `npm start` as `Tiles__CacheSeconds=0 npm start` and ask one copy, past Nginx: `curl -sI http://127.0.0.1:7800/food.cafes/12/3339/1788.pbf`. The `Cache-Control` header now reads `no-cache`.

> COMMON MISTAKE Answering 200 with an error message inside. Browsers, Nginx and caches read the status first, and keep a failure dressed as success. Return the matching code through `Problems`.

Check your understanding

1. What do the method, the path and the status code of an HTTP exchange each tell you?
2. An operator sets `Tiles.MaxZoom` to 3 and `Tiles.MinZoom` to 5. What happens, and when?
3. A browser asks for `/food.cafes/3/9/2.pbf`. Which status code should it receive, and why?
4. What is the difference between `DOWN` and `OUT_OF_SERVICE`?

Answers. 1. What to do, which resource, and how it went. 2. The program refuses to start, prints 'MaxZoom must not be below MinZoom.' with any other problem it finds, and ends with exit code 1, so no request meets the fault. 3. 404: at zoom 3 the grid has 8 columns, so `x` must lie between 0 and 7, and no tile has those coordinates. 4. `DOWN` means the database fails its ping; `OUT_OF_SERVICE` means the database may answer, but the first list of layers has not loaded yet.

> THINK IT OVER Every step in the line adds a moment to every request. How would you decide whether a new check belongs in Nginx, in middleware, in an endpoint filter or in the endpoint itself?

06

.NET Concurrency

Tessera serves many tile requests at once on a few processor cores, because a waiting request gives its thread back, no request shares data that can change, and a small pool queues in order what the database cannot yet take. C# and .NET 10 supply a tool for each. A time limit stops any wait that runs too long, and a cancel signal ends a wait that no one needs.

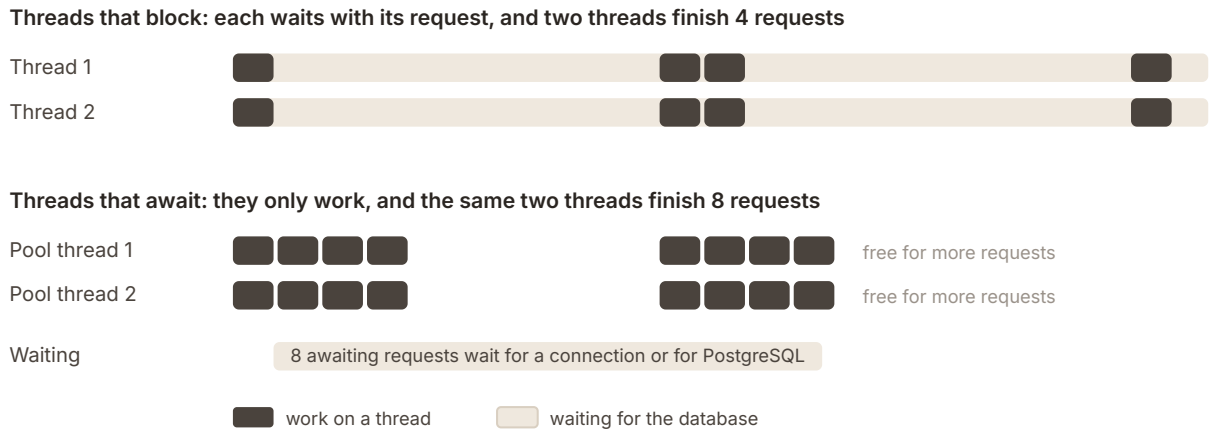
Concurrency lets one server handle many requests at once

Concurrency means that many tasks are in progress over the same period. ***Parallelism*** means that tasks run at the same instant, on different processor cores. A café with one barista shows the first: while the milk steams for one order, the barista takes the next. A second barista, working beside the first, shows the second.

A ***thread*** is one sequence of steps that a program runs in order. Each thread of the operating system reserves memory for its stack, the memory that holds its method calls, and the system spends work each time it switches from one thread to another, so a server keeps few. .NET keeps them in the ***thread pool***: a set of threads that take the next piece of waiting work, finish it, and come back for more. The pool starts with about one thread for each core, and adds more slowly when work queues up. Kestrel runs each request's code on a thread from this pool.

One view of the map asks for dozens of tiles at once, and each tile request spends most of its life waiting for PostgreSQL. What a thread does while it waits decides how many requests the server can serve.

EXHIBIT 6.1 Two threads, the same time: threads that block against threads that await



An illustration. In Tessera at most four queries a copy run in the database at once (the second hazard below).

FOUNDATIONS The CPU, cores and the operating system. An operating system shares processor time between programs (multitasking), and a processor with several cores runs several instructions at the same instant. A record whose fields never change is like a constant: any number of readers may share it.

Six tools meet six hazards

Each danger of running many requests at once has one answer in the code, and one setting or class that carries it.

EXHIBIT 6.2 The hazards of many requests at once, and Tessera's answer to each

HAZARD	TESSERA'S ANSWER	TOOL
A thread sits idle while the database works	Give the thread back while waiting	`async` and `await`
Too many queries swamp the database	Lend four connections a copy; queue the rest in order	Npgsql's pool, `Database.PoolSize`
A slow query or a long queue runs on	Bound every wait in time, on the service's own clock too	`QueryTimeoutSeconds`, `ConnectionWaitSeconds`, `Within`
A request no one waits for keeps its place	Pass a cancel signal to every wait	`CancellationToken`
Two threads change the same data at once	Share nothing that can change; swap the whole list	Immutable records, `Interlocked`, `Volatile`
A hung database holds `/health`	Wait one second at most	`Within`: a linked token and `WaitAsync`

Source: `tessera.json`, `Infrastructure/Settings.cs`, `Infrastructure/Sql/Database.cs`, `Infrastructure/LiveCatalogue.cs`

1. An awaiting request gives its thread back

Every method the service uses to reach the database while it serves is an ***asynchronous method***: it returns a `Task`, a promise of a result to come, and uses `await` where it must wait. Every query passes through `Database.Run`, whose helper `RunOnce` borrows a connection and runs the work on it:

```csharp
var connection = await _source.OpenConnectionAsync(limit).ConfigureAwait(false);
await using (connection.ConfigureAwait(false))
    return await work(connection, limit).ConfigureAwait(false);
```

At each `await` that must wait, the method stops and hands its thread back to the pool, and the thread serves another request. When the database's answer arrives, any free thread in the pool picks the method up where it stopped. Exhibit 6.1 shows the gain: two threads finish eight requests instead of four. The code still reads from top to bottom, as if it had the machine to itself.

`ConfigureAwait(false)` tells each `await` that the method need not return to the place it started from. ASP.NET Core keeps no such place, so here it changes little, but the build's analysers ask for it on every `await` in the service, and the build fails without it (rule CA2007).

2. A pool of four connections queues the rest, in order

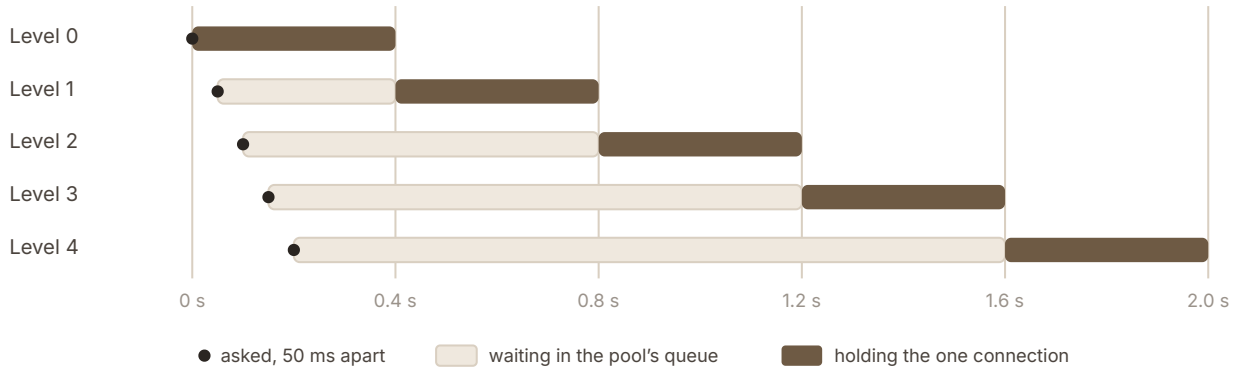
Opening a connection to PostgreSQL takes time, so a ***connection pool*** keeps a few open and lends one to each query. Npgsql, the library that talks to PostgreSQL, keeps the pool, and `tessera.json` sets its limits:

```json
"Database": {
  "PoolSize": 4,
  "ConnectionLifetimeSeconds": 3600,
  "QueryTimeoutSeconds": 10,
  "ConnectionWaitSeconds": 3
}
```

`Database.Open` copies each into Npgsql's own setting: `PoolSize` becomes `MaxPoolSize`, and `ConnectionWaitSeconds` becomes `Timeout`, which covers both opening a connection and waiting for a free one. Only one `Database` exists in the program, a singleton (chapter 5), so each copy of the service holds one pool of four. `npm start` runs two copies, so the database serves eight at most, well within the 20 that `db/Dockerfile` allows (chapter 12 counts the rest). A fifth query in one copy waits for a free connection, up to 3 seconds. Past that, Npgsql gives up with a `TimeoutException` inside its error. `Database.Error` reads that as 57014, PostgreSQL's code for a cancelled query, `DatabaseError.Of` sorts it as `Unavailable`, and the browser receives 503 with `Retry-After: 5` (N-22).

N-26 asks that the queue keep the order of arrival, so that no unlucky request loses its turn again and again. Npgsql lends each freed connection to the request that has waited longest, so the service needs no code of its own for the order, only tests that prove it. Exhibit 6.3 shows the first of them.

EXHIBIT 6.3 Five tiles, one connection: each waits its turn, and they finish in the order asked



Source: `PoolTests.Runs_queries_in_the_order_they_arrive`; a view that sleeps 0.4 seconds, a pool of one connection. Times are the test's design, not a measurement.

THE PROOF `PoolTests` proves N-26 against a real PostgreSQL, with a pool of one connection. `Runs_queries_in_the_order_they_arrive` asks for five different tiles, 50 ms apart, and expects them to finish in the order 0, 1, 2, 3, 4. `Gives_up_the_ping_while_every_connection_is_busy` holds the only connection for three seconds and expects the ping to give up within one and a half, and the next ping to answer once the hold ends. `Lets_a_cancelled_request_leave_the_queue` cancels a waiting query after 200 ms and expects it to leave within a second, long before the pool's own wait of 30 seconds ends. A later Npgsql that changed its order would fail the build.

3. Time limits stop slow work

No query may run for ever. `Database.QueryTimeoutSeconds`, 10, becomes Npgsql's `CommandTimeout`: after ten seconds Npgsql gives up on the query and asks PostgreSQL to cancel it. That failure, too, carries a `TimeoutException`, reads as 57014, and sorts as `Unavailable`, a fault that waiting may cure, so the tile answers 503, not 500. A slow query thus holds a connection for ten seconds at most, and a queued one

waits three seconds at most. `CatalogueTests` proves the first with a view that sleeps three seconds against a limit of one.

Npgsql's limits alone fail when the database freezes instead of refusing. Past the query's limit, Npgsql opens a second connection to ask the server to cancel the query, then waits for an answer that never comes; on 18 October 2026, a database frozen with `docker pause` made a 10-second limit answer after 16 seconds. `Database.Run` therefore keeps a clock of its own, set to the two waits together, 3 + 10 = 13 seconds as shipped, and runs every query through a helper, `Within`:

```csharp
var clock = CancellationTokenSource.CreateLinkedTokenSource(cancel);
clock.CancelAfter(limit);
var run = work(clock.Token);
// … a continuation disposes of the clock once the work ends
return await run.WaitAsync(limit, cancel).ConfigureAwait(false);
```

Two clocks run together. The token, tied to the caller's, fires at the limit, so a request still waiting for a connection leaves the pool's queue. `WaitAsync` throws a `TimeoutException` at the same moment, whatever the work is doing, and `Run` turns it into the same 57014 as any query past its time: a 503. The query left behind finishes, or fails, out of sight. With this clock the frozen database answered in 13.0 to 14.1 seconds. `FrozenDatabaseTests` pauses a real database, with limits of 1 and 2 seconds, and expects `Unavailable` within 3.5.

4. A cancel signal ends a wait no one needs

A ***cancellation token*** is a signal passed down from caller to callee, so the caller can say 'stop' to work it no longer needs. Tessera passes one to every wait: `Database.Command` bundles it with the SQL and its parameters, 'so no query leaves without all three', and `OpenConnectionAsync` takes it too, so a cancelled request leaves the pool's queue at once (N-26). Each kind of work carries the token of whoever asked for it:

- A tile query carries the request's own token. ASP.NET Core fills the `CancellationToken` parameter of `TileServer.Answer` with a signal that fires when the connection to the client closes.
- The list of layers carries the service's stop token, so a loading list stops when the program stops.
- The ping carries a token of its own, tied to its caller's, that also fires after one second (hazard 6).

Behind Nginx, the request's token rarely fires for a tile. The client the service sees is Nginx, and `proxy_ignore_client_abort on` keeps Nginx's connection open when a browser pans away, so the service builds the tile and Nginx keeps it for the next person who asks (chapter 4). The token fires when Nginx itself gives up, after its read timeout of 20 seconds. The service's own clock, 13 seconds, ends first; chapter 12 shows why it must.

5. Data that never changes needs no lock

A ***race condition*** happens when the result depends on which thread moves first. Even `count++` is three steps, read, add and write, so two threads can both read 7 and both write 8. .NET's `Interlocked.Increment` makes the three steps one, which no other thread can split.

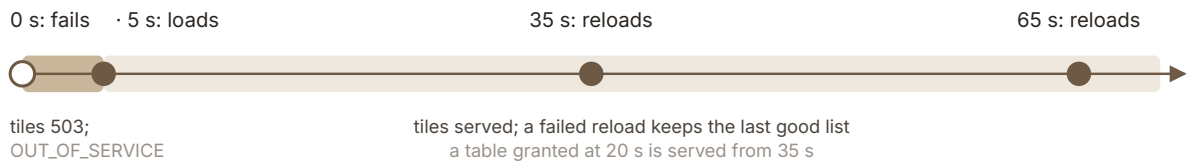
Tessera avoids the problem instead of guarding against it. Its only shared data is the list of layers, and that list is immutable (chapter 1): `LayerCatalogue` holds an `ImmutableDictionary`, and each `Layer` holds its columns in `ImmutableArray`s, so nothing can change them afterwards. A new list never changes the old one; it replaces it. `LiveCatalogue` keeps the current list in one field and swaps it in one ***atomic*** step, a step no other thread can see half done:

```csharp
// Null before the first load.
private LayerCatalogue? _current;
// … a request reads:
public LayerCatalogue? Current => Volatile.Read(ref _current);
// … a successful load swaps:
var old = Interlocked.Exchange(ref _current, loaded.Value);
```

`Volatile.Read` fetches the field afresh each time, never a stale copy a processor kept in its own small, fast memory. A request reads `Current` once and keeps that list to the end; it sees the old list or the new one, never half of each. The reload runs apart from the requests, as a ***hosted service***: `LiveCatalogue` is a `BackgroundService`, a loop that .NET starts and stops with the program. Its `ExecuteAsync` is short:

```csharp
while (!stoppingToken.IsCancellationRequested)
{
    await Refresh(stoppingToken).ConfigureAwait(false);
    if (Current is not null && every <= TimeSpan.Zero)
        return;
    await Task.Delay(Current is null ? retry : every, stoppingToken).ConfigureAwait(false);
}
```

EXHIBIT 6.4 When the list of layers loads: at once, every 5 seconds until it succeeds, then every 30



Source: `LiveCatalogue.ExecuteAsync`; `Catalogue.RefreshSeconds` is 30, and the retry is `Problems.RetryAfterSeconds`, 5. Times count from the end of each load.

As Exhibit 6.4 shows, the loop runs the load at once; it retries every five seconds until a list loads; then it rereads every `Catalogue.RefreshSeconds`, or stops when that is 0. A failed reload keeps the last good list, so a database that blinks for a moment leaves every layer published (N-23). When the program stops, the stop token ends `Task.Delay` at once, and the loop ends with it.

6. The ping keeps its own clock, for one second

`/health` must answer within a second, even when the database hangs. `Database.Ping` therefore runs `SELECT 1` through the same `Within`, on a clock of one second, `PingLimit`:

```csharp
try
{
    await Within(PingLimit, PingOnce, stop).ConfigureAwait(false);
    return true;
}
catch (Exception error)
    when (error is TimeoutException or OperationCanceledException or NpgsqlException
        && !stop.IsCancellationRequested)
{
    return false;
}
```

The two clocks guard the ping as they guard a query. `WaitAsync` stops the wait after one second, whatever the database does. The token takes the ping out of the pool's queue at the same moment, so a ping that gave up never takes a connection a tile is waiting for. The ping borrows from the same pool as the tiles, so it waits in the same queue: when every connection stays busy for more than a second, `/health` says `DOWN`, since no tile could reach a browser in good time either (N-25, N-26).

> TRY IT .NET 10 runs a single C# file with `dotnet run race.cs`. Write one in which two tasks, started with `Task.Run`, each run `count++` a million times on one shared `int`, then print it. Run it three times, then use `Interlocked.Increment(ref count)` instead and compare the answers. Last, start 10,000 tasks that each `await Task.Delay(1000)`, and time the run; then make each call `Thread.Sleep(1000)` instead, and time it again.

> COMMON MISTAKE Catching every `OperationCanceledException` and carrying on. A cancel is a request from the caller to stop, and swallowing it means the caller never learns that its work ended. `Ping` catches a cancel only when the caller's own token has not fired (`!stop.IsCancellationRequested`), so the caller's cancel escapes, and only the ping's own one-second limit counts as 'not well'. `LiveCatalogue` does the same, with `when (stoppingToken.IsCancellationRequested)`.

Check your understanding

1. What is the difference between concurrency and parallelism?
2. Two threads each run `count++` once on a count of 7. What can the count be afterwards, and why?
3. With two copies of the service, how many tile queries can run in the database at once, and what happens to the next one?
4. The database stops answering at start-up. What does `/health` say, and when does the service try again?

Answers. 1. Concurrency keeps many tasks in progress; parallelism runs them at the same instant. 2. 9 or 8: both threads may read 7 before either writes. 3. Eight, four in each copy; the next waits up to 3 seconds for a free connection, then receives 503 with `Retry-After: 5`. 4. 503 with `DOWN`, since the ping fails, and an unhealthy database outranks a list not yet loaded; the hosted service tries the load again every 5 seconds until it succeeds.

> THINK IT OVER A pool of four connections makes a fifth query wait up to three seconds rather than run at once. When is a quick 503 kinder to a user than a long wait, and how would you choose the pool's size and the length of its wait?

07
PostgreSQL

PostgreSQL holds every place Tessera serves, and it alone decides what the map may show: only a grant publishes a table, and only a restore proves a backup. Tessera runs PostgreSQL 18, a free relational database, with PostGIS 3.6 inside it. This chapter shows the image that starts it, the two roles that sign in to it, the libraries that carry the service's questions, the seed that fills it and the backups that keep it.

A relational database keeps tables and answers questions in SQL

A ***relational database*** keeps its data in tables. Each row is one record, such as one café in Macau; each column is one field, such as its name or its address. A ***primary key*** gives every row a value no other row shares: in Tessera, the column `fid`, numbered 1, 2, 3 and so on. You ask questions in SQL, such as `SELECT fid, name FROM food.cafes ORDER BY fid`.

PostgreSQL is a server in its own right: it listens on port 5432, and programs such as Tessera connect to it as clients. Each client signs in under a named ***role***, a user account with its own rights. Tables sit in ***schemas***, which work like the database's folders: Tessera's nine layers live in four of them, `food`, `outdoors`, `culture` and `transport`. Every change runs inside a ***transaction***, which succeeds whole or fails whole, like a bank transfer that never leaves money in mid-air.

> FOUNDATIONS Relational databases: tables, records, fields and primary keys, and SQL's `SELECT`, `FROM`, `WHERE` and `ORDER BY`. The database is also a server, reached over a network like any other.

One image makes every copy of the database alike

The file `db/Dockerfile` builds one database image, and the service, the seed and the tests all start it. A test therefore meets the database the office meets: the test fixture, `DatabaseFixture`, builds the image from `db/` and starts a fresh copy of it with Testcontainers for each class of tests. The image starts from a fixed tag, `postgis/postgis:18-3.6`, so every machine runs the same release until someone changes that line on purpose. It copies the set-up scripts into the folder the image runs on its first start, and changes five settings, each chosen from a measurement or a count (chapter 11). On the office machine, `npm start` runs the image as the container `tessera-db`. It publishes port 5432 on 127.0.0.1 alone, so no other machine can reach the database, and it keeps the data in a Docker volume, `tessera-data`, which outlives the container. One command, `docker volume rm tessera-data`, deletes every place for good; Appendix A marks it 'never run this'.

Two roles split the reading from the writing

The script `db/initdb/20-roles.sql` makes two roles on the database's first start. The tests run the same script, so they sign in as the shipped roles, never as hand-made copies:

```sql
\getenv reader_password TESSERA_READER_PASSWORD
-- …
CREATE ROLE tessera_reader LOGIN PASSWORD :'reader_password';
-- A second lock beside the grants: even a table granted for writing by mistake stays read-only.
ALTER ROLE tessera_reader SET default_transaction_read_only = on;

CREATE ROLE tessera_publisher LOGIN PASSWORD :'publisher_password';
-- …
SELECT format('GRANT CREATE ON DATABASE %I TO tessera_publisher', current_database()) \gexec
```

The service signs in as `tessera_reader`. It may read only what someone has granted to it, and every transaction it opens is read-only; the test `Keeps_the_reader_read_only` checks that setting. The data publisher signs in as `tessera_publisher`, a role that may make schemas: it owns the places' tables, and it alone loads and grants them. Each password comes from the environment, never from the file, and the database sets it once, on its first start; a missing password stops that start with an error that names the variable.

Two libraries carry every question from C# to the database

The service leaves PostgreSQL's network protocol to two libraries. ***Npgsql***, the .NET driver for PostgreSQL, opens the connections and keeps a few of them open in a pool for requests to share. ***Dapper***, a small library on top of it, sends a query with its values and turns each row of the answer into a C# record by matching column names to field names. One class, `Database`, in `Infrastructure/Sql/Database.cs`, holds both. Every query passes through it, and it hands back each failure as a value, a `DatabaseError` sorted by PostgreSQL's five-character error code, so no exception escapes. Every SQL text the service sends lives in one file, `Queries.cs`.

A grant publishes a table, and a revoke withdraws it

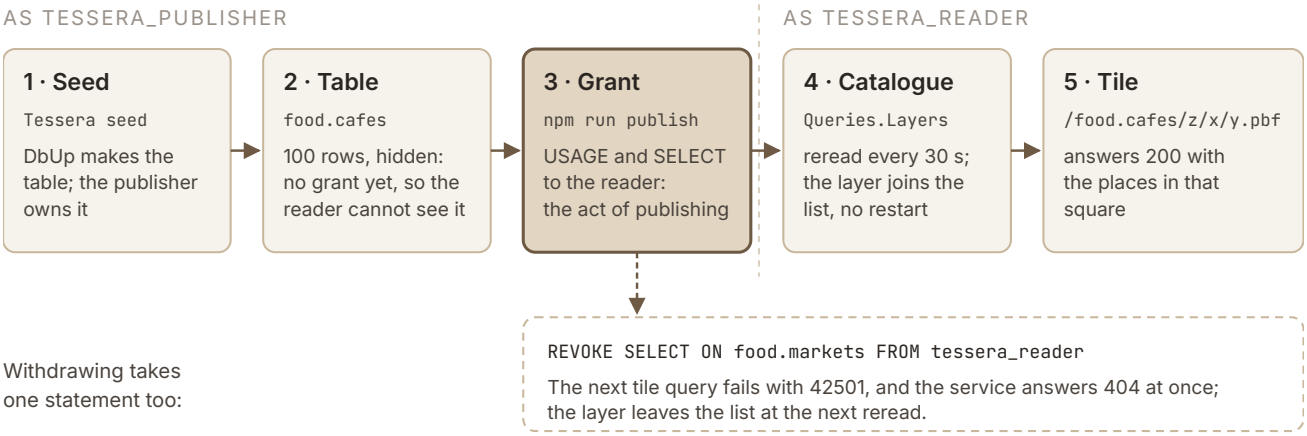
Tessera keeps no list of layers of its own: it asks PostgreSQL which tables its role may read, and publishes exactly those. A ***grant*** gives a role a right, such as the right to read a table, so only a grant publishes a table. The publisher's script, `scripts/publish.ts`, sends two statements as `tessera_publisher`: `GRANT USAGE ON SCHEMA food, outdoors, culture, transport` and `GRANT SELECT ON ALL TABLES IN SCHEMA` the same four, each `TO tessera_reader`.

`USAGE` lets the reader enter each schema; `SELECT` lets it read each table. The seed never grants: publishing stays the publisher's own act, and the test `Publishes_nothing_until_the_grant` proves that the seeded tables stay hidden until the grant.

PostgreSQL describes every table, column and right in tables of its own, the ***system catalogue***. The query `Queries.Layers` reads it as the reader, so the database answers for that role. It lists every table and view that has a geometry column of a fixed type (chapter 8) and passes two checks: `has_schema_privilege(n.oid, 'USAGE')` and `has_table_privilege(c.oid, 'SELECT')`. Dapper fills one `CatalogueRow` from each row of the answer.

These two checks make the grant the whole rule: a table the role may not read never reaches the list. `CatalogueReader.FromRows` then turns each row into a `Layer`, whose id joins the schema and the table name, such as `food.cafes`. It groups the layers by id and keeps only an id that belongs to one layer alone, since a dot may sit inside a name: schema `a.b` with table `c`, and schema `a` with table `b.c`, both give the id `a.b.c`. Serving the wrong table is worse than serving neither, and the test `Publishes_neither_of_two_tables_whose_ids_clash` proves that the service serves neither.

EXHIBIT 7.1 From a seed script to a tile: the grant is the one step that publishes



Sources: `scripts/publish.ts`, `Queries.cs`, `LiveCatalogue.cs`, `Features/Tiles.cs`; *the tests* `Sees_a_grant_and_a_revoke_within_one_refresh` *and* `Answers_404_for_a_table_revoked_after_start_up`

As Exhibit 7.1 shows, a grant reaches the map within 30 seconds, the default of `Catalogue.RefreshSeconds` in `tessera.json`. `LiveCatalogue`, a loop that runs beside the web service, rereads the list on that schedule and swaps the new list in whole. A ***revoke***, which takes a right away, works sooner still: once the grant has gone, the next tile query fails with the error code `42501`, 'insufficient privilege'. `DatabaseError.Of` sorts that code as `MissingTable`, and the tile route, `TileServer.Answer`, turns it into 404, 'not found'.

DbUp runs each seed script once, and never again

The seed fills the database from SQL files that the Python pipeline writes into `db/seed`, such as `0001-macau-places.sql`, which makes nine tables of 100 places each. The build copies them into a folder named `seed` beside the program. The command `Tessera seed` starts the program without its web server and runs ***DbUp***, a .NET library that runs each new script once, in the order of the scripts' names. The seed signs in as `tessera_publisher`, through the variable `TESSERA_PUBLISHER_DATABASE_URL`, and `npm start` runs it at every start as `dotnet src/Tessera/bin/Release/net10.0/Tessera.dll seed`. `SeedScripts.cs` sets DbUp up in one chain:

CSHARP

```csharp
// Infrastructure/Seeding/SeedScripts.cs
var upgrade = DeployChanges.To.PostgresqlDatabase(connection)
    .WithScriptsFromFileSystem(folder)
    .WithTransactionPerScript()
    // DbUp would otherwise read $name$ in a script as a value to fill in, …
    .WithVariablesDisabled()
    .JournalToPostgresqlTable(JournalSchema, JournalTable)   // tessera.schemaversions
    .LogToNowhere()
    .Build();
```

The repository holds one seed script, `0001-macau-places.sql`. A later script, such as `0002-macau-places-cleaned.sql`, appears only when the maintainer runs `npm run etl -- data/macau.pipeline.json` (mini-guide M1), and the repository keeps none. DbUp records each script it runs in its ***journal***, the table `tessera.schemaversions`, in a schema of its own, apart from the places. DbUp makes the table but not the schema, so the seed first sends `Queries.JournalSchema`, a `CREATE SCHEMA IF NOT EXISTS`. Each row of the journal holds a script's name, in the column `scriptname`, and the moment it ran. Each script runs in one

transaction with its journal row, so a script that fails changes nothing, stays out of the journal, and runs again once someone fixes it. The test `Undoes_a_script_that_fails` proves the first two.

DbUp knows a script by its name alone. It keeps no fingerprint of the script's text, so it misses any change to a script it has already run: it skips that script, as it skips every name in its journal, and the change never reaches the database. Once a script has run, its SQL stays fixed, as its first lines warn. New or corrected data goes in a new script, with the next number, and the pipeline refuses to write over a name that exists. The test `Loads_the_shipped_seed_once` seeds twice: the first run loads 900 places and nine spatial indexes, and the second finds nothing new to run.

> TRY IT After `npm start`, list what DbUp has run: `docker exec tessera-db psql --username postgres --dbname tessera --command "SELECT scriptname, applied FROM tessera.schemaversions"`. Then run `\dp food.cafes` in the same way, before and after `npm run publish`, and find the line `tessera_reader=r/tessera_publisher`: the grant, with `r` for read.

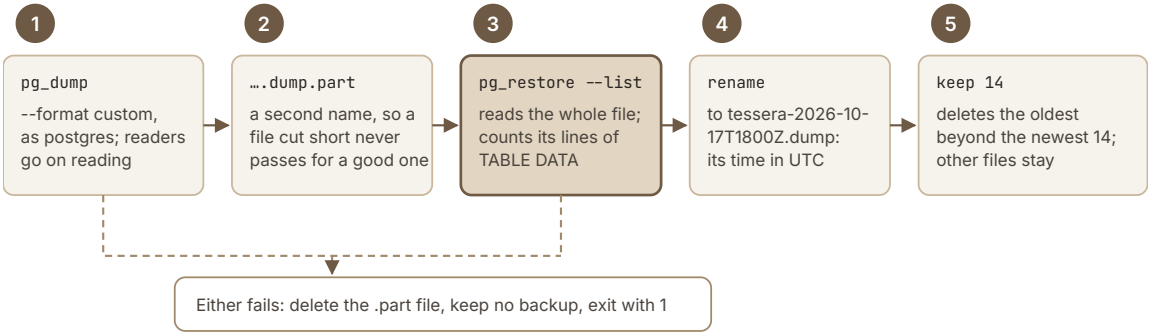
A backup counts only once it has been restored

The database holds the only copy of the published map: every table, every grant and DbUp's journal. If the disk fails, Tessera starts again from the newest backup, or never. A Windows scheduled task runs `scripts/backup.ts` at 02:00 each night; `npm run backup-task` prints the steps that set the task up. The script uses `pg_dump` to take a ***dump***: the whole database, written out as one file. It writes the dump in PostgreSQL's custom format, a compressed file from which `pg_restore` can put back the whole database or a single table. Then it reads the file back before it keeps it, as Exhibit 7.2 shows, so a damaged file fails tonight, not on the day someone needs it.

The backups go to `backups/`, which Git ignores, or to the folder that `TESSERA_BACKUP_FOLDER` names. The script's own notes advise a folder on a second disk, such as `/mnt/d/tessera-backups`, since a backup on the same disk dies with the disk.

The command `npm run restore -- backups/tessera-2026-10-17T1800Z.dump` puts a backup back with `pg_restore`. It replaces each object, never adds to it, and works in one transaction, so a failed restore changes nothing. The backup holds every table, every grant and DbUp's journal, so a later `npm start` runs only the scripts newer than the backup. It leaves out two things: the two roles, which the database makes on its first start, and `.env`, which holds the passwords. Keep a safe copy of `.env` away from the machine.

EXHIBIT 7.2 One night's backup: a file is kept only after it has been read back whole



Source: `scripts/backup.ts`, `scripts/lib/backup.ts`

Reading a file back proves it whole; only a restore proves it useful. Tessera has no runbook yet to ask for a trial restore; study plan 05, in `docs/study`, sets one as a drill on a spare database (back up, change the

data, restore, seed) and then has you write the runbook from it.

> COMMON MISTAKE Correcting a seed script that has already run. Fixing one misspelt name in `0001-macau-places.sql` changes nothing in the database: DbUp finds the script's name in its journal and skips it, without a word. Write the correction as a new script with the next number. A new script that replaces a table drops its grant with it, so run `npm run publish` afterwards; the test `Withdraws_a_replaced_table_until_the_grant` proves the grant goes.

Check your understanding

1. Which role does the service sign in as, and which two locks keep it from writing?
2. The publisher runs `npm run publish`. Within how long does the new layer appear, and why does the service need no restart?
3. Someone corrects one character in a seed script DbUp has already run, then runs `npm start`. What happens to the correction, and why?
4. The backup script reads each file back with `pg_restore --list`. Why does study plan 05 still ask for a restore drill?

Answers. 1. `tessera_reader`: it holds only the rights granted to it, `USAGE` and `SELECT`, and the setting `default_transaction_read_only = on` makes every transaction it opens read-only. 2. Within 30 seconds, `Catalogue.RefreshSeconds`: `LiveCatalogue` rereads `Queries.Layers` on a schedule and swaps in the new list. 3. Nothing: DbUp knows the script by its name, finds that name in `tessera.schemaversions` and skips it, so the correction never reaches the database. 4. Reading the list proves the file whole. Only a restore proves that the tables, the grants and DbUp's journal come back and the map works; a drill on a spare database proves it without risk to the office's own.

> THINK IT OVER `scripts/publish.ts` grants every table in four schemas at once. A new seed script that adds a private table to `food` would be published by the next `npm run publish`. Should the script grant table by table instead, and what would that cost the publisher each time the places change?

08
PostGIS

PostGIS turns PostgreSQL into a spatial database, and one SQL statement built from its functions turns a table of shapes into each tile Tessera serves. It adds types for points, lines and areas, and functions that measure them, compare them and move them between coordinate systems. Tessera runs PostGIS 3.6, and its C# code never handles a single longitude or latitude.

PostGIS keeps shapes in typed geometry columns

PostGIS adds a ***geometry*** type to PostgreSQL. Its kinds cover most of a map: points, line strings, polygons and their multi forms, each written as text such as `POINT(113.539569 22.183748)`. Every one of Tessera's nine layers holds points.

The seed gives each table its geometry column with one call: `SELECT AddGeometryColumn('food','cafes','geom',4326,'POINT',2);`. The call makes a typed column, `geom geometry(Point,4326)`, which holds only points, each of two numbers; the next section explains the 4326. `Queries.Layers` keeps only a column whose coordinate system has a number above 0, with `postgis_typmod_srid(a.atttypmod) > 0`, since an untyped column's numbers have no fixed meaning. The test `Publishes_what_is_granted_and_nothing_else` grants such a table, `stage.untyped`, and finds it missing from the list.

Every shape names the system its numbers belong to

Every geometry carries a ***spatial reference identifier*** (SRID), which says what its numbers mean. Tessera's tables use ***EPSG:4326***, number 4326 in the EPSG list of coordinate systems: longitude and latitude in degrees, longitude first, as a phone's GPS reports them. Web maps draw in ***Web Mercator***, EPSG:3857, which flattens the world into a square measured in metres, 20,037,508.34 m from the centre to each side. The same place has an address in each system, as a house has both a street address and a grid reference. `ST_Transform` converts one to the other. Each `Layer` carries its own SRID, so a table kept in Web Mercator serves equally well: the test `Places_each_feature_where_it_lies` asks for the same café from a table in each system.

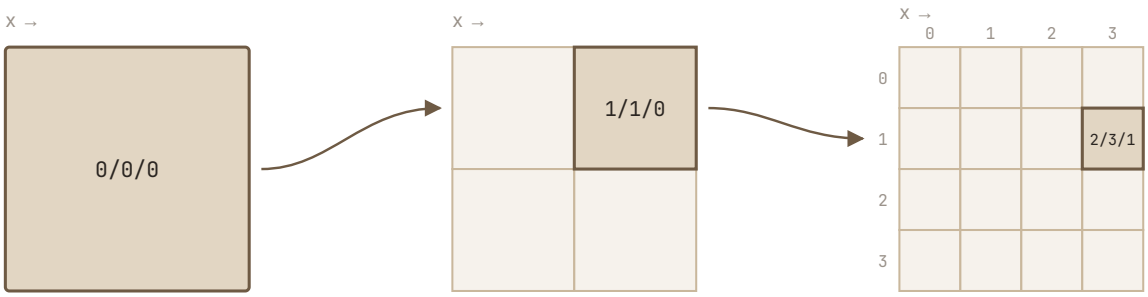
> FOUNDATIONS Coordinates on a graph, powers of two and binary numbers. A bitmap stores pixels; a vector image stores shapes. A vector tile is a vector image of one square of map, so it stays sharp at any size and the browser can restyle it.

The world is cut into a quadtree of tiles

A web map cuts the Web Mercator square into tiles, and each ***zoom level*** cuts every tile of the level above into four. At level 0 one tile covers the world; at level z the world is 2^z tiles wide and 2^z tall, so 4^z in all. This tree, in which each square has four children, is a ***quadtree***. A tile's address is three whole numbers, `z/x/y`: its level, its column counted from the west, and its row counted from the north. `TileAddress.Of` gives back `null` for any address off the grid, where `x` or `y` reaches 2^z or more, or `z` lies outside the levels served, 0

to 22 by default. Its constructor is private, so no other code can make an address, and the property test `Gives_only_addresses_on_the_grid` tries a hundred sets of random numbers to show that every address `Of` makes lies on the grid (chapter 1).

EXHIBIT 8.1 Three levels of the quadtree, and the path down to Macau



Level 0 · 1 tile
the whole Web Mercator square

Level 1 · 4 tiles
Macau lies in the north-east quarter

Level 2 · 16 tiles
column 3, row 1

Ten more splits: level 12 · 4,096 × 4,096 = 16,777,216 tiles · Macau's places fill 12/3339–3340/1788–1789

Source: `Domain/Tiles.cs`; tile numbers checked with `ST_TileEnvelope` in PostGIS 3.6

A tile's number holds the whole path down the tree, in binary. At level 12, the tile on this book's cover holds the map's home point: `12/3339/1788`. In twelve bits, x is `110100001011` and y is `011011111100`. Read the bits in pairs, one from x and one from y, left to right. The first pair, x 1 and y 0, picks the north-east quarter, `1/1/0`. The first two bits of each, 11 and 01, give `2/3/1`, as Exhibit 8.1 shades. Halving a tile's x and y, and dropping the remainder, gives its parent.

Two formulas turn a longitude and a latitude into a column and a row; the test class `ServiceTests` carries them as `TileOf`. Chapter 12 works them for the home point that `tessera.json` names, 113.55° E and 22.19° N, and finds 3339 and 1788. In the seeded data the four tiles `3339-3340` by `1788-1789` hold all 900 places: 359 in the cover's tile, 73 east of it, 98 south of it and 370 to the south-east.

One statement builds each tile

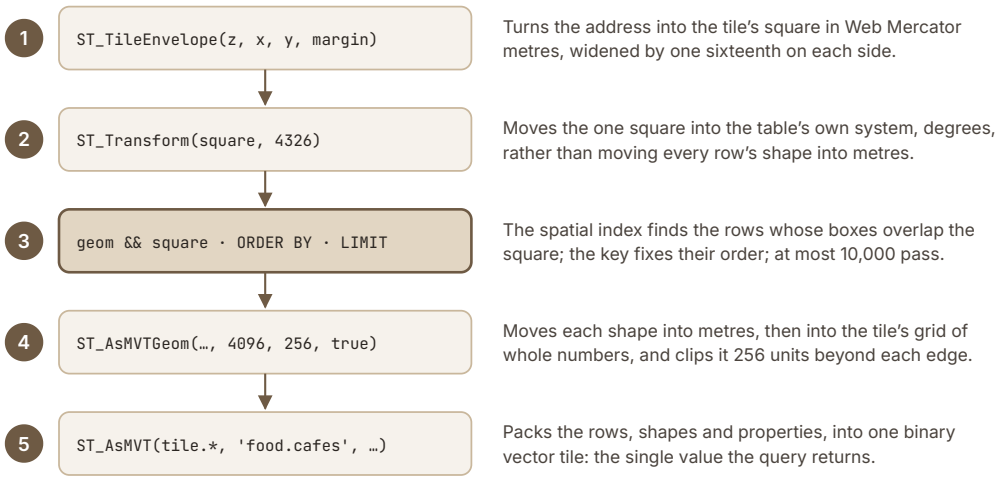
For every request, `Queries.Tile` writes the layer's quoted names into one SQL text and sends the numbers apart from it, as named parameters such as `@z`, which Dapper hands to Npgsql (chapter 9 explains why). For `food.cafes` it reads:

```sql
SELECT ST_AsMVT(tile.*, @layer, @resolution, 'tessera_shape') FROM (
  SELECT ST_AsMVTGeom(ST_Transform(t."geom", 3857), ST_TileEnvelope(@z, @x, @y),
                      @resolution, @buffer, true) AS tessera_shape,
         "fid", "id", "layer", "group", "name"  -- … and 18 more columns
  FROM "food"."cafes" AS t
  WHERE t."geom" && ST_Transform(ST_TileEnvelope(@z, @x, @y, margin => @margin), @srid)
  ORDER BY t."fid"
  LIMIT @limit
) AS tile
```

For the cover's tile the values are `@z` 12, `@x` 3339, `@y` 1788, `@resolution` 4,096, `@buffer` 256, `@margin` 0.0625, `@srid` 4326 and `@limit` 10,000. Most come from the `Tiles` section of `tessera.json`, which fills the record `TileSettings`; the SRID comes from the layer. The buffer is a strip beyond each edge of the tile (next section); the margin is the same strip as a share of the tile, which `Queries.Tile` works out as `Buffer` ÷ `Resolution`: 256 ÷ 4,096, one sixteenth. `ORDER BY` follows the table's primary key, so the same rows

always come out in the same order; a view has no key, and its tile has no order, as `TileTests.Gives_a_view_no_order` checks.

EXHIBIT 8.2 The tile query's five stages, inside out



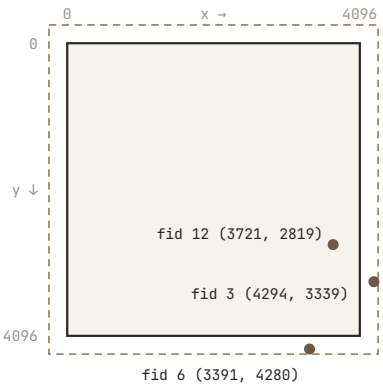
Source: `Infrastructure/Sql/Queries.cs`, `Domain/Settings.cs`

As Exhibit 8.2 shows, the database does every step of the spatial work and returns one value: the finished tile. `Database.Single` reads it as an array of bytes, and `TileServer` sends it with the content type `application/vnd.mapbox-vector-tile`. A square with no places gives an empty tile, and the service still answers 200.

A vector tile holds layers of features in the tile's own grid

A vector tile holds one or more named layers. Each of Tessera's tiles holds one, named after the layer's id, such as `food.cafes`. Each layer holds features, and each feature holds a geometry and its properties. The geometry is written in whole numbers in the tile's own grid, its ***extent***: 0 to 4,096 across, counted from the tile's top-left corner, with y growing downwards. The properties are the table's other columns of simple types, from `fid` to `sample`, which the browser reads to label and style each place. No degree or metre travels over the network: the browser knows each tile's level, column and row, and from them puts every point back on the map, as Exhibit 8.3 shows. The test `Places_each_feature_where_it_lies` does the same in C#: it reads a level-15 tile back with NetTopologySuite and finds the café `cafes-001` within 0.00001° of where the seed put it.

EXHIBIT 8.3 Three cafés of tile 12/3339/1788: one inside the tile, two beyond its edges, inside the buffer



The tile: 0 to 4,096 in each direction, from the top-left corner. Every point becomes two whole numbers, which keeps the tile small.

The buffer: 256 units beyond each edge, dashed. Café 3 stands at 113.5589° E, east of the tile's edge at 113.5547°, so its x, 4294, lies past 4096 but inside 4352. Café 6 lies south of the edge at 22.1874° N. The next tile draws each too, so its symbol shows whole.

The clip: ST_AsMVTGeom cuts away everything beyond the buffer. Macau's peninsula fills the tile's south-east corner; the rest of the tile is water and the mainland, with no places.

Source: the tile query run on the seed in PostGIS 3.6.4; of the seed's 100 cafés, 54 fall inside the tile and its buffer

A spatial index makes each tile fast

Without an index, every tile would read every row to find the few inside one square. A ***spatial index*** of the kind PostGIS calls ***GiST*** stores each shape's ***bounding box***, the smallest rectangle around it, in a tree of ever smaller boxes. The operator `&&` asks whether two boxes overlap. With the index, the database walks down the tree and skips whole branches that lie elsewhere, as a street atlas sends you straight to one page and one square. The seed builds the index for each table with `CREATE INDEX "cafes_geom_geom_idx" ON "food"."cafes" USING GIST ("geom")`, then runs `ANALYZE`, which refreshes the table's statistics.

The index holds boxes in degrees, so the query must compare degrees with degrees. On a table of 100 cafés, PostgreSQL reads every row anyway, since that costs less than asking the index (chapter 11 shows the plan). Switch off ***sequential scans***, which read every row in turn, with `SET enable_seqscan = off`, and `EXPLAIN` prints the plan each form gets:

```text
t.geom && ST_Transform(square, 4326)    →  Bitmap Index Scan on cafes_geom_geom_idx
ST_Transform(t.geom, 3857) && square    →  Seq Scan on cafes t   (Disabled: true)
```

Transform the column, and the database must convert every row before it can compare any: with a million places, the second form would read every place for every tile, while the first reads only the branches that touch the square.

> TRY IT In the running database, `docker exec --interactive --tty tessera-db psql --username postgres --dbname tessera`, run `SELECT ST_AsText(ST_Transform(ST_TileEnvelope(12, 3339, 1788), 4326));` and check that the home point lies inside the corners. Then work out by hand, with chapter 12's formulas, the tile at level 13 that holds the home point, and check it in the same way.

> COMMON MISTAKE Writing latitude first. PostGIS takes *x* then *y*: longitude, then latitude. `POINT(22.19 113.55)` sets a latitude of 113.55°, which no place on Earth has. The pipeline's `swapCoordinates` filter (mini-guide M1) swaps such a pair back before it reaches a seed script.

Check your understanding

1. What do the numbers in a column of SRID 4326 mean, and which SRID does a tile use?
2. How many tiles cover the world at level 12, and which four tiles are the children of `2/3/1`?
3. Why does the tile query transform the tile's square, and not the table's shapes, in its `WHERE` clause?
4. Café 3 comes out of `ST_AsMVTGeom` as `POINT(4294 3339)`. Where does it lie, and why does the tile keep it?

Answers. 1. Longitude and latitude in degrees; Web Mercator, EPSG:3857, in metres. 2. 4^{12} = 16,777,216; `3/6/2`, `3/7/2`, `3/6/3` and `3/7/3`. 3. So that the comparison uses the spatial index, which holds boxes in the table's own system. 4. 198 units beyond the right edge, inside the 256-unit buffer; the tile keeps it so that its symbol shows whole where the two tiles meet.

> THINK IT OVER Tessera leaves every spatial calculation to the database and passes on the finished tile. What does that choice gain, and what does it cost if the database becomes the slowest part of the system?

09
Web Application Security

Tessera answers each of six common attacks with its own defence, and stacks the defences so that no single failure opens the service. Nginx holds three of them, the C# code two, and PostgreSQL the last. xUnit tests in `tests/` prove the code's defences against a real database, and the edge checks in `scripts/lib/edge.ts` knock on Nginx from outside, in raw bytes. This chapter reads the defences against the OWASP Top 10 of 2025, names the gaps that remain, and shows how to run a free scanner, ZAP, by hand.

Two principles shape every defence

Defence in depth stacks several barriers, as a castle has a moat, a wall and a keep, so that the system stays shut when one barrier fails. ***Least privilege*** gives each part only the powers it needs, as a cashier holds the key to the till, not to the vault. Tessera follows both: a request meets Nginx, then Kestrel, the web server inside ASP.NET Core, then the C# code, and last a database role that may only read.

The list of threats comes from ***OWASP***, the Open Worldwide Application Security Project, a charity that publishes free security guidance. Its ***Top 10*** ranks the commonest kinds of weakness in web applications. Exhibit 9.2 reads the latest edition, of 2025, against Tessera. Four stories carry the security work: S-01, S-06, S-07 and S-30. The file `docs/requirements/security-requirements.md` turns their criteria into 26 requirements, SR-01 to SR-26, together with the criteria of other stories that guard a role, a grant or the office network. Each requirement names the test that checks it, adds 'in part' where the test covers only some of it, or says 'No test yet'.

> FOUNDATIONS Threats to systems (SQL injection, denial of service, interception) and ways to prevent them (validation, encryption, access levels, penetration testing).

Each defence stops one attack

Six attacks, six defences: Exhibit 9.1 pairs them, and the sections below take them one at a time.

EXHIBIT 9.1 Six attacks and Tessera's answers

ATTACK	WHAT THE ATTACKER WANTS	TESSERA'S DEFENCE	STORY
SQL injection	To run their own commands on the database	Bound parameters; names quoted by `SqlName`	S-01
Disguised address	To reach a route by a back way	Nginx and the service refuse it: 400	S-06
A stranger's request	To use the service from outside the office	Nginx answers the office network alone: 403	S-07
Flooding	To crowd out every other visitor	A rate and a cap for each machine: 429	S-30

Reaching too far	To read a table never published, or change one	A read-only role that sees granted tables alone	US-03
Leaky errors	To learn the system's insides	Problem documents that repeat nothing	US-07

Source: `docs/user-stories/user-stories.md`, `docs/requirements/security-requirements.md`

1. Parameters and quoted names stop SQL injection

SQL injection happens when a program glues a visitor's text into a database command. Tessera's tile query keeps every value apart from the command. The tile numbers travel as named ***parameters***, such as `@z`: values that Dapper and Npgsql send beside the command, which the database treats as data, whatever they hold. A table's name cannot travel as a parameter, so it enters the text another way: the address names a layer, and the service looks that layer up in the list of granted tables, then writes the table's name, quoted, into the query.

```csharp
// Queries.cs: values stay out of the text; each name arrives as a quoted SqlName
FROM {layer.Table.Sql} AS t
WHERE t.{layer.Geometry.Sql}
      && ST_Transform(ST_TileEnvelope(@z, @x, @y, margin => @margin), @srid)

// SqlName.cs: wraps the name in double quotes and doubles any quote inside
public static SqlName Quote(string name) =>
    new($"\"{name.Replace("\"", "\"\"", StringComparison.Ordinal)}\"");
```

A doubled quote cannot end the name early, so an attacking name stays one odd name. `SqlName`'s constructor is private, so only `Quote` and `Qualified` can make one, and the compiler refuses any other way into a query's text. Three kinds of test check the defence:

- **Every address, one text.** The property test `Keeps_every_tile_address_out_of_the_text`, in `TileTests`, tries random levels, columns and rows and finds the query's text equal, each time, to the approved text in `approved/tile-query.sql` (SR-05). `Quotes_every_name_safely`, in `SqlTests`, tries random names and finds no lone quote inside any of them.
- **An attack changes nothing.** `Answers_an_attack_with_404_and_keeps_every_row`, in `ServiceTests`, asks the running service for the layers `x; DROP TABLE cafes` and `food.cafes'; --`. No grant publishes either name, so each finds no layer in the list and answers 404 as a problem document, as any unknown name does. The test counts the rows of every table before and after, and finds each count unchanged (SR-08).
- **An odd name is served, not run.** `Serves_the_awkward_name`, in the integration tests' `CatalogueTests`, grants a real table named `odd "name"`, quotes and all, and its tile answers 200. `Serves_a_granted_table_named_as_an_attack_and_keeps_every_row`, in `ServiceTests`, goes further: it grants a table named `x"; DROP TABLE food.cafes; --`, finds its one place in the tile, and finds every table's rows unchanged.

In every case the name stays a name and never becomes SQL. One rule in `BoundaryTests`, `Only_Infrastructure_talks_to_the_database`, closes the side door: no code outside the Infrastructure layer may use Npgsql, Dapper or DbUp. A second, `Keeps_every_sql_text_in_queries_cs`, in the unit tests' `SourceTests`, reads every C# file of the service and finds SQL text in `Queries.cs` alone (SR-06).

2. Disguised addresses reach no route

A ***malformed URL*** hides its real shape until a web server tidies it, as `%2e%2e` hides `..`, the step that climbs up one folder. S-06 sets one rule: never repair an address. Nginx itself answers 400 to a target that climbs

above the root, or that holds a stray `%`, an invalid code such as `%zz`, `%00`, a raw control character, or any `%2e` at all, since no layer, page or file of the map holds one (chapter 4 shows the rule). Its `location = /malformed` writes the answer as a problem document whose title reads 'Bad Request' and which never repeats the address (SR-09, SR-10); a browser that asks for `/malformed` by name gets the 404 of any unknown address.

The service guards itself as well, for a request that reaches a copy directly, past Nginx. Kestrel decodes `%2e` and drops each `..` with the folder before it, so `/x/%2e%2e/food.cafes/0/0/0.pbf` would reach the tile route as a real tile and answer 200. `Targets.IsDisguised` reads the target as it was sent, before routing, and the step that calls it answers 400 with a problem document:

```csharp
public static bool IsDisguised(string target) =>
    target.Any(char.IsControl) || PathOf(target).Split('/').Any(IsDotFolderInDisguise);
// …
private static bool IsDotFolderInDisguise(string folder) =>
    folder.Contains('%', StringComparison.Ordinal)
    && folder.Replace("%2e", ".", StringComparison.OrdinalIgnoreCase) is "." or "..";
```

A folder written plainly as `..` is no disguise, and an encoded dot inside a longer name, as in `food%2Ecafes`, climbs nowhere, so both pass. An encoded slash, `%2F`, and a backslash match no route, and the service answers 404 (SR-11). The edge checks send twelve such targets through Nginx byte for byte, and expect the exact status, 400 or 404, with the matching title and no trace of the target. `Refuses_a_dot_folder_in_disguise_with_400`, `Answers_a_backslash_with_404` and `Answers_a_raw_control_character_with_400`, in `ServiceTests`, send the same tricks to a copy directly, and `TargetTests` tries the rule on its own.

3. Nginx answers the office network alone

Nginx answers 403 to any address outside `TESSERA_OFFICE_NETWORK`, before the cache or any other rule runs, and never passes the request on (SR-17; chapter 4). `npm start` refuses to start when that setting is missing, misspelt, or wider than a prefix of 8, such as `0.0.0.0/0` (SR-19).

4. Rate limits keep one machine from crowding out the rest

In a ***denial of service***, one client asks so much that honest users get no answer. Nginx caps each office machine at a burst of 300 tiles, then 100 a second, and at 32 requests at once, and answers 429 with `Retry-After: 1` past either cap (SR-20 to SR-23); chapter 4 shows how, and chapter 12 works the numbers. The machine Tessera runs on meets no limit, so its own checks never trip one (SR-24, no test yet).

5. A read-only role limits what any mistake can reach

The service signs in to PostgreSQL as `tessera_reader`, a role that sees only the tables granted to it, and whose every transaction is read-only (`default_transaction_read_only = on`); chapter 7 shows both roles. An injection that slipped past every other wall would therefore meet a role that can change nothing and see nothing unpublished. `Keeps_the_reader_read_only` checks the setting, and `Publishes_what_is_granted_and_nothing_else` checks that `stage.never_granted`, which no one granted, stays out of the list (SR-02).

6. Problem documents tell a stranger nothing

An error page that shows SQL or a stack trace hands an attacker a map of the system. Every Tessera error, from the service or from Nginx, is a problem document of four fields at most, with no `traceId` and no stack

trace (SR-13); chapter 5 shows how `Problems.Shape` trims each one.

The map page adds a ***Content Security Policy***: rules that tell the browser where the page may load scripts, styles and other files from. Its first rule, `default-src 'self'`, allows only the page's own site, so a script slipped into a place's name cannot fetch another site's code. `MapRoutes.Policy` holds the whole policy, and `Serves_the_map_page_with_the_area_written_in` checks that the page carries it.

Secrets and the supply chain need guarding too

Two more risks lie outside any request: a password that leaks through Git, and a library that brings a flaw with it. Each database password lives in `.env`, which Git ignores, and the shipped `tessera.json` holds no password and listens on 127.0.0.1 alone (SR-26). The ***software supply chain*** is every library, image and tool the build pulls in. ***Dependabot***, GitHub's robot, reads `.github/dependabot.yml` and watches eight lists of pinned versions: the NuGet packages, the .NET SDK in `global.json`, the scripts' and the map's npm packages, the pipeline's Python packages, the database and Nginx images, and the actions in `build.yml`. Each Monday at 06:00, Taipei time, it opens a ***pull request***, a proposed change to the code, for each newer release, and the build tests it before anyone merges it (chapter 10). Minor and patch releases arrive grouped, one pull request for each list, so a week's updates make one review; a major release, which may break things, arrives alone. A ***cooldown*** of seven days holds back each release for a week, so one withdrawn in its first days never reaches the office. The database image never moves to a new major version of PostgreSQL this way: such a version keeps its data in a form the old one cannot read, so the move is a planned upgrade. The requirements file names both safeguards, `.env` and Dependabot, as having no story yet, and so no test.

EXHIBIT 9.2 The OWASP Top 10 of 2025, as it touches Tessera

ENTRY	TESSERA'S ANSWER, OR ITS GAP
A01 Broken Access Control	Office network only (S-07); a read-only role; grants publish
A02 Security Misconfiguration	No `traceId` in errors; the service listens on 127.0.0.1 alone. Gap: Nginx names its version
A03 Software Supply Chain Failures	Pinned versions; Dependabot each Monday
A04 Cryptographic Failures	Gap: plain HTTP, no TLS
A05 Injection	Parameters and quoted names (S-01); a Content Security Policy
A06 Insecure Design	Security stories with tests written first
A07 Authentication Failures	No visitor logs in; each database role has its own password, kept in `.env`
A08 Software or Data Integrity Failures	Every push and pull request built and tested; DbUp runs each seed script once
A09 Security Logging & Alerting Failures	Crashes, failed catalogue reads and the cause of each tile's 500 logged. Gap: no alert reaches anyone
A10 Mishandling of Exceptional Conditions	Each database failure comes back as a value, sorted into four kinds; a fault carries no detail

Source: owasp.org/Top10/2025; `docs/study/06-security-and-maintenance.md`

The known gaps

Tessera's defences leave four gaps, named here so that someone can close them. The largest is **plain HTTP**. Nginx listens on port 80 with no ***TLS***, the encryption behind HTTPS, so anyone on the office network could read or alter the tiles and the map's scripts on their way. No ADR records a decision on TLS, and no story asks for it. The study plans in `docs/study` name the gap and set its cure as an exercise. Plan 01 asks for an S- story for TLS; plan 06 ends with ADR 0012, 'TLS on the office network', put in place with a certificate on Nginx and port 80 sending browsers on to 443. Among its options is a private certificate authority. A certificate authority vouches that a server is the one it claims to be; an office may need its own, since its network has no public name for a public authority to vouch for.

The other three gaps are smaller. Nginx names its version in its `Server` header, `nginx/1.29.8` on the day this chapter was checked, a clue to which known flaws apply; `server_tokens off;` would hide it. The service logs a crash, a failed read of the list of layers, and the cause of each tile's 500 (chapter 1), but no alert reaches anyone. And study plan 06 records that Dependabot's alerts still wait to be switched on, so a flawed version waits for Monday like any other.

A scanner checks the defences from outside

A ***penetration test*** goes beyond the tests: it attacks a system, with its owner's permission, to find weaknesses before criminals do. ZAP, a free scanner, automates part of that work. Its ***baseline scan*** follows every link it finds for a minute, then reads each reply with passive rules, which look for faults such as a missing header; it sends no attacks. Tessera's build leaves ZAP out, and the Try it box shows how to run it by hand.

> TRY IT Start Tessera with `npm start`, then run ZAP's baseline scan from a throwaway container on the same machine: `docker run --rm --network host -t ghcr.io/zaproxy/zaproxy:stable zap-baseline.py -t http://127.0.0.1/`. Read each WARN line: expect a warning about the `Server` header that leaks a version. Which other warnings appear, which are real, and which is a ***false positive***, an alarm about a weakness that is not there? Scan only systems you own: an attack without permission breaks the Computer Misuse Act 1990.

> COMMON MISTAKE Building a query with a layer's name from the address, quoted by hand. One missed case opens the door. Look the name up in the list of granted tables, and let one tested function, `SqlName.Quote`, write every name into SQL.

Check your understanding

1. Why can a layer's name not travel as a bound parameter, and what does Tessera do instead?
2. One office machine sends 500 tile requests in its first second. Roughly how many pass, and what do the rest receive?
3. Why does Nginx check the office network before it looks in its cache?
4. Why may ZAP's baseline scan run against the office's own service, while an active scan needs more care?

Answers. 1. A parameter carries a value, never part of the command's shape; the service looks the name up among granted tables and writes it quoted by `SqlName`. 2. About 400: the burst of 300 and 100 more over the second; the rest get 429 with `Retry-After: 1`. 3. So a stranger receives no kept tile either. 4. The baseline scan sends no attacks and only reads replies; an active scan sends real attacks, which may change data or slow the service.

> THINK IT OVER Tessera's build runs no scanner; ZAP waits for someone to run it by hand. What would ZAP's baseline scan in `.github/workflows/build.yml` catch on every push, what would it cost, and how would you stop a warning you have accepted, such as the `Server` header, from failing every build?

10
Build, Test and Release

One command builds Tessera, one command tests it, and GitHub repeats both on every push, so a change ships only once every kind of test has passed. The .NET SDK builds the service from a few shared files, with the compiler's strictest checks. Three test projects then prove it, beside the tests of the pipeline, the scripts, the map and the browser. A release is one folder that holds the program, its settings, its seed scripts and the map.

The .NET SDK builds the service from a few shared files

The ***.NET SDK*** is the toolkit that builds, tests and runs C#: the command `dotnet` reads each project file, downloads the packages it names from NuGet, compiles the code and runs the tests. Tessera keeps the rules every project shares in four small files at the top of the repository, so every project builds alike.

EXHIBIT 10.1 The four files that fix how every project builds

FILE	WHAT IT FIXES
`global.json`	The SDK, 10.0.100, which may roll forward to a newer feature band with fixes but never to .NET 11; and the test runner, Microsoft.Testing.Platform
`Tessera.slnx`	The solution: the service, `src/Tessera`, and the three test projects under `tests/`
`Directory.Build.props`	For every project: .NET 10, C# 14, nullable checks, every warning an error, every analyser rule, and no culture data, so `22.19` never reads as `22,19`
`Directory.Packages.props`	The one version of each package: Npgsql 10.0.3, Dapper 2.1.89, DbUp 7.0.1, xunit.v3 4.0.2, FsCheck 3.4.0, Testcontainers 4.16.0, ArchUnitNET 0.13.4, Microsoft.Testing.Extensions.CodeCoverage 18.11.2 and the rest

Source: `global.json`, `Tessera.slnx`, `Directory.Build.props`, `Directory.Packages.props`

As Exhibit 10.1 shows, a version lives in one place. `Directory.Packages.props` holds ***central package management***: each project names a package, and this one file names its version, so the service and its tests never disagree. The service's project, `src/Tessera/Tessera.csproj`, copies three things beside the program: `tessera.json`, the built map from `web/dist` as `wwwroot/`, and the seed scripts from `db/seed` as `seed/`.

> FOUNDATIONS Translators: the C# compiler turns the code into ***intermediate language***, which the .NET runtime turns into the processor's own instructions as the program runs. Testing with normal, boundary and erroneous data: `TileTests` accepts `22/4194303/0`, the last column at level 22; refuses `1/2/0`, one step beyond the grid; and refuses `23/0/0` and `-1/0/0`, which name no level the service serves.

The compiler refuses faults before any test runs

The cheapest fault to fix is one the compiler finds, so `Directory.Build.props` makes every warning stop the build. Chapter 2 explains each check; these are the lines that switch them on:

```xml
<LangVersion>14</LangVersion>
<Nullable>enable</Nullable>
<TreatWarningsAsErrors>true</TreatWarningsAsErrors>
<EnforceCodeStyleInBuild>true</EnforceCodeStyleInBuild>
<!-- … -->
<AnalysisLevel>latest</AnalysisLevel>
<AnalysisMode>All</AnalysisMode>
```

Two packages add rules of their own: Meziantou.Analyzer adds checks .NET lacks, and ExhaustiveMatching stops the build when a `switch` over `DatabaseError` misses a case. Both run only while building, so neither ships. `EnforceCodeStyleInBuild` runs the editor's style rules in every build, so GitHub and the editor agree; one of them, IDE0130, makes each file's namespace match its folder, which the architecture tests rely on (chapter 3).

Each kind of test answers its own question

A sound suite has many fast tests that each check one rule, and a few slow ones that check the parts together. Tessera's three test projects use ***xUnit***, version 3, a test framework that builds each test project as a program of its own; `global.json` names Microsoft.Testing.Platform as its runner. So `dotnet run --project tests/Tessera.UnitTests` runs one project, and `dotnet test` runs all three. Exhibit 10.2 lists every kind, fastest first. The .NET projects hold 167 cases in all: 6 architecture, 94 unit and 67 integration, counted in the code of 18 October 2026; a theory counts once for each row of data it tries.

EXHIBIT 10.2 The kinds of test, what each proves, and how many cases the .NET classes ran

CLASS	IT PROVES	TOOLS	CASES
Tessera.UnitTests: no database and no network; 94 cases in seconds			
`TargetTests`	A dot folder in disguise or a control character is spotted; a clean address passes	xUnit	17
`TileTests`	Addresses on the grid, the tile query's text, a tile's two headers, the log line of a 500	xUnit, FsCheck, an approved file	17
`SettingsTests`	`tessera.json` fills the settings; every bad value and key is named at once	xUnit	16
`SourceTests`	The code names no machine, keeps its SQL in `Queries.cs`, and lists the map's nine layers	xUnit, the files as text	13
`CatalogueTests`	Rows become layers; the last good list survives a failure	xUnit	10
`MapTests`	The map page's filling-in, and the nine page names	xUnit	10
`DatabaseErrorTests`	Each SQLSTATE code sorts into one kind of failure	xUnit	9

`SqlTests`	Every name comes out as one quoted name	xUnit, FsCheck	2
Tessera.ArchitectureTests: 6 cases in 8 to 13 s on a quiet machine, up to 45 on a busy one			
`BoundaryTests`	The borders between the layers and modules	ArchUnitNET	6
Tessera.IntegrationTests: 67 cases, the slowest of the three			
`ServiceTests`	The whole service over HTTP, against the real database, raw addresses included	Testcontainers	42
`CatalogueTests`	Publishing by grant, a grant seen within one refresh, 503 when the database fails	Testcontainers	8
`SeedingTests`	Each seed script runs once, in the order of its name, and a failed one leaves nothing behind	Testcontainers, DbUp	5
`StartTests`	The program refuses to start and names every problem; `/health` answers within a second	The real program	5
`PoolTests`	Queries wait in the order they arrive; the ping gives up after a second	Testcontainers	3
`DatabaseTests`	The shipped roles, and a reader that cannot write	Testcontainers	3
`FrozenDatabaseTests`	A paused database answers 503 within the settings' limits	Testcontainers	1
Around the service			
Pipeline	Filters, properties for any input, outcomes as recorded	pytest, hypothesis, golden files	—
Scripts	The pure parts of each zx script, in nine test files	`node --test`	—
Map	The map's logic and components, in a fake page	Vitest, jsdom	—
Browser	The map in Chromium; three approved images	Playwright	—
Nginx checks	403, 429, 304, gzip, malformed and disguised addresses, a copy failing	`scripts/lib/edge.ts`	—

Source: `tests/`, counted on 18 October 2026; ADR 0010 for the architecture tests' time; `etl/tests`, `scripts/lib`, `web/`

The integration tests meet the real thing. They use ***Testcontainers***, a library that starts Docker containers for a test and removes them afterwards. `DatabaseFixture` builds the image from the project's own `db/Dockerfile`, with `ImageFromDockerfileBuilder`, so a test signs in as the shipped roles, never as hand-made copies. Each test class starts a database of its own, and `xunit.runner.json` lets two classes run at once, no more, since the build machine may have two cores. `ServiceFixture` goes further: it loads the seed, grants the nine layers and starts the real service on a free port.

Some unit tests check a rule for every input, not a chosen few: the property tests of chapter 1, such as `TileTests.Gives_only_addresses_on_the_grid`, run each rule on a hundred inputs that FsCheck makes up. The pipeline does the same with hypothesis, and adds ***golden files***, which hold each case's recorded script, log and exit code; every run must match them exactly (mini-guide M1). Playwright drives Chromium through Nginx, and for UI-17 compares the map with three ***approved images***, screenshots that someone has checked and accepted, at levels 16, 18 and 20. It allows 1 pixel in 100 to differ (`maxDiffPixelRatio: 0.01`).

One story passes through every kind of test

S-01, ‘keep SQL out of every request’, shows how the kinds work together, and Exhibit 10.3 follows it. The story lists its ‘Tests to write first’. In ***test-driven development***, you write each test before its code: watch the test fail, write the least code that makes it pass, then tidy the code while every test stays green.

EXHIBIT 10.3 S-01 through each kind of test

KIND	TEST	WHAT IT PROVES
Unit	`SqlTests.Quotes_a_name_and_doubles_each_inner_quote`	`odd "name"` comes out as one quoted name
Property	`SqlTests.Quotes_every_name_safely`	Any string at all opens and closes with a quote, with no lone quote between
Approved file	`TileTests.Matches_the_approved_tile_query`	The query’s text matches `approved/tile-query.sql`
Property	`TileTests.Keeps_every_tile_address_out_of_the_text`	For any tile, the text stays the approved text
Unit	`TileTests.Sends_the_layer_and_the_tile_as_parameters`	The layer and the numbers travel as parameters alone
Architecture	`BoundaryTests.Only_Infrastructure_talks_to_the_database`	No other layer touches Npgsql, Dapper or DbUp
Integration	`ServiceTests.Answers_an_attack_with_404_and_keeps_every_row`	Two attacking names get a 404 problem document, and every table keeps its rows
Integration	`CatalogueTests.Serves_the_awkward_name`	A granted table named `odd "name"` serves a tile

Source: `tests/Tessera.UnitTests`, `tests/Tessera.ArchitectureTests`, `tests/Tessera.IntegrationTests`

The ***approved file*** is a test’s expected output kept as a file. `approved/tile-query.sql` holds the tile query’s text word for word. Change the query, and the test fails until someone reads the new text and copies it over the old file, so no change to the SQL slips through unread.

Every test run measures coverage against two floors

Code coverage measures which lines and branches the tests run; a branch is one of the two ways out of an `if` or a `?:`. Each test project names `Microsoft.Testing.Extensions.CodeCoverage`, so its program records the lines it runs when given `--coverage`. `npm test` and the GitHub build run each suite with the options `scripts/lib/coverage.ts` gives, and each suite writes a Cobertura file, an XML record, to `coverage/`. Then `node scripts/coverage.ts` has ReportGenerator, a tool pinned in `dotnet-tools.json`, merge the three into one report, `coverage/report/index.html`, and fails when the tests cover less than either floor: 94 per cent of the lines, 84 per cent of the branches.

EXHIBIT 10.4 Coverage of the service's code on one run, by test project, and the floors

PROJECT	LINES RUN	BRANCHES RUN
`Tessera.UnitTests`	270 of 554 (48.7%)	152 of 268 (56.7%)
`Tessera.IntegrationTests`	508 of 554 (91.7%)	201 of 268 (75.0%)
`Tessera.ArchitectureTests`	0	0
All three, merged	648 of 674 (96.1%)	267 of 312 (85.5%)
The floors	94%	84%

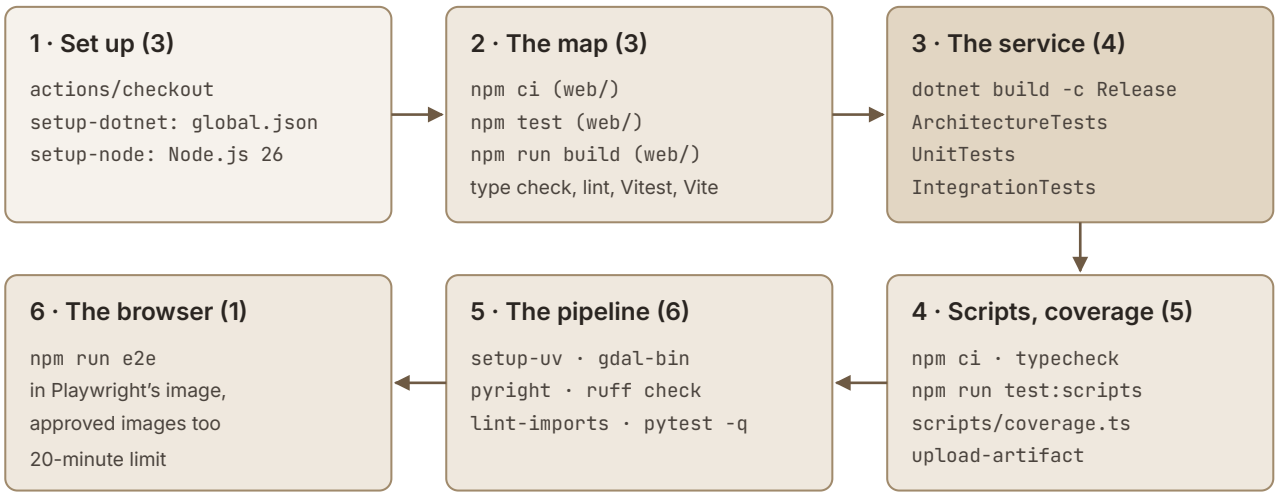
Source: `coverage/` after the run of 18 October 2026; the floors from `scripts/lib/coverage.ts`. ReportGenerator counts 674 lines as it merges the suites' 554, so only the merged figures meet the floors.

The architecture tests read the built program and never run it, so they rightly cover nothing. The lowest file is `Program.cs`, 20 of 31 lines, most likely because `StartTests` starts the real program as a separate process, which the collector cannot see. Coverage shows what no test runs, not what the tests prove: a test that runs a line and checks nothing still counts. Chapter 12 works out how much untested code the floors allow.

GitHub runs every check on every push

Continuous integration runs the checks on a fresh machine for each change. `.github/workflows/build.yml` runs on each push to `main` and on each pull request, on a fixed Ubuntu 24.04, the office machine's own system. Exhibit 10.5 groups its twenty-two steps.

EXHIBIT 10.5 The twenty-two steps of build.yml, in six groups, in order



Source: `.github/workflows/build.yml`

The map builds first, so the service's build can copy `web/dist` into itself. Then one `dotnet build --configuration Release` builds everything, and each test project runs with `dotnet run --no-build` and the coverage options, the architecture tests first, since they catch a broken border in seconds. Testcontainers builds the database on the runner's own Docker. After the scripts' checks, `scripts/coverage.ts` merges the three records and checks the floors, and the report stays on GitHub for a week, even when a floor fails. The browser tests run last, inside Playwright's own Docker image, whose Chromium and fonts draw the

approved images alike on any machine, so GitHub now compares them too; no GitHub run has yet proved it.

A second file keeps the parts current. `.github/dependabot.yml` has Dependabot propose each newer release of every pinned version as a pull request, each Monday, and this build tests each one before anyone merges it; chapter 9 explains the lists, the groups and the week's cooldown.

A release is one folder with the map inside

A release is the output of `npm run build`. The command installs the map's tools, builds the map into `web/dist`, empties the release's `wwwroot/`, so no old map file ships beside the new ones, and runs `dotnet build --configuration Release`. The result sits in `src/Tessera/bin/Release/net10.0/`: `Tessera.dll`, `tessera.json`, the map in `wwwroot/` and the seed scripts in `seed/`. The service finds the map in `wwwroot` beside the program. With `DATABASE_URL` set, `dotnet src/Tessera/bin/Release/net10.0/Tessera.dll` serves tiles and the map on port 7800, as `Urls` in `tessera.json` says; the same program with the word `seed` runs the seed scripts instead (Appendix A lists the commands).

In the office, `npm start` runs two copies of the program, on ports 7800 and 7801, behind Nginx. There the release process ends: no project names a version, and the repository holds no tags and no workflow that publishes a release. Adding them is open work. They would follow ***semantic versioning***: numbers of the form `MAJOR.MINOR.PATCH`, where a change that breaks old uses raises MAJOR, a new feature raises MINOR, and a fix raises PATCH.

> TRY IT Watch a floor fail. Run `npm test`, open `coverage/report/index.html`, and find the lines of `Program.cs` no test runs: should a test reach them at all? Then, on a branch, raise `lines` in the `floors` of `scripts/lib/coverage.ts` to 97, and run `node scripts/coverage.ts` again: it names the shortfall and ends with exit code 1. Put the floor back; never lower one to pass a change.

> COMMON MISTAKE Rerunning a failed build until it passes. A test that passes at the second attempt hides a real fault, often a race between two tasks or a wait that is too short; `PoolTests`, which times queries, is the kind to watch. Read the failure, fix its cause, and let the build go green honestly.

Check your understanding

1. What does `global.json` fix, and what would go wrong on a laptop without it?
2. Why does `DatabaseFixture` build its database from `db/Dockerfile` rather than from a plain PostgreSQL image?
3. What does an approved file catch that a property test cannot?
4. Why do the architecture tests cover no line of the service?

Answers. 1. The SDK, 10.0.100 or a newer feature band of .NET 10, and the test runner, Microsoft.Testing.Platform. Without it, a laptop with another SDK could build with other rules, or run the tests another way, and pass where GitHub fails. 2. So the tests meet the shipped roles, grants and settings, never a hand-made copy. 3. Any change in the exact text, such as a reordered clause, even where every property still holds. 4. They read the built program and check which types depend on which; they never run its code.

> THINK IT OVER The full set of checks takes many minutes on every push, and Tessera has one developer. If you could keep only three steps of `build.yml`, which would you keep, and which faults would you then accept reaching the office?

11
Performance

Tessera stays fast because every stage of a tile request caps what it may cost, and a query plan, not a guess, shows where the remaining time goes. Earlier chapters set each cap. This one lines them up, reads the plan of the real tile query, follows a tile through Nginx's store and the browser's, and ends with the measure Tessera still lacks: a load test in the repository. Chapter 12 shows the arithmetic behind every number.

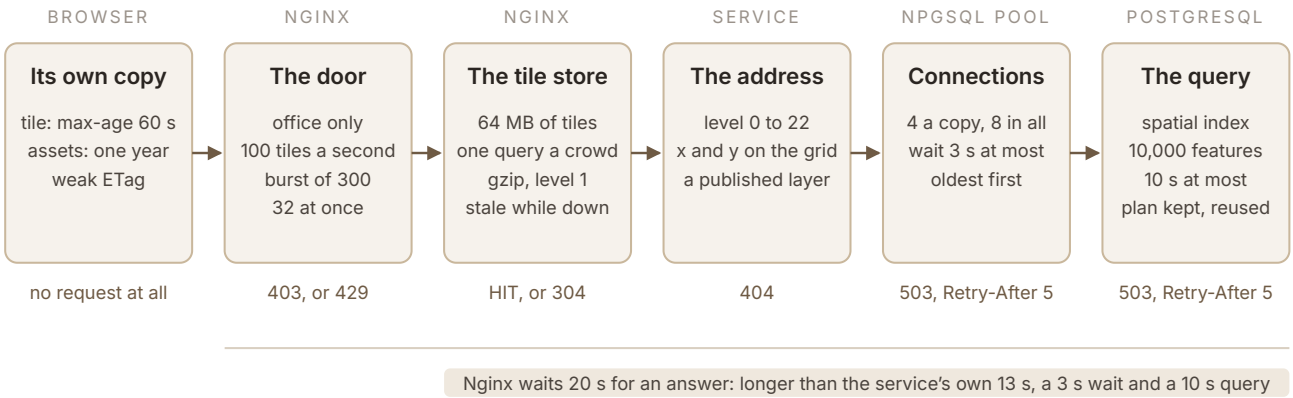
Each stage of a tile request caps its cost

A slow tile costs the visitor time, and a costly one costs every other visitor too, so each stage refuses work it cannot afford. ***Latency*** is the time from request to answer. The stages keep it low in three ways. The early stages answer from a stored copy, so the later ones never run. The door refuses a visitor who asks too much. The last stages cap how long they wait and how much they build, and refuse clearly rather than hang.

> FOUNDATIONS Linear and binary search, and the cost of an algorithm. A spatial index does for shapes what binary search does for a sorted list: it skips most of the data unread. A cache is a stored answer, like a variable that keeps the result of a costly subroutine so the program need not call it again.

No requirement sets a target time for a tile. The file `docs/requirements/non-functional-requirements.md` asks instead that a tile cost less to send again (NFR-01 to NFR-17) and that the service stay steady when the database is slow or down (NFR-18 to NFR-40). Chapters 4 and 6 show the code that meets them; this chapter shows what each costs and what it saves.

EXHIBIT 11.1 One tile request, from left to right: each stage caps its cost, and most refuse in their own way



Top row: what each stage holds or checks. Brown row: what the visitor receives when that stage answers or refuses.

Source: `nginx/default.conf.template`, `tessera.json`, `Features/Tiles.cs`, `Domain/Tiles.cs`, `Infrastructure/Sql/Database.cs`

As Exhibit 11.1 shows, the cheapest answer comes first. A tile the browser holds costs nothing; a tile Nginx holds costs no query; only a tile no one holds reaches the pool and PostgreSQL. The rest of the chapter

takes the stages from the inside out, since the query is where the time goes.

A query plan shows where the time goes

PostgreSQL can explain any query it runs. Put `EXPLAIN (ANALYZE, BUFFERS)` in front of the query, and PostgreSQL runs it and prints its ***query plan***: the steps it chose, the time each took, and the pages each read. A page is one 8 KB block of a table. To capture a plan, copy the text of `Queries.Tile` from `Infrastructure/Sql/Queries.cs`, and put the tile's numbers and the layer's settings in place of the parameters `@z`, `@x`, `@y` and the rest, as below. The seed's cafés carry 23 columns besides the shape; the listing cuts them.

```sql
EXPLAIN (ANALYZE, BUFFERS)
SELECT ST_AsMVT(tile.*, 'food.cafes', 4096, 'tessera_shape') FROM (
  SELECT ST_AsMVTGeom(ST_Transform(t."geom", 3857), ST_TileEnvelope(12, 3339, 1788),
                      4096, 256, true) AS tessera_shape, "fid", "id", "name" -- …
  FROM "food"."cafes" AS t
  WHERE t."geom" && ST_Transform(ST_TileEnvelope(12, 3339, 1788, margin => 0.0625), 4326)
  ORDER BY t."fid"
  LIMIT 10000
) AS tile;
```

The numbers come from `tessera.json`: a resolution of 4,096, a buffer of 256, so a margin of 256 ÷ 4,096 = 0.0625, and at most 10,000 features. The `WHERE` clause turns the tile's square into the table's own system, SRID 4326, and leaves the table's shapes in theirs. That way the operator `&&`, 'the boxes overlap', can use the spatial index (chapter 8). The seed's table `food.cafes` holds 100 places, and 54 of them fall in this tile and its buffer. Its plan holds a surprise: PostgreSQL walks the primary key, `cafes_pk`, reads all 100 rows, keeps 54 and removes 46, in about a millisecond (1.0 to 1.1 ms over three runs). For a table that fits in a few pages, reading every row costs less than asking the index. The spatial index earns its place as the table grows.

To see the index at work, the author ran an experiment outside the repository. A test table, `food.many`, held 100,000 made-up points over Macau's box, with the seed's kind of index, and the query asked for the tile at level 16 that holds the home point, 16/53439/28623. Exhibit 11.2 shows the plan PostgreSQL chose.

EXHIBIT 11.2 The plan of one tile from 100,000 points, with the spatial index (trimmed; PostgreSQL 18.6, Tessera's database image)

```text
Aggregate  (actual time=1.963..1.965 rows=1.00 loops=1)
  ->  Limit  (actual time=1.347..1.551 rows=305.00 loops=1)
        ->  Sort  (actual time=0.765..0.782 rows=305.00 loops=1)
              Sort Key: t.fid
              ->  Bitmap Heap Scan on many t  (actual time=0.138..0.691 rows=305.00 loops=1)
                    Recheck Cond: (geom && '0103…'::geometry)
                    Buffers: shared hit=259
                    ->  Bitmap Index Scan on many_geom_idx  (actual time=0.102..0.102 …)
                          Buffers: shared hit=9
Planning Time: 183.615 ms
Execution Time: 2.068 ms
```

Source: the third of three runs of the query above against tile 16/53439/28623 of a test table, `food.many`, made for this chapter on 17 October 2026; it is not in the repository.

Four rules read any plan.

1. **Read from the inside out.** The most indented line runs first and feeds the line above it: the index finds the rows, the heap scan fetches them, the sort orders them, and the aggregate packs them into one tile.
2. **Read the actual figures.** `actual time=0.138..0.691` means the first row came after 0.138 ms and the last after 0.691 ms. `rows` counts the rows a step produced, and `loops` how often it ran; multiply the two for the rows in all.
3. **Look for the index.** `Bitmap Index Scan on many_geom_idx` means the spatial index did its work. `Seq Scan`, a sequential scan (chapter 8), on a large table means PostgreSQL read every row.
4. **Read the buffers.** `shared hit` counts the 8 KB pages found in PostgreSQL's own memory; `read` counts pages fetched from the operating system or the disk, which cost far more.

With the index dropped, the same tile took 18 to 51 ms over three runs, against 2 to 6.5 ms with it, and read 1,052 pages instead of 280. The plan said `Seq Scan on many t`, with `Rows Removed by Filter: 99695`, to keep 305 rows. The index made the tile about ten times faster on that machine, and the gap grows with the table, since a sequential scan reads every row. Four faults cause most slow tiles, and each leaves its own mark in the plan.

WHAT THE PLAN SHOWS	THE CAUSE	THE FIX
`Seq Scan` on a large layer	No spatial index, or `ST_Transform` on the table's column	`CREATE INDEX … USING GIST (geom)`, as the seed does for every table; transform the tile's square, never the column
Estimated rows far from the actual rows	Stale statistics after a load	`ANALYZE` the table; the seed runs it after each one
The `Limit` reached at a low level, with most time above it	Too many shapes for one tile	Raise the layer's lowest level, or publish a simpler view for low levels
A large `read` count on every run	Tables larger than PostgreSQL's memory	More `shared_buffers`, if the machine can spare it

The service plans each query once

The plan above spent 184 ms on planning and 2 ms on running. That first figure came from a fresh `psql` session, which loads PostgreSQL's own tables of names as it plans, so it overstates the cost; still, planning a query afresh each time costs more than running a short one. A ***prepared statement*** is a query that the database parses and plans once, keeps, and reruns with new values. Bound parameters make this possible: every tile of a layer sends the same text, with only `@z`, `@x` and `@y` changing, so one kept plan serves them all, and S-01's rule against SQL injection (chapter 9) pays for itself in speed. Npgsql prepares a query for you once the same text has run three times on a connection, as `Database.Open` sets:

CSHARP

```
// A query text sent three times on a connection becomes a prepared statement: the
// database parses and plans it once, keeps the plan, and reruns it with new values
// each time. On 14 October 2026 it raised the rate from 1,580 to 2,408 tiles a
// second, with no tile cache: one 30-second k6 run per figure, random tiles at
// levels 13 to 18, on a 2-processor machine that the service, PostGIS and k6
// shared, against the build of that day. The figures compare the two settings, not
// an office server. The k6 script was not kept; the notes live in Git alone, in
// docs/changes/2026-10-14-measured-under-load.md (commit a36a011, deleted in
// 99d5002).
MaxAutoPrepare = 32,
AutoPrepareMinUsages = 3,
```

The database's settings follow from measurements

The database image sets five settings, and the comments in `db/Dockerfile` give each one's reason. The first rests on a measurement from phase 0, the project's first trial. Just-in-time compilation turns a query into machine code before it runs. With it, a tile query took 7.1 seconds the first time and 0.016 seconds at best; without it, 0.001 seconds. Compiling a query costs far more than running a short one, so Tessera switches it off.

EXHIBIT 11.3 The five settings in the database image, and why each holds

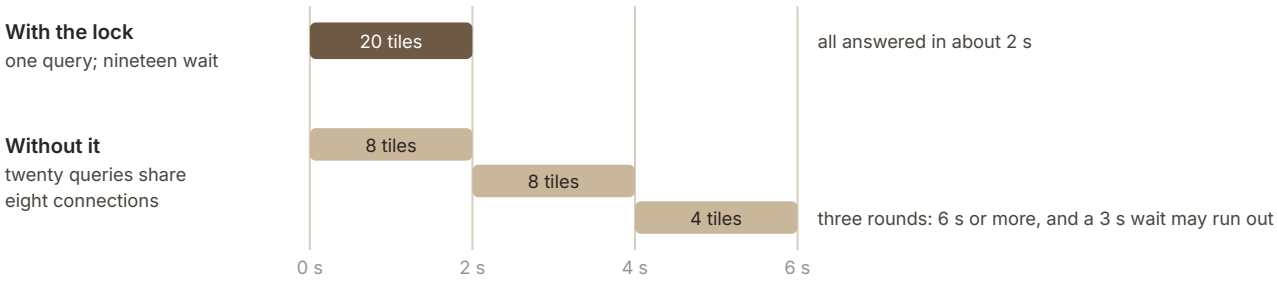
SETTING	VALUE	WHY
`jit`	off	A tile query took 0.001 s without it, 0.016 s at best with it, and 7.1 s at first
`max_connections`	20	Two copies of the service hold at most 4 connections each (`Database.PoolSize`), so 8; the rest leave room for the seed, `npm run publish` and `psql`
`shared_buffers`	256MB	The office machine shares its memory with the service, Nginx and Windows
`effective_cache_size`	2GB	The planner may count on about 2 GB more of the tables in the operating system's own cache
`random_page_cost`	1.1	On an SSD a read from anywhere costs about as much as the next read in order, so the planner prefers the spatial index; the default, 4, suits a spinning disk

Source: `db/Dockerfile`. The phase-0 timings come from its comment, which cites phase 0's `settings.md`; that file is not in this repository.

Nginx's store turns a crowd into one query

Nginx keeps each tile it passes on, in a store of 64 MB (chapter 4), so a second visitor's request costs no query. The store alone fails the first visitors, who arrive together before any copy exists. `proxy_cache_lock on` lets the first request build the tile and makes the rest wait for its answer. The edge check in `scripts/lib/edge.ts` proves it for N-24 with twenty requests at once for a tile that takes two seconds to build, as Exhibit 11.4 shows.

EXHIBIT 11.4 Twenty requests at once for one new tile that takes 2 seconds to build, with and without Nginx's lock



Source: `scripts/lib/edge.ts`, the N-24 check, which passes when all twenty answer 200 in 1.8 to 4 seconds; the lower row is the check's own comment worked out, not a measurement

Two more savings travel with every answer. When a browser's copy has expired, it sends a conditional request, which asks whether the tile has changed; if it has not, the answer is a 304 with no body (chapter 4). And Nginx compresses each answer with gzip at level 1, the fastest; a tile gains little from a higher level, as Exhibit 11.5 shows.

EXHIBIT 11.5 What gzip saves: one tile from the seed, and two of the map's files

ANSWER	PLAIN, BYTES	LEVEL 1, BYTES	LEVEL 9, BYTES
Tile 12/3339/1788 of `food.cafes`, 54 places	17,575	5,991 (34%)	5,502 (31%)
The map's main script, `index-Dn_nHZ4b.js`	65,113	23,196 (36%)	–
MapLibre, `maplibre-DoL_LsL2.js`	1,027,298	330,383 (32%)	–

Source: the tile built on 17 October 2026 from `db/seed/0001-macau-places.sql` with every column and the shipped buffer of 256, as the service sends it, and the files in `web/dist/assets/`, packed with the `gzip` program at `-1` and `-9`; Nginx's own gzip may differ by a few bytes

The browser's own copy costs no request at all

The cheapest request is the one the browser never sends. Every tile carries `Cache-Control: public, max-age=60`, built by `TileServer.CacheControl` from `Tiles.CacheSeconds` in `tessera.json`. For 60 seconds the browser reuses its copy and asks nothing. Set the number to 0, and the header becomes `no-cache`, which makes the browser check every time. Nginx keeps a tile for the same `max-age`, so one number rules both stores.

The map's own files follow two other rules, both set in `Features/Map.cs`. The page itself carries `no-cache`, so a new release reaches every browser at its next visit. The scripts and styles under `/assets/` carry `public, max-age=31536000, immutable`: 365 days, and a promise that the file stays the same. A year is safe because each file's name holds a fingerprint of its content, such as `index-Dn_nHZ4b.js`: a new release brings new names, and the page, checked each time, points to them. Errors carry `no-store`, set by `Problems.Shape`, so no store anywhere keeps a failure.

> WATCH OUT Nginx sends a stored tile without an `Age` header, the header that would tell the browser how long the tile has already waited. By that reasoning, which no test checks, a browser that receives a tile near the end of its 60 seconds in Nginx's store keeps it for 60 seconds more. A changed place could then take up to two minutes to reach every screen, not the minute that N-18 promises.

Tessera lacks a load test

Tests check each cap above on its own, but nothing in the repository measures the whole service under steady traffic. A ***load test*** sends a steady stream of requests and records every answer's time. The 2,408 tiles a second in `Database.cs` came from such runs, made with k6 outside the repository: one 30-second run for each figure, random tiles at levels 13 to 18, no tile store, and one machine with two processors shared by the service, PostGIS and k6. Its comment says the figures compare two settings, not an office server, and no script of it was kept. Its result comes as a ***percentile***: sort the times from fastest to slowest, and the p95 is the time that 95 requests in 100 stay within. An average hides the slow few: if 96 tiles take 20 ms and four take 1,000 ms, the average is 59.2 ms, a time no request took; the p95 is 20 ms and the p99 1,000 ms. Tessera's browser tests already work out a p95 in this way, in `web/e2e/drag.measure.ts`, for the time between frames while the test drags the map.

> TRY IT Write the load test Tessera lacks, as study plan 4 asks (pomodoro 12 and the capstone). Install k6, a free load-testing tool. Write `load/tiles.js` to ask for the tiles of one view of all nine layers through Nginx at 100 requests a second for one minute, and run it three times. Record the p50, p95 and p99 of each run, and the count of each status. Then raise the rate until the first 429 or 503 appears, and name the stage in Exhibit 11.1 that sent it. Run k6 from a second machine on the office network: Nginx exempts the machine it runs on, 127.0.0.1, from every limit.

When a figure disappoints, the order of work stays the same: read the query plan first, and look at the C# code only once the plan is sound. A millisecond saved in C# means nothing beside a query that scans a large table without its index.

> COMMON MISTAKE Measuring with a warm store and calling the result the service's speed. A second run of the same tiles meets Nginx's store and the browser's, and measures neither PostgreSQL nor the service. Ask for tiles no store holds, or read `X-Cache-Status` and count the `MISS` answers apart.

Check your understanding

1. Why did PostgreSQL ignore the spatial index on the seed's table of 100 cafés?
2. What does `Rows Removed by Filter: 99695` tell you about a plan?
3. Why does a prepared statement help the tile query more than most queries?
4. Why may the map's scripts be kept for a year while a tile is kept for 60 seconds?

Answers. 1. The table fits in a few pages, so reading every row costs less than asking the index; the index pays only as the table grows. 2. PostgreSQL read about 100,000 rows to keep a few hundred: a sequential scan where an index should work. 3. Every tile of a layer sends the same text with new parameters, so one kept plan serves thousands of requests, and the query runs in about a millisecond, less than planning it would take. 4. Each script's name holds a fingerprint of its content, so a new release brings a new name; a tile keeps its address while its places change.

> THINK IT OVER Nginx's store and the browser's spare the database almost every request on a quiet day. What would a load test that passes on a quiet day promise about the first morning after a new seed, when every stored tile has expired?

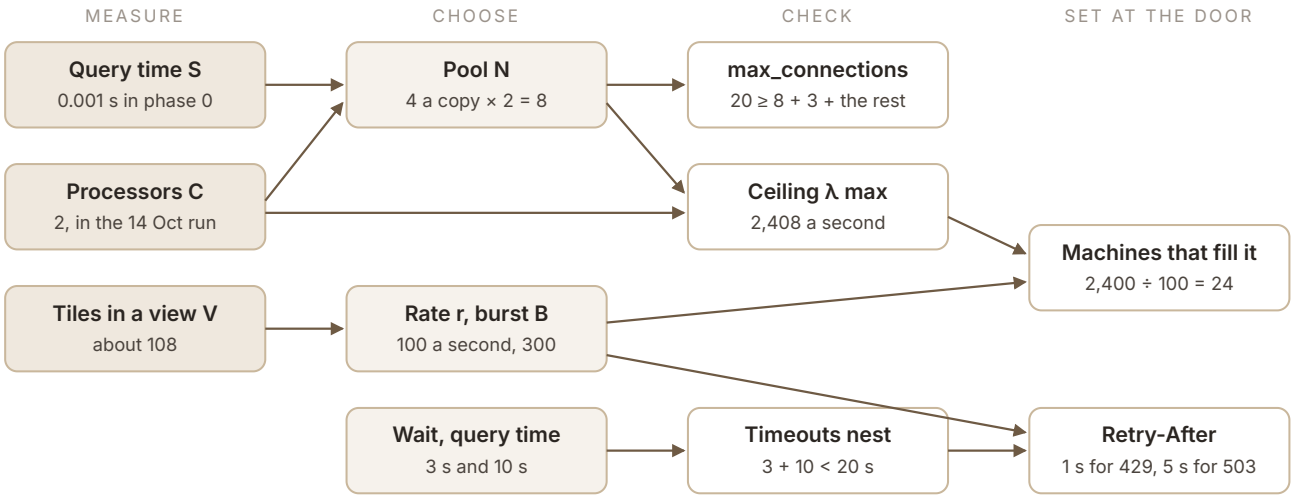
12
The Mathematics of the Limits

One measured number, the time a tile query takes, sizes the pool, the queue and the doors; the other limits follow from counting, powers of two and a few inequalities that a school student can check. This chapter works out each limit from the code's own numbers, with a worked example for each: the tiles in the grid and the tile at a point, the bits a number needs, Little's law for the pool, the leaky bucket at the door, the timeouts that nest, the backups kept and a coverage floor.

One small model links every number

Earlier chapters named many numbers: 4 connections a copy, 100 tiles a second, a burst of 300, a 3-second wait, a 10-second query, a 20-second proxy timeout. Each rests on a measurement or a count. Exhibit 12.1 shows how they depend on one another: what you measure on the left decides what you choose and check on the right.

EXHIBIT 12.1 How the numbers depend on one another: measure on the left, set at the door on the right



Source: `tessera.json`, `nginx/default.conf.template`, `db/Dockerfile`; *the 2,408 tiles a second on two processors comes from the comment in* `Infrastructure/Sql/Database.cs`, *which S-30 and ADR 0007 round to about 2,400*

> FOUNDATIONS Powers and indices, rearranging a formula, rates and proportion, inequalities, the floor of a number, and binary: how many values *n* bits can hold. Every result in this chapter needs nothing harder.

The grid holds 4^z tiles at level z

Each level splits every tile into four, so the count of tiles grows as a power of four (chapter 8 draws the quadtree). At level *z* the world is 2^z tiles wide and 2^z tiles tall, as the comment on `TileAddress` says, so *x*

wide and 2^z tiles tall, as the comment on `TileAddress` says, so *x* and *y* each run from 0 to $2^z - 1$.

TILES AT ONE LEVEL

$$n(z) = 2^z \times 2^z = 4^z \tag{12.1}$$

At level 12, 4^{12} = 16,777,216 tiles. At level 22, the highest Tessera serves (`Tiles.MaxZoom`), 4^{22} = 17,592,186,044,416.

Macau covers few of these tiles. The `Area` section of `tessera.json` gives Macau's box: longitude 113.52° to 113.61°, latitude 22.10° to 22.22°. Exhibit 12.2 counts the tiles over that box at each level, worked out with formula 12.2 below. Up to level 11 the box fits in one or two tiles a side, so the count grows slowly; from level 14 the factor of four takes over. The map asks for tiles up to the service's level 15 and draws higher levels by enlarging those. The limit comes from `TILE_MAX_LEVEL`, 16, in `web/src/map/style.ts`, less one, since MapLibre counts its 512-pixel tiles one level lower (mini-guide M2).

EXHIBIT 12.2 The tiles over Macau's box at each level, for one layer

LEVEL Z	COLUMNS	ROWS	TILES AT THIS LEVEL	ALL LEVELS 0 TO Z
12	2	3	6	22
13	3	4	12	34
14	5	7	35	69
15	10	13	130	199
16	18	25	450	649

Source: the `Area` section of `tessera.json`, put through formula 12.2 at each corner

Nine layers to level 15 make at most 9 × 199 = 1,791 tiles. The cover's tile of cafés, with 54 places and every column, holds 17,575 bytes (chapter 11). Nginx's store of 64 MB (`max_size=64m`) holds 64 × 1,048,576 = 67,108,864 bytes. That makes room for 67,108,864 ÷ 17,575 ≈ 3,818 tiles of that size, so every tile of every layer fits, twice over.

A point on the Earth becomes a tile through two formulas

Web maps flatten the Earth with Web Mercator (chapter 8), a projection: a rule for drawing the round Earth on a flat sheet. PostGIS calls it EPSG:3857, and `ST_TileEnvelope` uses it. A longitude becomes a column by a straight share of the world's width. A latitude becomes a row through a logarithm, which stretches the map towards the poles so that shapes keep their angles.

FROM DEGREES TO A TILE

$$x = \lfloor\, 2^z \times (lon + 180) \div 360 \,\rfloor \tag{12.2}$$

$$y = \lfloor\, 2^z \times ½ \times (1 - \ln(\tan\varphi + 1 \div \cos\varphi) \div \pi) \,\rfloor \tag{12.3}$$

where φ (Greek 'phi') is the latitude in radians, ln the natural logarithm, and ⌊ ⌋ the ***floor***: the whole number at or below, since a tile's number is the count of whole tiles to its left or above it.

The integration tests hold the same two formulas in C#, in `ServiceTests.TileOf`, which finds the tile a test asks for:

```csharp
var size = 1 << zoom;
var x = (int)Math.Floor((longitude + 180) / 360 * size);
var radians = latitude * Math.PI / 180;
var y = (int)Math.Floor((1 - (Math.Log(Math.Tan(radians) + (1 / Math.Cos(radians))) / Math.PI))
    / 2 * size);
```

The cover of this book names the tile `12/3339/1788`, which holds the map's home point in `tessera.json`. Exhibit 12.3 finds it from the home point's longitude, 113.55°, and latitude, 22.19°, step by step.

EXHIBIT 12.3 Worked example: the tile at level 12 for the home point, longitude 113.55°, latitude 22.19°

STEP	WORKING	RESULT
Tiles across at level 12	2^{12}	4,096
Share of the world's width	(113.55 + 180) ÷ 360	0.815417
Column	⌊ 4,096 × 0.815417 ⌋ = ⌊ 3,339.95 ⌋	**x = 3339**
Latitude in radians	22.19 × π ÷ 180	0.387289
tan φ + 1 ÷ cos φ	0.407889 + 1 ÷ 0.925937	1.487877
Natural logarithm, then ÷ π	ln 1.487877 = 0.397350; ÷ 3.141593	0.126480
Share of the world's height	(1 − 0.126480) ÷ 2	0.436760
Row	⌊ 4,096 × 0.436760 ⌋ = ⌊ 1,788.97 ⌋	**y = 1788**

Check it with any calculator set to radians, or in PostGIS: `ST_TileEnvelope(12, 3339, 1788)` *returns the tile's square*

Run the formulas backwards, and column 3339 covers longitudes 113.467° to 113.555°, while row 1788 covers latitudes 22.187° to 22.269°. The home point lies near the tile's south-east corner, 0.003° north of its southern edge, so the floor matters: rounding to the nearest whole number would turn 3,339.95 into 3340 and 1,788.97 into 1789, the tile one step south-east.

Inside a tile, PostGIS draws on a grid of 4,096 cells a side (`Tiles.Resolution`), with a buffer of 256 cells beyond each edge (`Tiles.Buffer`). `Queries.Tile` turns the buffer into the share `ST_TileEnvelope` takes, the parameter `@margin`: 256 ÷ 4,096 = 0.0625, one sixteenth. A tile at level z is 40,075,017 m × cos φ ÷ 2^z wide, where 40,075,017 m is the Earth's distance round the equator in Web Mercator. At the home point's latitude, 22.19°, a level-12 tile is about 9,059 m wide, and its buffer 566 m. A level-15 tile is about 1,132 m wide, and one cell 1,132 ÷ 4,096 ≈ 0.28 m, finer than any symbol needs: the 'within 30 cm' that the comment on `TILE_MAX_LEVEL` names.

Every tile number fits a 32-bit int

A C# `int` holds 32 bits, and one of them carries the sign, so it holds whole numbers up to 2^{31} − 1, 2,147,483,647. A `long` holds 64 bits, up to 2^{63} − 1. Chapter 2 shows how `TileAddress.Of` refuses any level above 30, so the shift `1 << z`, which works out 2^z, never grows too large for an `int`. Both 2^{22}, the columns at level 22, and 2^{30} fit; 2^{31} is one past the top. To count all the tiles at a level, a program would need `1L << (2 * z)`, a `long`. The service counts no tiles, and `src/` holds no `long`. Two appear in the

tests: `TileTests` folds random numbers onto a level with `1L << z`, and `ServiceTests` reads `count(*)` as a `long`, since PostgreSQL returns a count as a `bigint`, its 64-bit whole number.

Little's law turns a rate and a time into a count

Stand by a supermarket's tills. If 2 shoppers join the queues each minute and each spends 5 minutes queuing and paying, then on average 10 shoppers are at the tills. That is ***Little's law***, which John Little proved in 1961, and it holds for any steady system, whatever the pattern of arrivals.

> LITTLE'S LAW
>
> $$L = \lambda \times W \tag{12.4}$$
>
> where L is the average count inside the system, λ the arrivals each second, and W the average time each one spends inside.

Apply it to the pool: the 'system' is the set of busy connections, and each tile query stays inside for its query time, *S*. Phase 0 measured a tile query at 0.001 s. At 700 tiles a second, the load N-26's known limit names, $L = 700 \times 0.001 = 0.7$: on average less than one connection is busy. Two copies of the service hold 4 connections each (`Database.PoolSize`), so $N = 8$.

Turn the law round. With *N* connections busy all the time, the pool finishes at most $N \div S$ queries a second. That figure is the ceiling on its ***throughput***, the work it finishes each second. With 8 connections and 0.001 s, that is 8,000 a second, but only if each query has a processor to itself. The comment in `Database.cs` records a measured 2,408 tiles a second, on 14 October 2026, on a machine with two processors shared by the database, the service and the load tool. So the processors, not the connections, set the ceiling. That run is not in the repository; a load test kept there (chapter 11) would let anyone repeat it.

> THE CEILING, AND THE TIME INSIDE AT THE CEILING
>
> $$\lambda_{max} = min(N \div S,\ processors \div processor\ time\ a\ tile) \tag{12.5}$$
>
> At 2,400 tiles a second with all 8 connections busy, Little's law gives $W = 8 \div 2{,}400 \approx 0.0033$ s: each query does about 1 ms of work and waits the rest for a processor. More connections would only lengthen that wait, which is why the pool stays small.

As traffic nears the ceiling, waiting grows fast. The ***utilisation*** ρ (Greek 'rho') is the share of the ceiling in use, $\lambda \div \lambda_{max}$. The simplest model of a queue, one server with random arrivals, gives the time inside as a multiple of the bare query time:

> WAITING NEAR FULL LOAD (ONE-SERVER MODEL)
>
> $$W \div S = 1 \div (1 - \rho) \tag{12.6}$$
>
> At N-26's 700 tiles a second, $\rho = 700 \div 2{,}400 \approx 0.29$, 'about a third of capacity', and a request takes 1.4 times the bare time. At $\rho = 0.5$ it takes twice as long; at 0.8, five times; at 0.9, ten times.

The pool's 3-second wait, `Database.ConnectionWaitSeconds`, sets how long a queue may form before the service answers 503. A queue of *Q* requests empties at the ceiling's rate, so the last waits $Q \div \lambda_{max}$. The door caps each office machine at 32 requests at once (`limit_conn connections 32`), and ADR 0007 plans for up to 20 machines, so at most $20 \times 32 = 640$ requests wait. At 2,400 a second they clear in $640 \div$

2,400 ≈ 0.27 s, far inside 3 s. Now put $\lambda_{max} = 8 \div S$ into $Q \div \lambda_{max} \le 3$ and rearrange: the queue clears in time while $8 \div S \ge 640 \div 3$, that is, while the average query stays under $8 \times 3 \div 640 = 0.0375$ s, 37.5 ms.

The database must accept every connection the pools may open, with room for more, and `max_connections=20` in `db/Dockerfile` leaves that room. PostgreSQL keeps 3 connections for its superusers, the roles that hold every right (`superuser_reserved_connections`, which the image leaves at PostgreSQL's default, 3). So 20 − 8 − 3 = 9 remain for the seed, `npm run publish` and `psql`. A third copy of the service would leave 5.

The rate limit is a leaky bucket

Nginx's `limit_req` follows the ***leaky bucket***. Picture a bucket for each office machine, with a hole in its base. Each tile request pours in one cup. The hole lets out 100 cups a second (`rate=100r/s`), one every 10 ms. The bucket holds 300 cups above the steady flow (`burst=300`), and `nodelay` serves those at once rather than spacing them out. A request that would overflow the bucket receives 429.

> THE MOST ONE MACHINE MAY SEND
>
> $$A(t) \approx B + r \times t = 300 + 100\,t \tag{12.7}$$
>
> over any stretch of t seconds that starts with an empty bucket. In one second, about 400 tiles; in a minute, about 6,300; over an hour, the burst hardly counts, and the rate is 100 a second.

The numbers fit the map. A screen 1,920 pixels wide and 1,080 tall shows about $4 \times 3 = 12$ of MapLibre's 512-pixel tiles. Each of the nine layers asks for its own, so a fresh view asks for about $9 \times 12 = 108$ tiles: the 'about a hundred' that ADR 0007 and S-30 name. A full bucket holds $\lfloor 300 \div 108 \rfloor = 2$ whole fresh views at once, and drains one view's tiles in $108 \div 100 \approx 1.1$ s. Three views at once overflow it: about 300 tiles pass at once, the rest get 429, and the bucket is empty again three seconds later.

The rate also sizes the office. If every machine asked at the full rate and no store held a tile, $2{,}400 \div 100 = 24$ machines would fill the database, as S-30 and ADR 0007 note. ADR 0007 asks for new numbers once the office passes 20 machines. The second cap, 32 requests at once for each machine, follows from Little's law: at 100 tiles a second, 32 open at once means each takes $32 \div 100 = 0.32$ s. That cap matters only when tiles turn slow, as the edge check shows with forty requests for a tile that takes two seconds: at least eight receive 429.

Each refusal tells the browser when to return, in whole seconds. A 429 from the bucket carries `Retry-After: 1`: in one second the hole lets out 100 cups, room for nearly a whole view. A 503 carries `Retry-After: 5`, set both by Nginx's `@unavailable` and by `Problems.RetryAfterSeconds` in the service. A database that has stopped needs longer than a second to return, and until the first list of layers loads, the service itself tries again every 5 seconds, the same constant (N-23).

Every timeout nests inside the one outside it

A request crosses several layers, and each layer keeps a clock. If an outer clock runs out first, the outer layer gives up while the inner one still works, and the work goes to waste; so each inner limit must be shorter than the one around it, with room to spare.

THE SERVICE'S WORST CASE INSIDE NGINX'S WAIT

$$T_{wait} + T_{query} < T_{proxy} \;:\; 3 + 10 = 13 < 20\ s \tag{12.8}$$

T_{wait} is `ConnectionWaitSeconds`, T_{query} is `QueryTimeoutSeconds`, and T_{proxy} is `proxy_read_timeout`, whose comment states this sum. With 7 seconds to spare, the service answers its own 503 first, and only a copy that hangs meets Nginx's limit.

The sum holds because the service keeps it on a clock of its own. Npgsql's limits alone overran it on a frozen database, answering after 16 seconds, so `Database.Run` now gives up at $T_{wait} + T_{query}$ itself (chapter 6). Measured with the database paused, the answer came in 13.0 to 14.1 seconds, which leaves 20 − 14.1 = 5.9 seconds for a busy machine.

Nginx's crowd lock waits as long as the first request may take: `proxy_cache_lock_timeout` is 20 seconds, the same as `proxy_read_timeout`. A waiting request therefore never sends a second query to the database while the first may still finish. The health check keeps a shorter clock of its own: `Database.Ping` gives up after one second, so `/health` answers well inside every other limit.

The backup keeps fourteen nights, which span thirteen days

A nightly task runs the backup at 02:00, and `backupsToRemove` in `scripts/lib/backup.ts` picks the backups to delete so that the newest $K = 14$ stay (`defaultKeep`). It sorts the backups' names and removes the first $n - K$, where n counts the backups in the folder, or none when $n \le K$.

Fourteen backups have thirteen gaps between them, so the oldest is 13 days older than the newest: a ***fencepost*** count, like the fence of 14 posts that has 13 panels. You can undo a mistake found on a Monday by restoring any night of the last 13 days. The sort works because of the names' shape: the backup taken at 02:00 in Macau on 18 October is `tessera-2026-10-17T1800Z.dump`, since 02:00 there is 18:00 UTC the day before. The name writes the year, month, day, hour and minute in fixed widths, largest first and in UTC, so letter order is time order. A name such as `9-10-2026` would sort after `17-10-2026`, since the character 9 comes after 1.

The coverage floors leave a small margin

The build fails below 94 per cent of the lines or 84 per cent of the branches (chapter 10), and an inequality shows what each floor allows. The run of 18 October 2026 reached 648 of the service's 674 lines, 96.1 per cent, across the three test projects. Suppose a change adds n lines that no test runs.

UNTESTED LINES THE FLOOR OF 0.94 ALLOWS

$$648 \div (674 + n) \ge 0.94 \;\Leftrightarrow\; n \le 648 \div 0.94 - 674 \approx 15.4 \tag{12.9}$$

So 15 new lines without a test would still pass; the 16th would fail the build. The branches leave less: the same run reached 267 of 312, and $267 \div (312 + n) \ge 0.84$ gives $n \le 267 \div 0.84 - 312 \approx 5.9$, so 5 untested branches pass and the 6th fails.

`scripts/lib/coverage.ts` cuts each share to one decimal place rather than rounding it, so a share just under a floor never rounds up to pass. A floor close below today's figure catches a change that adds untested code, yet lets a small fix through. Set it at today's figure, and the next honest change that removes a tested line fails the build; set it far below, and it guards nothing. Tessera's floors sit about two points

below the figures, and its comment says to raise a floor when the figures rise and never to lower one to pass a change.

To adjust a limit, follow its arrows

Exhibit 12.1 also says what to change. When a measurement moves, recompute the box it feeds and every box its arrows lead to, and leave the rest alone.

EXHIBIT 12.4 What to recompute when a number changes

WHEN THIS CHANGES	RECOMPUTE	THEN CHECK
Query time *S*, after new data or a lost index	The ceiling (12.5) and the utilisation (12.6)	The queue still clears in 3 s while *S* stays under 37.5 ms
Processors, on a new office machine	The ceiling, then the machines that fill it	The pool: more connections help only with more processors
A third copy of the service	$N = 12$	`max_connections`: 12 + 3 leaves 5 for the rest
`QueryTimeoutSeconds` or `ConnectionWaitSeconds`	Their sum (12.8)	It stays under `proxy_read_timeout`, and the lock's timeout with it
More layers, so a bigger view *V*	Views in one burst, $\lfloor 300 \div V \rfloor$	The burst still holds at least one view, and the store still holds every tile

Source: the formulas of this chapter, with `tessera.json`, `nginx/default.conf.template` *and* `db/Dockerfile`

> TRY IT Find the tile of your own school at level 15 with formulas 12.2 and 12.3, on paper and a calculator. Then open `https://tile.openstreetmap.org/15/x/y.png` with your *x* and *y* in place, and check that the picture shows your school. Which way does *y* grow, and why does the formula need ⌊ ⌋ rather than rounding?

> COMMON MISTAKE Sizing the pool for the visitors. Twenty machines can share eight connections: connections beyond the processors only wait for a processor, as formula 12.5 shows. Size the pool for the processors, and let the door and the queue hold the crowd.

Check your understanding

1. Which tile at level 1 holds Macau, and why?
2. 300 tile queries a second each take 20 ms. How many connections are busy on average?
3. A visitor's bucket is empty. How many tiles can the visitor receive in the next two seconds?
4. If `QueryTimeoutSeconds` rose to 18, what would go wrong, and what else must change?

Answers. 1. 1/1/0: Macau lies east of 0° longitude, so in the right-hand column, x = 1, and north of the equator, so in the top row, y = 0. 2. L = 300 × 0.020 = 6. 3. About 300 + 100 × 2 = 500 (formula 12.7). 4. 3 + 18 = 21 s no longer stays under Nginx's 20 s. Nginx could give up while the service still worked, count a timeout against that copy, and send the request to the other copy (`proxy_next_upstream error timeout`), which runs the query again. Raise `proxy_read_timeout` and `proxy_cache_lock_timeout` above 21 s, with room to spare, or shorten the wait.

THINK IT OVER Every formula here rests on one number, the query time, measured once in phase 0 on another set of tables, and on one load run that the repository does not keep. When should Tessera measure them again, and which test would tell you that they have changed?

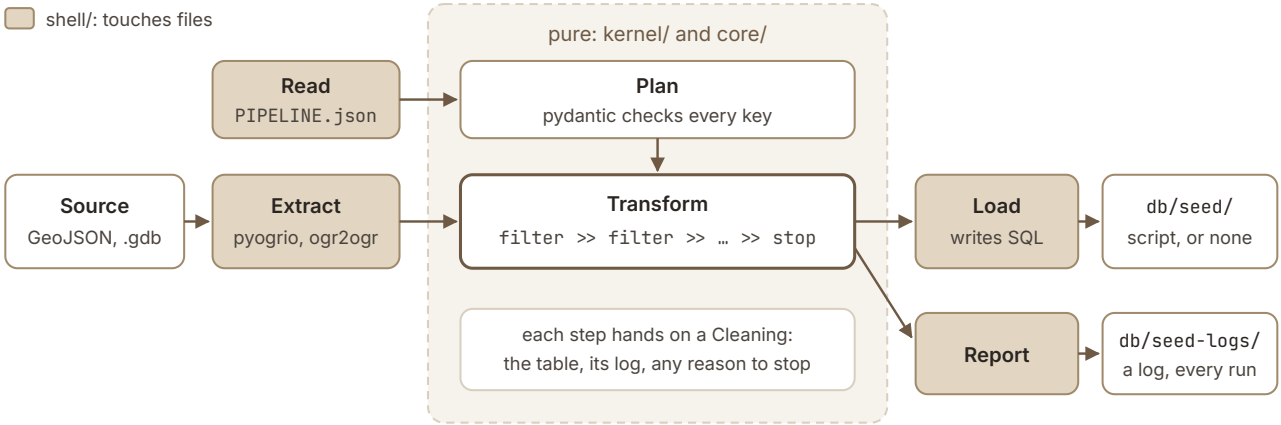
M1
The Python Pipeline

The pipeline cleans a file of places through a chain of pure steps, writes one SQL seed script, and logs what it did to every row. It lives in `etl/` and needs Python 3.13 or later. `npm run etl` starts it; DbUp later runs the script it wrote, once. Tessera shares the pipeline with JTessera, its Java twin; only the library that runs the scripts differs.

Extract, transform, load: the world touches only the edges

The pipeline does three jobs in order, a pattern called ***ETL***. It **extracts** each table from the source, **transforms** the rows by cleaning them, and **loads** the surviving rows as SQL. GDAL, a free library for map data, reads geodatabases, GeoJSON, shapefiles and more. pyogrio, a Python library built on GDAL, reads a table into a ***GeoDataFrame***: a table of rows with one column of shapes. When a table holds true curves, fields coded by text or another coordinate system, GDAL's own program, `ogr2ogr`, first copies it into a GeoPackage, a map format held in one file.

EXHIBIT M1.1 One run of the pipeline: only the shaded stages touch files or run programs



Source: `docs/design/etl-pipeline.md` ; `etl/src/tessera_etl/`

As Exhibit M1.1 shows, everything between Extract and Load is pure: it reads no file and runs no program, so the tests check every cleaning rule with a table made in memory.

Eight filters clean a table, and stop rules guard it

A pipeline file, such as `data/macau.pipeline.json`, names three things: the source, the script to write, and each table with its filters in order. Each filter works on the rows the one before it kept.

FILTER	A ROW FAILS, OR IS FIXED, WHEN
`dropEmptyShapes`	it has no shape, or an empty one: it fails
`repairShapes`	its shape is invalid, such as an edge that crosses itself: it is rebuilt, or fails if it comes out empty

`swapCoordinates`	it lies outside the area, but would lie inside with longitude and latitude swapped: they are swapped back
`insideArea`	its shape does not touch the area's box: it fails
`tidyText`	its text has stray spaces or mixed Unicode forms: it is tidied, and empty text becomes null
`requireFields`	a named field is missing or blank: it fails
`dropDuplicateIds`	it repeats an earlier row's id: it fails
`dropNearDuplicates`	its name matches an earlier row's, and the two lie within the given number of metres: it fails

The area that `swapCoordinates` and `insideArea` need comes from the pipeline file's own `area`. Without one, it comes from the `Area` section of `tessera.json`, the service's settings file, when that file lies in the folder where the run starts. One box then serves both the map and the cleaning.

The pipeline drops and logs each failed row. Under `"onFail": "stop"`, a failed row ends the whole run instead. Two ***stop rules*** then guard each table. The run stops when the filters dropped more than `droppedOverPercent` per cent of the table's rows (10 by default, 5 in the Macau file), or when fewer than `rowsUnder` rows remain (1 by default). A stopped run writes no script, but it still writes its log:

```text
== food.cafes, where layer = 'cafes'
Read 100 rows.
  tidyText: kept 100, fixed 0, dropped 0
  insideArea: stopped, fixed 0, dropped 0
Stopped; nothing kept.
  Row 1: stopped by insideArea: its shape lies outside the area
```

Steps join with >>, which stops at the first stop

A ***step*** is a function that takes a context and returns a changed copy of it. The context, `Cleaning`, holds the table, its log and any reason to stop. The kernel borrows Python's shift operator, `>>`, to mean 'then', as the language F# uses `>>` to join two functions:

EXHIBIT M1.2 The whole kernel: a step, and the operator that joins two

```python
@dataclass(frozen=True, slots=True)
class Step[C: Stoppable]:
    apply: Callable[[C], C]

    def __call__(self, context: C) -> C:
        return self.apply(context)

    def __rshift__(self, then: "Step[C]") -> "Step[C]":
        def both(context: C) -> C:
            done = self(context)
            return done if done.stop is not None else then(done)
        return Step(both)
```

Source: `etl/src/tessera_etl/kernel/pipes.py`

In Exhibit M1.2, `a >> b` runs `a`, then runs `b` on what `a` left, unless `a` stopped. The function `chain`, in `core/steps.py`, folds a list of steps into one with `reduce(operator.rshift, steps, IDENTITY)`: it starts from `IDENTITY`, the step that changes nothing, and joins on each step in turn. An empty list therefore still

makes a chain. Each filter is a ***judge***: a function that reads the whole table at once and gives a verdict, `Fixed` or `Failed`, on each row it would change or refuse. `filter_step` turns any judge into a step, so every filter drops, logs and stops in the same way.

> FOUNDATIONS A step is a subroutine with one parameter and one return value. Joining two steps is ***function composition***: $(g \circ f)(x) = g(f(x))$. The filters do the **validation** you meet at school: presence, range and type checks.

A pure core and a thin shell, with a tool to guard the border

The code lives in three packages, and each may use only the packages below it. `kernel/` holds `Step` and `Result`, and knows nothing of places. `core/` holds every rule, with each filter in a module of its own in `core/filters/`. `shell/` reads files, runs pyogrio and `ogr2ogr`, and prints. import-linter, a tool that checks which module may import which, enforces three rules, or contracts, set in `etl/pyproject.toml`. The kernel and the core may not import pyogrio, `os`, `subprocess` or the shell, among others. The kernel may import no table and no shape. No package may call the one above it.

pydantic, a library that checks data against a declared shape, reads the pipeline file into frozen models, which nothing can change once made. Each model sets `extra="forbid"`, so it refuses a misspelt key. The reader names every fault at once, before it reads any table.

Five tools check the pipeline

pyright checks every type, in its strict mode. Each `match` on a verdict ends with `assert_never`, so pyright refuses a `match` that misses a case. **ruff** checks the style. **import-linter** checks the three contracts. **pytest** runs 253 tests, 128 of them on golden files, which hold each case's recorded result (chapter 10). Among these are 27 pipeline files whose exit code, output, script and log must match byte for byte, and 96 faulty pipeline files with the message each must give. **hypothesis** makes up inputs and shrinks any failure to its smallest form. It checks, for instance, that `(a >> b) >> c` runs as `a >> (b >> c)` does. `npm test` runs all five tools.

From a new source to the database in four steps

1. `npm run inspect -- data/parks.gdb` describes each layer and prints a starting pipeline file, which the maintainer saves and edits.
2. `npm run etl -- data/parks.pipeline.json` runs the pipeline through uv, a tool that runs Python projects (`uv run --project etl tessera-etl`). It ends with exit code 0 when it wrote the script, 1 when a table stopped or failed, and 2 when the command, the file, the source or the script's name is wrong.
3. The maintainer reads the script and its log, in `db/seed-logs/`, and commits both: together they record what entered the database, and what was left out and why. The repository holds only `0001-macau-places.sql`; the cleaned Macau script, `0002-macau-places-cleaned.sql`, and its log appear once `npm run etl` writes them.
4. `npm start` builds the service, which copies every script in `db/seed` beside the program, then runs `Tessera seed`. DbUp runs each script it has not run before, once, in the order of the scripts' names, and records it in the table `tessera.schemaversions` (chapter 7). `npm run publish` then grants the tables to the service.

> WATCH OUT DbUp knows a script by its name alone. Change the SQL of a script that has run, and DbUp never runs it again: the database keeps the old rows and gives no warning. The pipeline therefore refuses to write a script whose name exists. New or corrected data goes in a new script with the next number.

> TRY IT Run `npm run etl -- data/macau.pipeline.json`, then open the log it writes in `db/seed-logs/`. For each table, which filter dropped or fixed the most rows, and why? Then run the same command again and read the message and the exit code. What stops the second run, and what would you change to load corrected data?

Check your understanding

1. Why can the tests check every filter without GDAL or a file?
2. A filter set to `"onFail": "stop"` fails row 7. Which later steps run, and what does the run write?
3. A pipeline file spells `fields` as `feilds`. What stops the mistake, and when?

Answers. 1. The filters live in `core/`, which may touch no file; a test hands each a table made in memory. 2. None: `>>` runs no step after a stop. The run writes its log, naming row 7, but no script. 3. pydantic forbids keys a model does not name, so the run names the fault before it reads a table, and ends with exit code 2.

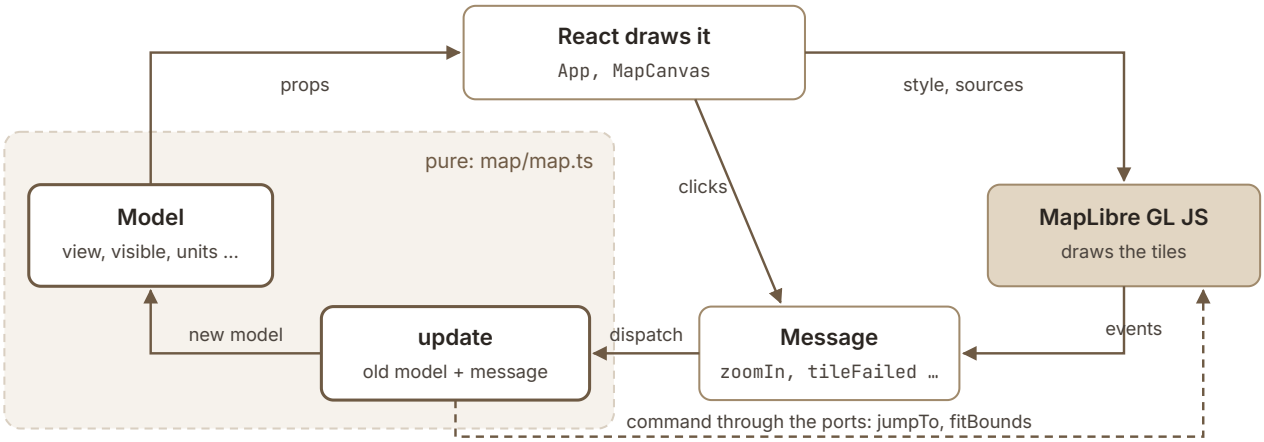
M2
The Map in the Browser

In the map, one pure function decides everything the page shows, and a thin shell hands each decision to MapLibre GL JS, which draws the vector tiles. The map lives in `web/`: about 3,200 lines of TypeScript 6 beside its tests, with React 19, a library that builds a page from parts. Vite, a build tool, packs the map into `web/dist`. `npm run build` makes that folder first; `dotnet build` then copies it beside the program as `wwwroot/`, and the service serves it from there. This guide shows four things: the loop at the map's heart, its checks on data from the service, its clustering of places, and its tests.

One model, a closed set of messages, and a pure update

The map follows the ***Elmish pattern***, named after the Elm language, through the library react-elmish. The pattern has four parts, all in `web/src/map/map.ts`. The ***model*** is one read-only value that holds everything the page shows: the view, the layers switched on, the units of the scale bar, the pointer's position, any notice and two flags. A ***message*** is a value that says what happened, such as a click. `init` makes the first model. `update` makes the next model from the old one and a message, and changes nothing else.

EXHIBIT M2.1 The Elmish loop: React draws the model, events become messages, and update makes the next model



Source: `web/src/map/map.ts`, `web/src/app/App.tsx`, `web/src/map/map-handle.ts`

As Exhibit M2.1 shows, `update` never touches MapLibre. It returns any work outside the model, such as moving the map, as a ***command***. A command goes out through one of two ***ports***, `jumpTo` and `fitBounds`: functions passed in from outside, so a test can pass fakes. One type lists every message, so the messages form a closed set, and react-elmish's `UpdateMap` type demands one handler for each message's name:

```ts
export type Message =
  | { name: "zoomIn" }
  // …
  | { name: "toggleUnits" }
  // …
  | { name: "tileFailed"; problem: Problem }
  // …
export const update: UpdateMap<Props, Model, Message> = {
  // …
  toggleUnits(_msg, model) {
    return [{ units: model.units === "metric" ? "imperial" : "metric" }];
  },
  tileFailed({ problem }) {
    return [{ notice: noticeOf(problem) }];
  },
  // …
```

Each handler returns the fields that change, and a command where one is due. Delete a handler and the program stops compiling.

> FOUNDATIONS The loop is selection on a record. A message is a record with a `name` field, and `update` picks one branch for each name, as a `CASE` statement picks one for each value. The way the pins drop onto the map, in `map/landing.ts`, is a ***state machine***, like the state-transition diagrams you draw at school: three stages, five events, and a pure function that gives the next stage.

Five modules, each importing only those before it

Each folder of `web/src` is a module, and its `index.ts` is its public face: the one file that other modules may import. `shared/` holds plain values such as `Result` and the area; `design/` the colours, pins and motion; `layers/` the nine layers and each tile's address; `map/` the model, the update and the shell around MapLibre; and `app/` the page. A module may import only the modules before it in that list; `src/architecture.test.ts` fails the build when one breaks the rule.

Valibot checks data at the door, and ts-pattern checks every case

The map checks data from outside once, where it enters. Valibot, a library that checks data against a description of its shape, reads two things the service sends. `areaOf`, in `shared/area.ts`, reads the area the service writes into the page. It refuses a blank name, a box whose west edge lies at or beyond its east edge (or its south edge at or beyond its north), and a home level outside 0–22. On any of these faults, the map falls back to Macau. `readProblem`, in `shared/problem.ts`, reads an error answer as an RFC 9457 problem document. Each returns a `Result` instead of throwing an exception.

ts-pattern, a library that matches a value against a list of cases, covers the rest with its `match`. A `match` that ends with `.exhaustive()` compiles only while it handles every case. The map draws three kinds of image: a pin, a cluster and a tooltip. Add a fourth kind, and every such `match` on the kinds stops compiling until it handles the new one.

The browser clusters the places from the tiles it has loaded

To cluster places is to draw a crowd of nearby places as one circle with a count. MapLibre can cluster only a GeoJSON source, but the service sends vector tiles, so the map bridges the two (ADR 0005). It loads one vector source for each layer switched on. It asks for new tiles up to web-map level 16, which matches the service's level 15, since MapLibre's tiles are 512 pixels wide (chapter 12). `feedPlaces` reads the places of every loaded tile. `placeCollection`, a pure function, keeps each place once by its `id`, since a place near a tile's edge arrives in two tiles. A clustered GeoJSON source, with a radius of 48 pixels, then groups the places up to level 15. Each new set of data makes MapLibre cluster again, so the feed sets new data only when the places' fingerprint (their ids joined into one string) changes. The cost is an edge effect: a cluster at the screen's edge counts only the loaded tiles, and grows as the visitor pans.

Three rules bind the map to the service

ADR 0006 sets a small contract: any service that keeps it can serve the map unchanged.

1. **Tiles on the page's own origin,** the same address and port as the page. `/{layer}/{z}/{x}/{y}.pbf` answers with a vector tile, and every error as a problem document. The map names its nine layers in its own code and loads no list.
2. **The page, with the area written in.** `/` and `/{layer}.html` answer the map's `index.html`, with one `<script id="tessera-area">` block added before `</head>`, so the map needs no extra request to learn its area.
3. **The built files beside the page.** They lie under `/assets/`, and the browser keeps each for a year, since a file's name changes whenever its content does. `Tessera.csproj` copies them into the release, and `MapRoutes.Render` writes the area's block into the page.

Vitest checks the rules; Playwright checks the page

Four kinds of test check the map's rules, and each test names its story, such as UI-17. Vitest, a test runner for TypeScript, runs three of them: **unit tests** beside the code, which check the pure parts with values alone; **page tests** in jsdom, a pretend browser without WebGL, the browser's way to draw with the graphics card, where `testing/fake-maplibre.ts` stands in for MapLibre; and **whole-code tests**, which read the source as text to check the module walls and the size of the built files. Playwright, a tool that drives a real browser, runs the fourth kind, the **browser tests**, in Chrome, in a fixed window of 1,280 by 800 pixels. Chrome draws in software, with SwiftShader, and loads the map through Nginx from two copies of `Tessera.dll`. In `web/`, `npm test` runs the type check, ESLint (a style checker) and Vitest. At the root, `npm run e2e` runs the browser tests; with `TESSERA_PLAYWRIGHT_IN_DOCKER=1`, as the GitHub build sets it, Playwright runs inside its own Docker image, whose Chromium and fonts draw UI-17's approved images alike on any machine.

> TRY IT Start Tessera with `npm start`, and set `TESSERA_BASE_URL` to one of its copies, such as `http://127.0.0.1:7800`, so the development server passes tile requests on to it. In `web/`, run `npm run dev` and open `http://localhost:5173` with the browser's Network tab open. Zoom in, and watch each `.pbf` tile arrive. Which level is the last to bring new tiles?

Check your understanding

1. Why does `update` move the map through a port rather than call MapLibre itself?
2. The service writes an area whose box has its west edge east of its east edge. What does the page show?
3. Why must every tile carry each place's `id`?

Answers. 1. So `update` stays pure: it returns a command, and a test passes fake ports and checks the command with values alone. 2. Valibot refuses the area, and `areaOf` falls back to Macau. 3. A place near a tile's edge arrives in two tiles; `placeCollection` keeps it once by its `id`, so no place is counted twice in a cluster.

M3
The Scripts

Every script splits in two: a thin file in `scripts/` runs the commands, and a file in `scripts/lib/` makes every decision as a pure function, which `node --test` checks in seconds. The scripts are TypeScript, run by Node.js 26 with Google's zx: about 2,100 lines in all. `package.json` holds fourteen `npm` commands. Twelve each start one entry script; the other two, `typecheck` and `test:scripts`, check the scripts themselves. A thirteenth entry script, `boot.ts`, runs when Windows starts, and a fourteenth, `coverage.ts`, runs inside `npm test`. Eleven helper files in `scripts/lib/` serve them all, and nine test files check them: one beside each of the eight pure helpers, and one beside `lib/build.ts`.

TypeScript with zx, because one language serves the map and the scripts

ADR 0002 chose TypeScript 6 over Bash, the usual language of Linux scripts, and over C# files run by `dotnet run`, for three reasons. The map is TypeScript already, so the scripts share its language and its tools. Types catch a wrong path before a run on the office machine. Pure functions let a test check each decision without a shell. Google's ***zx*** keeps the ease of a shell all the same. Its `$` runs a command written as a template string, a string with values set inside `${ }`, and it quotes each value, so a password with a space cannot split a command.

Node.js 26 runs each `.ts` file by stripping out its types. `tsconfig.json` therefore sets `erasableSyntaxOnly`, so the type check, `tsc --noEmit`, refuses any syntax that would leave code behind once the types go, such as `enum`.

The pattern: the edge asks, the core decides

Each script reads its inputs, hands them to pure functions, and runs commands only at the end. `office-port.ts` shows the pattern at its smallest. It asks the machine whether it runs WSL2, and which networking mode WSL2 uses, then prints what `networkingSteps` and `firewallSteps`, in `lib/office.ts`, return. Those two take text and return text, so their test needs no Windows. The decisions behind `npm start` live in `lib/plan.ts`. One of them is how long to wait before starting a stopped copy again:

```ts
export function restartDelay(quickFailures: number): number {
  return Math.min(30_000, 1000 * 2 ** Math.max(0, quickFailures - 1));
}
```

After 0, 1, 2, 3, 4 and 5 quick failures, the wait is 1, 1, 2, 4, 8 and 16 seconds. The wait doubles each time up to a cap of 30 seconds, so a copy that cannot start still rests between tries.

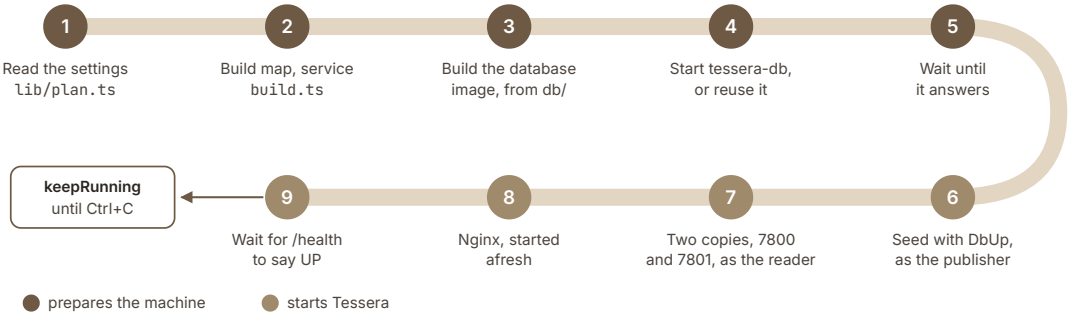
> TRY IT In `scripts/lib/plan.test.ts`, the test of `restartDelay` lists eight counts of quick failures and the wait for each. Add `-1` to the list, with the wait you expect, and run `npm run test:scripts`. Explain the answer from `Math.max`, then undo the change.

> FOUNDATIONS Testing `restartDelay` is testing with **normal, boundary and erroneous data**. Normal: 3 quick failures give 4 seconds. Boundary: 0, the fewest, gives 1; 6 give 32, which the cap cuts to 30. Erroneous: a negative count, which the Try it above asks you to test.

npm start runs nine steps, then keeps the copies alive

`npm start` runs `scripts/start.ts` with `--env-file-if-exists=.env`, so `.env` supplies its passwords. Its nine steps run in a fixed order, since each needs the one before it.

EXHIBIT M3.1 The nine steps of `npm start`, in the order `start.ts` runs them



Source: `scripts/start.ts`

As Exhibit M3.1 shows, the build comes first, so the release carries the map and every seed script; it empties the release's map folder before the copy, so no old map file ships beside the new ones. The seed, `dotnet Tessera.dll seed`, runs at every start, but DbUp runs only the scripts it has not run before, so a second start leaves the database unchanged. zx keeps what the seed prints to itself, so `start.ts` prints the seed's report, the scripts it ran or that it had none to run, through `seedReport` in `lib/plan.ts`. Nginx starts afresh each time, since an old container may hold old ports.

The last step never ends. `keepRunning`, in `lib/supervise.ts`, runs one copy until told to stop. A copy that stops on its own starts again after `restartDelay`. A copy that ran for over a minute counts as healthy, so its next failure waits one second again. The loop takes the outside world as parameters, so its test passes in fake copies and a fake clock: dependency injection by hand (chapter 5).

Each other script does one job

Three more scripts publish the places and keep them safe.

EXHIBIT M3.2 The other scripts, what each does, and where its decisions live

COMMAND	WHAT IT DOES	PURE PARTS
`npm run publish`	Grants the four schemas and every table in them to `tessera_reader`, as `tessera_publisher`	`lib/plan.ts`
`npm run backup`	Writes the database with `pg_dump` to a file ending `.part`, reads it back with `pg_restore --list`, and renames it once it proves whole; keeps the newest 14	`lib/backup.ts`
`npm run restore -- FILE`	Puts a backup back with `pg_restore`, in one transaction, so a failed restore changes nothing	`lib/plan.ts`

Source: `package.json`; `scripts/`

Unlike `npm start`, whose line in `package.json` passes `--env-file-if-exists=.env`, `backup.ts` and `restore.ts` read `.env` themselves, since the nightly task starts `backup.ts` without `npm`. Exhibit M3.2 leaves out the scripts that other chapters cover: `inspect` and `etl` (M1), `office-port`, `boot-task` and `backup-task` (M4), and `build`, `test` and `e2e` (chapter 10).

Check your understanding

1. Why may a script use no `enum`?
2. A copy fails five times in a row, each within a minute. How long does the loop wait before the sixth start?
3. Why does `npm run boot-task` print PowerShell instead of running it?

Answers. 1. Node.js only strips types. An `enum` makes code, which stripping cannot remove, so `erasableSyntaxOnly` makes `tsc` refuse it. 2. `restartDelay(5)` is 1,000 × 2^4 = 16,000 milliseconds: 16 seconds. 3. Registering a Windows task needs an administrator, and the script runs inside Ubuntu as an ordinary user.

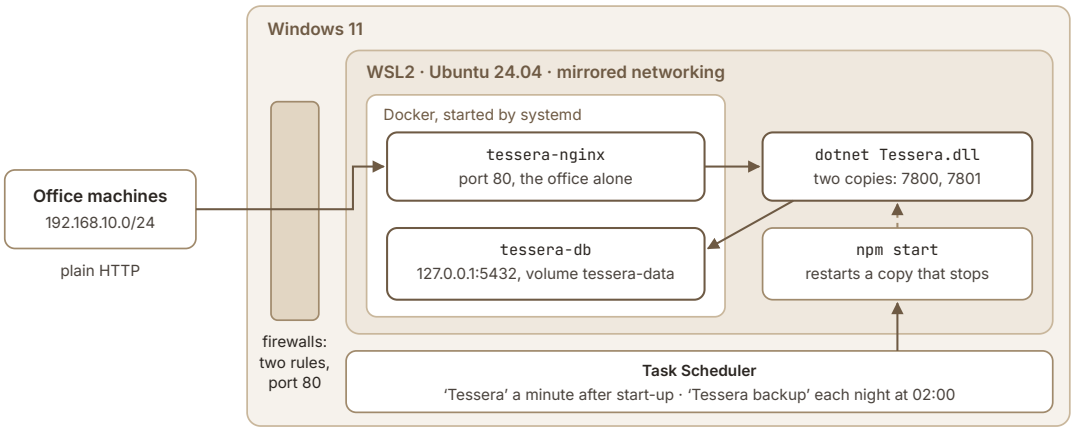
M4
The Office Machine

Tessera runs on one Windows 11 machine: inside Ubuntu 24.04 on WSL2, Docker holds the database and Nginx, two copies of the .NET program serve the tiles, and two Windows tasks start it all at boot and back up the database each night. Each part comes back by itself after a failure or a restart. This guide shows how the parts nest, how a request from the office reaches port 80, what the two tasks do, and where the settings live. Tessera has no runbook yet, a guide to running it day by day: the scripts' own comments and the delivery plan, `docs/guides/delivery-plan.md`, serve in its place.

Four parts nest inside one machine

WSL2, the Windows Subsystem for Linux, runs a real Linux kernel, the heart of the operating system, in a light ***virtual machine***. A virtual machine is a program that behaves as a whole computer, as an emulator lets one console's games run on another. Ubuntu 24.04 runs in it. Inside Ubuntu, Docker Engine runs two containers: the database and Nginx. Beside them run the two copies of the service, each started by `npm start` as `dotnet Tessera.dll`.

EXHIBIT M4.1 The office machine, from the office's request to the database



Source: `scripts/start.ts`, `scripts/lib/plan.ts`, `scripts/lib/boot.ts`, `scripts/lib/backup.ts`

As Exhibit M4.1 shows, each part has its own way of coming back. Docker restarts both containers after a crash or a reboot, since `npm start` runs them with `--restart unless-stopped`. `npm start` restarts a copy that stops. The start-up task starts `npm start` itself after each reboot.

> FOUNDATIONS Operating systems and networks. An operating system shares one machine's hardware between programs; a virtual machine runs a whole operating system as one of those programs. A home router uses network address translation (NAT) to let many devices share one public address, so a server beyond the router sees one address for them all.

Mirrored networking lets Nginx see who asks

By default, WSL2 gives Ubuntu an address of its own, hidden behind Windows, as a home router hides the devices behind it: this is NAT mode. Tessera needs ***mirrored networking***, in which Ubuntu shares the Windows machine's own address. A request to the Windows machine on port 80 then reaches Nginx directly, and Nginx sees the address of the office machine that sent it. Nginx's office-only rule and its limit of 100 tiles a second for each machine (chapter 4) both need that address.

`npm run office-port` checks the mode and prints the lines that switch it on. It then prints the two firewall rules, Windows' and Hyper-V's, that let port 80 in. It prints these lines instead of running them, since each needs Windows and an administrator.

> TRY IT From another office machine, ask for the example tile through Nginx: `curl -I http://OFFICE-MACHINE/food.cafes/12/3339/1788.pbf`, with the office machine's name in place of `OFFICE-MACHINE`. The tile holds 54 of the seed's 100 cafés: 39 inside its square, around the map's home point, and 15 in the strip beyond its edges (chapter 8). Then, on the office machine, run `docker logs --tail 5 tessera-nginx`. Whose address does the last line show: the sender's, or the office machine's own?

systemd starts Docker, so the start-up task can start Tessera

systemd is the first program Ubuntu runs, and it starts the others. WSL2 starts systemd only when `/etc/wsl.conf` holds `[boot]` and `systemd=true`. `sudo systemctl enable docker` then makes Docker start with Ubuntu. `npm run boot-task` checks both settings first.

It then prints the PowerShell, Windows' own command language, that registers the ***start-up task***, named 'Tessera'. Windows runs the task a minute after start-up, whether or not anyone signs in, and if it fails, tries again up to three times, a minute apart. The task starts Ubuntu and runs `scripts/boot.ts`. That script moves a `start.log` over 10 MB aside to `start.log.1`, so the log cannot fill the disk, then runs `npm start`'s script and logs to `start.log`. The task names the repository's folder and the Node.js program, so after moving either, run `npm run boot-task` again.

A nightly task backs up the database

`npm run backup-task` prints the PowerShell for a second task, 'Tessera backup', which runs `scripts/backup.ts` each night at 02:00, or as soon as the machine switches on after a night it was off. Chapter 7 follows one night's backup and shows how it proves each file whole before keeping it. The backup holds every table, every grant and DbUp's record of the seed scripts, so a restore leaves the next `Tessera seed` nothing to run twice.

.env holds the passwords, and tessera.json holds the rest

Settings come from two places. Machines, ports and passwords come from environment variables (chapter 5), which `.env` supplies; the service's other settings live in `tessera.json`. Copy `.env.example` to `.env` and fill in each password; Git ignores `.env`, so no password reaches the repository.

The file names five settings to fill in: `TESSERA_DB_PASSWORD`, the database owner's password, for `npm start` alone; `TESSERA_READER_PASSWORD` and `TESSERA_PUBLISHER_PASSWORD`, for the service's role and the seed's; `TESSERA_HTTPPORT`, 80; and `TESSERA_OFFICE_NETWORK`, such as `192.168.10.0/24`. The rest are optional: `TESSERA_SERVICE_PORT`, the first copy's port, 7800 by default; where the database answers; and

`TESSERA_BACKUP_FOLDER` and `TESSERA_BACKUP_KEEP`, `backups/` and 14 by default. From the passwords, `npm start` writes the two database addresses the program reads: `DATABASE_URL` for each copy, and `TESSERA_PUBLISHER_DATABASE_URL` for the seed.

`tessera.json` sits beside the program and holds the area, the tile limits, the database pool and `Catalogue.RefreshSeconds`, the 30 seconds within which a new grant reaches the map. Any environment variable overrides the key it names, with `__` for each step down, such as `Tiles__CacheSeconds=120` (ADR 0003). `npm start` uses the same rule to give each copy its own port, through `Urls`.

> WATCH OUT The database gives its two roles their passwords only on its first start. A new password in `.env` alone locks the service out. Change it in the database first, with `\password` in `psql`, then in `.env`, then restart Tessera.

The delivery plan proves the machine twice

The delivery plan tests the office machine at the start of the work and again at the end. On Monday 19 October, the first day, you install WSL2 with Ubuntu 24.04, Docker Engine, the .NET 10 SDK, Node.js 26 and an editor; the day ends when the repository's own `npm test` passes. In the first week you run `npm run office-port` on the office machine and ask whoever runs the office network for a reserved address, one that never changes. On Monday 7 December, the release, you run `npm test` from a clean copy, then `npm start` on the office machine, and open the map from another office machine.

> NOTE The plan's own release holds Parts A and B of the stories: one copy of the service, with no cache and no limits. The repository, its worked answer, already runs everything this guide shows; the plan adds the second copy, the start-up task and the limits in its second part, after 8 December.

> TRY IT Write the runbook Tessera lacks. Study plan 5, `docs/study/05-deployment-and-operations.md`, asks for `docs/guides/runbook.md`, answer first, covering start, stop, health, a failing copy, a database that is down, restore, and a password change. Base each step on the scripts this guide names, then follow your runbook word for word through one restore on a spare database.

Check your understanding

1. Why would NAT mode break Nginx's limit of 100 tiles a second for each machine?
2. The machine is off from 01:00 to 03:00. What happens to Tessera and to the night's backup?
3. Why does the backup write a `.part` file first?

Answers. 1. Every request would seem to come from one address, so the whole office would share one limit, and the office-only rule could not tell machines apart. 2. A minute after Windows starts, the start-up task opens Ubuntu; systemd starts Docker, which brings back both containers; `npm start` starts the two copies. The backup task, set to start when available, runs the missed backup as soon as Windows is up. 3. A backup cut short by a full disk or a power cut never passes for a good one: only a whole, readable file gets its final name.

> THINK IT OVER Every backup lives on the office machine's own disk unless `TESSERA_BACKUP_FOLDER` names another. What would the office lose if that disk failed tonight, and what would it take to lose nothing?

A

Everyday Commands

About forty commands do a working day's work, and most go through `npm`, whose scripts in `package.json` run .NET, Docker, Node.js and Python in the right order. Type them in the Ubuntu terminal, from the repository's root, unless a line names another folder. Appendix B lists the Linux commands around them.

Set up once

TASK	COMMAND
Install the scripts' tools, zx and TypeScript	`npm install`
Make your settings file, then give each password a value	`cp .env.example .env`
Check WSL2's networking, and print the firewall steps for port 80	`npm run office-port`
Print the PowerShell that starts Tessera with Windows	`npm run boot-task`
Print the PowerShell that backs up each night at 02:00	`npm run backup-task`

Build and test

TASK	COMMAND
Build the map into `web/dist`, then the service, in Release, with the map inside it	`npm run build`
Run every check: scripts, pipeline, map, service, browser; fails below 94 per cent of the service's lines or 84 per cent of its branches (96.1 and 85.5 on 18 October 2026)	`npm test`
Merge the three suites' coverage records again, and check the floors; open `coverage/report/index.html` to see each line no test runs	`node scripts/coverage.ts`
Run only the quick checks: the scripts' types, then their tests	`npm run typecheck`; `npm run test:scripts`
Build only the service and its three test projects; needs `web/dist` from a map build	`dotnet build`
Check the borders between the service's layers, on the built program	`dotnet run --project tests/Tessera.ArchitectureTests`
Run the unit and property tests, with no database	`dotnet run --project tests/Tessera.UnitTests`
Run the integration tests, which start the packaged database in Docker	`dotnet run --project tests/Tessera.IntegrationTests`

Run the pipeline's tests, in `etl/`	`uv run pytest`
Run the map's type check, style check and tests, in `web/`	`npm test`
Run the browser tests and the Nginx checks, on a database of their own	`npm run e2e`
The same, with Playwright inside its own Docker image, as the GitHub build runs it	`TESSERA_PLAYWRIGHT_IN_DOCKER=1 npm run e2e`

Run Tessera

TASK	COMMAND
Build, start the database, run any new seed script, then start two copies and Nginx	`npm start`
Stop both copies; the containers keep running	Ctrl+C, in the window that runs `npm start`
Serve tiles and the map on 127.0.0.1, port 7800, without the scripts; needs `DATABASE_URL`	`dotnet src/Tessera/bin/Release/net10.0/Tessera.dll`
Run each new script in `db/seed` once, as `tessera_publisher`; needs `TESSERA_PUBLISHER_DATABASE_URL`	`dotnet src/Tessera/bin/Release/net10.0/Tessera.dll seed`
Serve the map alone, with tiles passed on to a running copy, at `http://localhost:5173`, in `web/`	`TESSERA_BASE_URL=http://127.0.0.1:7800 npm run dev`

Bring in places, and keep them safe

TASK	COMMAND
Describe a source, and print a starting pipeline file	`npm run inspect -- data/parks.gdb`
Clean a file of places into the next seed script and its log	`npm run etl -- data/parks.pipeline.json`
Grant the nine layers to the service, which serves them within 30 seconds	`npm run publish`
Back up the database to `backups/`, keeping the newest 14	`npm run backup`
Restore a backup, in one transaction	`npm run restore -- backups/tessera-2026-10-17T1800Z.dump`

Ask the service, and read its logs

TASK	COMMAND
Ask, through Nginx, whether the service is healthy: `UP`, or 503 with `DOWN` or `OUT_OF_SERVICE`	`curl -s http://127.0.0.1/health`
Fetch the example tile's headers, through Nginx	`curl -I http://127.0.0.1/food.cafes/12/3339/1788.pbf`
Read the last 100 lines the database or Nginx printed	`docker logs --tail 100 tessera-db` (or `tessera-nginx`)
Follow what the start-up task's `npm start` prints	`tail -f start.log`

Talk to the database

TASK	COMMAND
Open a `psql` prompt as the owner, `postgres`	`docker exec --interactive --tty tessera-db psql --username postgres --dbname tessera`
List a schema's tables; show who may read them (at the `psql` prompt)	`\dt food.*`; `\dp food.*`
List the seed scripts DbUp has run (at the `psql` prompt)	`TABLE tessera.schemaversions;`
Change a role's password (M4 gives the order of steps)	`\password tessera_reader`

Look after Docker

TASK	COMMAND
List the containers, both running and stopped	`docker ps --all`
Start the database again after it stopped	`docker start tessera-db`
Delete the volume that holds the database, and every place in it: never run this	`docker volume rm tessera-data`

Three habits save the most trouble: back up before any change you may want to undo; read `start.log` before a restart buries the evidence; and add a new seed script rather than edit an old one, since DbUp never runs an edited script again.

B

Linux Commands for Every Day

About forty Linux commands cover nearly every task outside Tessera's own scripts: moving between folders, reading and finding files, guarding them, watching programs, checking the disk and the network, packing archives and installing software. Each table gives the command, what it does in plain words, and an example from Tessera's world that you can type in the Ubuntu terminal. The examples run from the repository's folder, wherever you cloned it. Appendix A lists the commands that belong to Tessera itself.

A command has three parts

Most commands follow one pattern: the program's name, then **options** that change how it behaves, then the things it works on. In `ls -l db/seed`, `ls` lists files, `-l` asks for the long form, and `db/seed` names the folder. The sign `~` stands for your home folder, `.` for the folder you are in, and `..` for the one above it. Add `--help` to most commands for a short summary. For the full manual, type `man` and the command's name, such as `man ls`; press `q` to leave it.

Move around and look

COMMAND	WHAT IT DOES	EXAMPLE
`pwd`	Shows the folder you are in	`pwd`
`cd`	Moves to another folder; `cd -` goes back	`cd etl`
`ls`	Lists a folder; `-la` adds hidden files, sizes and dates	`ls -la` (shows `.env`)

Work with files and folders

COMMAND	WHAT IT DOES	EXAMPLE
`mkdir -p`	Makes a folder, and any missing folders above it	`mkdir -p /mnt/d/tessera-backups`
`cp`	Copies a file; `-r` copies a folder	`cp .env.example .env`
`mv`	Moves or renames	`mv data/parks.json data/parks.pipeline.json`
`rm`	Deletes for good; there is no recycle bin	`rm start.log.1`

Read files

COMMAND	WHAT IT DOES	EXAMPLE
`cat`	Prints a whole file	`cat tessera.json`
`less`	Shows a long file one screen at a time; `/word` searches, `q` quits	`less docs/guides/delivery-plan.md`

`head`, `tail`	Show the first or last lines; `tail -f` keeps showing new lines as they arrive	`tail -f start.log`
`wc -l`	Counts lines	`wc -l scripts/*.ts`
`nano`	An editor in the terminal; Ctrl+O saves, Ctrl+X leaves	`nano .env`

Find files and text

COMMAND	WHAT IT DOES	EXAMPLE
`grep`	Prints the lines that hold a pattern; `-r` searches a folder, `-n` adds line numbers, `-i` ignores case	`grep -rn "US-36" etl/src`
`find`	Finds files by name, size or age	`find db -name "*.sql"`
`sort`, `uniq -c`	Sort lines; count repeated ones. Here `cut` keeps each line's first field, the sender's address, so the count gives each office machine's requests	`docker logs tessera-nginx | cut -d" " -f1 | sort | uniq -c`

Join commands together

Small commands join into larger ones. A ***pipe***, `|`, hands one command's output to the next as its input. A ***redirection***, `>`, writes the output to a file instead of the screen; `>>` adds to the end of the file instead of replacing it.

SYMBOL	WHAT IT DOES	EXAMPLE
`|`	Passes output on to the next command	`docker logs tessera-nginx | grep " 429 "`
`>`, `>>`	Writes output to a file; adds to a file	`npm test > test.log`
`&&`	Runs the second command only if the first succeeded	`npm run build && npm start`
`$( )`	Puts one command's output inside another	`echo "dotnet: $(which dotnet)"`
`2>&1`	Sends error messages to the same place as the output	`dotnet build > build.log 2>&1`

Permissions and owners

Every file has an owner and three sets of rights: for the owner, for the owner's group, and for everyone else. Each set may read (`r`), write (`w`) and run (`x`). As numbers, read is 4, write 2 and run 1, so `600` lets the owner read and write and gives everyone else nothing: the right setting for a file of passwords.

COMMAND	WHAT IT DOES	EXAMPLE
`ls -l`	Shows each file's rights, owner and group	`ls -l .env`
`chmod`	Changes a file's rights	`chmod 600 .env`
`chown`	Changes a file's owner	`sudo chown $USER backups`
`sudo`	Runs one command as root, the user with every right; asks for your password	`sudo apt update`
`id`	Shows who you are and the groups you belong to	`id` (look for `docker`)

Programs that are running

A running program is a ***process***, and each has a number, its process ID. Ctrl+C stops the program running in the window in front of you. The commands below find and stop the rest.

COMMAND	WHAT IT DOES	EXAMPLE
`ps aux`	Lists every process	`ps aux | grep Tessera.dll`
`pgrep -af`	Finds processes by name, with their command lines	`pgrep -af Tessera.dll`
`top`, `htop`	Show the busiest processes, updated live; `q` quits	`htop`
`kill`	Asks a process to stop; `-9` forces it, as a last resort	`kill 4127`

Disk, memory and the machine

COMMAND	WHAT IT DOES	EXAMPLE
`df -h`	Shows free space on each disk, in units a person reads easily, such as G for gigabytes	`df -h /`
`du -sh`	Shows how much space a folder uses	`du -sh backups/`
`free -h`	Shows memory in use and free	`free -h`

The network

COMMAND	WHAT IT DOES	EXAMPLE
`curl`	Downloads what a web address serves; `-I` shows the headers alone, `-s` hides progress	`curl -s http://127.0.0.1/health`
`ping`	Checks that another machine answers; `-c 3` stops after three tries	`ping -c 3 github.com`
`ip addr`	Lists this machine's network addresses	`ip -4 addr`
`ss -ltnp`	Lists the ports programs listen on	`ss -ltnp | grep 7800`

Archives

COMMAND	WHAT IT DOES	EXAMPLE
`tar -czf`	Packs a folder into one compressed file	`tar -czf seed-logs.tar.gz db/seed-logs`
`tar -xzf`	Unpacks it	`tar -xzf seed-logs.tar.gz`
`zip -r`, `unzip`	Make and open `.zip` files, which Windows opens directly	`zip -r docs.zip docs`

Software and services

Ubuntu installs software through its ***package manager***, `apt`, which downloads each program and everything it needs from Ubuntu's own servers. systemd starts and watches the programs that run in the background, such as Docker.

COMMAND	WHAT IT DOES	EXAMPLE
`sudo apt update`	Refreshes the list of available versions	`sudo apt update`
`sudo apt install`	Installs a program	`sudo apt install gdal-bin htop`
`systemctl status`	Shows whether a service runs, and its last log lines	`systemctl status docker`
`sudo systemctl restart`	Stops and starts a service	`sudo systemctl restart docker`
`dotnet --version`, `node --version`	Show which release runs: the .NET 10 SDK and Node.js 26	`dotnet --list-sdks`

Between Ubuntu and Windows

COMMAND	WHAT IT DOES	EXAMPLE
`code .`	Opens the current folder in VS Code	`code .`
`wslinfo`	Reports WSL's settings, such as its networking mode	`wslinfo --networking-mode`
`wsl --shutdown`	Stops WSL completely; type it in PowerShell, and open Ubuntu to start WSL again	`wsl --shutdown`

Keep the repository on the Linux side, under `~`. Files under `/mnt/c` live on the Windows disk, and builds there run several times slower.

Keys that save time

KEY	WHAT IT DOES
Tab	Completes a command or a file's name; press it twice to see the choices
Up arrow	Brings back the previous command
Ctrl+R	Searches the commands you typed before; type part of one, then press Enter
Ctrl+C	Stops the program that is running
Ctrl+L	Clears the screen

> COMMON MISTAKE Typing `sudo rm -rf` with a path you have not read twice. `rm -rf` deletes a folder and everything in it. Linux asks no question and keeps no copy, and a stray space, as in `rm -rf ~ /old`, deletes your whole home folder, with the repository and `.env` inside it. Run `ls` on the same path first, and use `sudo` only when a command refuses to work without it.

Glossary

This glossary explains, in plain words, every term the guide sets in bold italics. The index gives the page that explains each one.

304 Not Modified
The answer, sent with no body, that tells a browser its own copy of a tile is still good.

.NET SDK
The toolkit, run as the command `dotnet`, that builds, tests and runs C# programs.

Analyser
A check that runs inside the C# compiler and stops the build when the code breaks its rule, such as a `switch` that misses a case.

Approved file
A test's expected output, kept as a file; the test fails until someone reads any change and copies it over the old file.

Approved image
A screenshot of the map, checked and accepted by a person, that a browser test compares with a fresh one.

Architecture decision record
A short note of one design choice: the decision, its facts, the options set aside and the cost. Tessera keeps its records in `docs/adr`.

Architecture test
A test that checks a program's structure, such as which folder may use which, rather than its behaviour.

Asynchronous method
A method that returns a `Task`, a promise of a result to come, and hands its thread back while it waits.

Atomic
Done in one step that no other thread can see half done.

Baseline scan
The quick scan of ZAP, a free security scanner: it follows links and reads the replies, but sends no attacks.

Bounding box
The smallest rectangle that holds a shape.

Cancellation token
A signal passed from a caller down to each piece of work it starts, so the caller can say 'stop' to work it no longer needs.

Cast
A conversion of a value from one type to another, such as from `int` to `double`.

Central package management
Naming each package's version once, in `Directory.Packages.props`, so every project uses the same one.

Client–server model
A design in which clients send requests and a server sends back responses.

Code coverage
The share of a program's lines and branches that the tests run. Tessera measures it on every test run, and the build fails below 94 per cent of the lines or 84 per cent of the branches.

Cohesion
How closely the contents of one part belong together. Good design keeps it high.

Command
In the map, a request for work outside the pure update, such as moving the map, handed out through a port.

Compiler
A translator that checks a whole program and turns it into code a machine can run.

Concurrency
Many tasks in progress over the same period, though not always at the same instant.

Conditional request
A request that carries the tag of the copy the browser holds, so the server can answer 304 when nothing changed.

Connection pool
A small set of open database connections, each lent to one query at a time; Tessera's holds four for each copy of the service.

Container
A sealed box in which Docker runs one program with the files it needs, the same way on every machine.

Content Security Policy
A header that tells the browser where a page may load scripts, styles and other files from.

Continuous integration
Building and testing every change on a fresh machine as soon as someone pushes it to the shared repository.

Cooldown

Dependabot's wait of seven days after a release before it proposes it.

Coupling

How much one part depends on another. Good design keeps it low.

Dapper

A small .NET library that sends a query with its values and turns each row of the answer into a C# record.

DbUp

A .NET library that runs each SQL script it has not run before, once, in the order of the scripts' names, and records each name in its journal.

Defence in depth

Several barriers, one behind another, so that the rest still stand when one fails.

Delegate

C#'s name for the type of a function, so that a function can be passed as a value.

Denial of service

An attack that asks so much of a server that honest users get no answer.

Dependabot

GitHub's robot, which proposes each newer release of the packages, tools and images that a project pins at exact versions.

Dependency injection

Handing each part what it needs from outside, rather than letting it build its own.

Dump

A file that holds a whole database, from which `pg_restore` can rebuild it.

Elmish pattern

A design in which one model holds the page's state and a pure update makes the next model from each message.

Endpoint

The one method that ASP.NET Core chooses to answer a request.

Endpoint filter

A step that runs around a chosen group of endpoints and no others, such as `PageHeaders` around the map's page.

Environment variable

A named value that a program reads from the system it runs on, such as a database's address.

EPSG:4326

The coordinate system of longitude and latitude in degrees, as GPS reports them.

ETag

A short tag that names a response's exact contents, so a browser can ask whether its copy changed.

ETL

Extract, transform, load: read data, clean it, and write it where it is used.

Exception

.NET's way to break off a method when something goes wrong; Tessera catches each one from the database at the edge and turns it into a value.

Extent

The size of a vector tile's own grid: 4,096 units a side in Tessera.

False positive

A weakness that a scanner reports where none exists.

Fencepost error

Counting the posts when you meant the gaps between them, or the reverse: 14 backups span 13 days.

Floor function

The largest whole number at or below a given number, written $\lfloor x \rfloor$.

Forward proxy

A proxy that acts for clients, such as a school's web filter.

Framework

Ready-made code that runs your program and calls your parts, such as ASP.NET Core.

Function composition

Joining two functions so that the output of the first is the input of the second.

Functional core, imperative shell

A design in which pure functions make every decision and a thin shell does the work with side effects, such as running a query.

GeoDataFrame

In Python, a table of rows with one column of shapes.

Generic type

A type that takes other types as parameters, such as `Result<T, E>`.

Geometry

PostGIS's type for a shape: a point, a line or an area, in a stated coordinate system.

GiST index

The kind of spatial index PostGIS uses: it keeps shapes' bounding boxes in a tree, so a search skips whole branches.

Golden file

A recorded output that every later run of a test must match exactly.

Grant
A database statement that gives a role a right, such as reading a table; in Tessera, the act of publishing.

Guard
A `when` clause that adds a condition to one arm of a `switch`.

gzip
A way of packing data into fewer bytes, which the browser unpacks.

Health check
An address, `/health` in Tessera, that a watcher asks to learn whether the service can do its job.

Host
The code that builds the web service from its parts and starts it: `TesseraHost` in Tessera.

Hosted service
A loop that .NET starts and stops with the program and runs beside the requests, such as the reload of the list of layers.

HTTP
The Hypertext Transfer Protocol: the rules by which browsers and web servers exchange requests and responses.

Immutable
Never changing once made; to 'change' an immutable value, you make a new one.

Intermediate language
The instructions the C# compiler writes, which the .NET runtime turns into the processor's own as the program runs.

Journal
DbUp's table, `tessera.schemaversions`, that records the name of each script it has run.

Judge
In the pipeline, a function that reads a whole table and gives a verdict, fixed or failed, on each row it would change or refuse.

Kestrel
The web server inside ASP.NET Core, which accepts each connection and reads each request.

Lambda
A function without a name, written where it is needed.

Latency
The time from a request to its answer.

Layer (of code)
A band of code that may call the bands beneath it, never those above; Tessera's module has three: Domain, Infrastructure and Features. Elsewhere in the guide, a layer is one published table of places, such as `food.cafes`.

Leaky bucket
A model of a rate limit: each request pours into a bucket that drains at a steady rate, and a request that would overflow it is refused.

Least privilege
Giving each part only the powers it needs.

Little's law
The rule that the average count inside a steady system equals the arrivals each second times the time each one stays: $L = \lambda W$.

Load test
A test that sends a steady stream of requests and records the time of every answer.

Logic error
A mistake that compiles and runs but gives the wrong answer; only a test or a careful reader catches it.

Malformed URL
An address that hides its real shape until a server tidies it, such as `%2e%2e` for `..`.

Message
In the map, a value that says what happened, such as a click, which the update turns into the next model.

Method group
The name of an existing method, passed as a value without calling it.

Microservices
A design that splits the work into separate programs that talk over the network.

Middleware
The small steps every request passes through in ASP.NET Core; each may inspect the request, change the response or answer at once.

Minimal API
ASP.NET Core's way of joining a path to a method in one line, with no controller class around it.

Mirrored networking
A mode of WSL2 in which Ubuntu shares the Windows machine's own network address, so Nginx sees who sent each request.

Model
In the map, the one read-only value that holds everything the page shows.

Modular monolith

One program, built and shipped as one unit, whose inside is divided into parts with fixed borders.

Module

A part of a program that owns one job and shows the rest a small public face.

Monolith

A program built and shipped as one unit.

Namespace

A named group of types, such as `Modules.Tessera.Domain`; in Tessera each matches its folder.

Npgsql

The .NET driver for PostgreSQL, which opens connections, keeps them in a pool and sends each query.

Nullable type

A type written with a question mark, such as `Layer?`, whose value may be missing; the compiler makes the caller check it.

Options pattern

ASP.NET Core's way of copying one section of the settings into a record and checking it.

Overflow

A result too large for its type, which wraps round to a wrong value, such as `1 << 31` in an `int`.

OWASP

The Open Worldwide Application Security Project, a charity that publishes free security guidance.

OWASP Top 10

OWASP's list of the commonest kinds of weakness in web applications.

Package manager

A program, such as Ubuntu's `apt`, that installs software with everything it needs.

Parallelism

Tasks running at the same instant, on different processor cores.

Parameter

In a query, a value sent beside the SQL text, such as `@z`, which the database treats as data whatever it holds.

Passive health check

Watching a server only through the requests passed to it, and resting it after a set number of failures.

Pattern matching

Choosing a result by testing a value's shape, such as its type or a property, rather than its exact value.

Penetration test

An attack on a system, made with its owner's permission, to find weaknesses before criminals do.

Percentile

A point in a sorted list of times: the p95 is the time that 95 requests in 100 stay within.

Pipe

The sign `|`, which hands one command's output to the next as its input.

Port

A number that picks one program on a machine, such as 80 for Nginx.

Port (in the map)

A function passed into the map's update from outside, through which a command leaves, so a test can pass a fake.

Prepared statement

A query that the database parses and plans once, keeps, and reruns with new values.

Primary key

A column whose value no two rows share, such as `fid`.

Problem document

A short JSON report of an error, in the shape that RFC 9457 sets, with nothing from inside the server.

Process

A running program, with a number of its own, its process ID.

Property test

A test that checks a rule for many made-up inputs rather than a few chosen ones, and shrinks any failure to the smallest input that still fails.

Proxy

A program that acts for another, passing requests on.

Pull request

A proposed change to the code, which others review and the build tests before it joins the main branch.

Pure function

A function whose answer depends only on its inputs and which does nothing besides return it.

Quadtree

A tree in which each square has four children: the shape of the tiles across zoom levels.

Query plan

The steps PostgreSQL chose to run a query, which `EXPLAIN` prints with the time and pages each took.

Race condition

A fault whose result depends on which thread moves first.

Railway-oriented programming

Joining steps that may fail so that the first failure switches the run to a second track, which every later step passes along.

Rate limit

A cap on how many requests a client may make over time.

Record

In C#, a type that groups named values into one and compares by value, like a row of a table.

Redirection

The sign `>`, which writes a command's output to a file instead of the screen.

Reference type

A type whose variable holds an address that leads to an object, which a copy then shares.

Relational database

A database that keeps its data in tables of rows and columns and answers questions in SQL.

Reverse proxy

A proxy that acts for servers: clients talk to it, and it passes each request inside.

Revoke

A database statement that takes a right away again; in Tessera, the act of withdrawing a layer.

Role

A database account with its own rights, such as `tessera_reader`.

Route constraint

A rule in a route's pattern, such as `:int`, that a part of the path must meet before the route may answer.

Schema

A named group of tables inside a database, like a folder, such as `food`.

Semantic versioning

Numbering releases `MAJOR.MINOR.PATCH`: a breaking change raises MAJOR, a new feature MINOR, and a fix PATCH.

Sequential scan

Reading every row of a table in turn, without an index.

Service (dependency injection)

A part registered with ASP.NET Core, which builds it and hands it to whatever asks for its type.

Side effect

Anything a function does besides return its answer, such as reading a database or the clock.

Singleton

A service built once and shared by the whole program, such as Tessera's one `Database`.

Smart constructor

A type whose own constructor is private, so only its static methods, which check each value, can make one.

Software supply chain

Every library, image and tool a build pulls in.

Spatial index

An index over shapes that lets the database find those in one area without reading every row.

SQL injection

An attack in which text glued into a database command changes what the command does.

SRID

A spatial reference identifier: the number that says which coordinate system a shape's numbers belong to.

Start-up task

The Windows task, 'Tessera', that starts Ubuntu and Tessera a minute after Windows starts.

State machine

A set of stages, the events that move between them, and a rule that gives the next stage.

Status code

The number that opens an HTTP response and says how the request went, such as 200 or 404.

Step

In the pipeline, a function that takes a context and returns a changed copy of it.

Stop rule

A rule that stops the pipeline's run when the filters drop too many of a table's rows, or leave too few.

Subnet

A range of network addresses that share their first bits, such as `192.168.10.0/24`.

System catalogue

PostgreSQL's own tables, which describe every table, column and right.

systemd

The first program Ubuntu runs, which starts and watches the others, such as Docker.

Test-driven development

Writing each test before its code: watch it fail, write the least code that passes, then tidy while every test stays green.

Testcontainers

A library that starts Docker containers for a test and removes them afterwards.

Thread

One sequence of steps that a program runs in order.

Thread pool

.NET's set of threads that each take the next piece of waiting work, finish it, and come back for more.

Throughput

The work a system finishes each second.

Tile

One square of the map, named by its zoom level, column and row, such as `12/3339/1788`.

TLS

The encryption behind HTTPS, which stops anyone on the way from reading or altering what travels.

Transaction

A group of database changes that succeeds whole or not at all.

Type

A statement of which values a variable may hold and what you may do with them.

User story

A short statement of one thing a person needs, with the criteria that say when it is done, such as US-07 'Serve a tile'.

Utilisation

The share of a system's capacity in use, written ρ.

Value type

A type whose variable holds the value itself, so a copy is a separate value.

Vector tile

A tile that holds places as shapes and properties rather than as a picture, so the browser can draw and style them itself.

Virtual machine

A program that behaves as a whole computer, with its own operating system.

Web Mercator

The flat projection of the Earth that web maps use, EPSG:3857, measured in metres.

xUnit

The test framework of Tessera's three .NET test projects.

Zoom level

The number that says how far the map is zoomed in; each level splits every tile of the level above into four.

zx

Google's library for writing scripts in TypeScript, whose `$` runs a command and quotes each value in it.

Index

Each term's page is where the book first explains it.

0–9

304 Not Modified, 26

A

analyser, 10
approved file, 58
approved image, 57
architecture decision record, 7
architecture test, 19
asynchronous method, 35
atomic, 37

B

baseline scan, 53
bounding box, 48

C

cancellation token, 37
cast, 13
central package management, 55
client–server model, 28
code coverage, 58
cohesion, 18
command, 79
compiler, 12
concurrency, 34
conditional request, 26
connection pool, 35
container, 22
Content Security Policy, 52
continuous integration, 59
cooldown, 52
coupling, 18

D

Dapper, 41
DbUp, 42
defence in depth, 49
delegate, 9
denial of service, 51
Dependabot, 52
dependency injection, 29
dump, 43

E

Elmish pattern, 79
endpoint, 28
endpoint filter, 31
environment variable, 30
EPSG:4326, 45
ETag, 25
ETL, 75
exception, 9
extent, 47

F

false positive, 53
fencepost error, 72
floor function, 68
forward proxy, 22
framework, 28
function composition, 77
functional core, imperative shell, 8

G

generic type, 14
GeoDataFrame, 75
geometry, 45
GiST index, 48
golden file, 57
grant, 41
guard, 15
gzip, 25

H

health check, 32
host, 19
hosted service, 38
HTTP, 28

I

immutable, 7
intermediate language, 55

J

journal, 42
judge, 77

K

Kestrel, 28

L

lambda, 8
latency, 61

layer (of code), 18
leaky bucket, 71
least privilege, 49
Little's law, 70
load test, 65
logic error, 13

M
malformed URL, 50
message, 79
method group, 8
microservices, 17
middleware, 28
minimal API, 31
mirrored networking, 87
model, 79
modular monolith, 17
module, 17
monolith, 17

N
namespace, 17
.NET SDK, 55
Npgsql, 41
nullable type, 14

O
options pattern, 30
overflow, 13
OWASP, 49
OWASP Top 10, 49

P
package manager, 94
parallelism, 34
parameter, 50
passive health check, 25
pattern matching, 14
penetration test, 53
percentile, 65
pipe, 93
port, 22
port (in the map), 79
prepared statement, 63
primary key, 40
problem document, 32
process, 94
property test, 11
proxy, 22
pull request, 52
pure function, 7

Q
quadtree, 45
query plan, 62

R
race condition, 37
railway-oriented programming, 8
rate limit, 24
record, 10
redirection, 93
reference type, 13
relational database, 40
reverse proxy, 22
revoke, 42
role, 40
route constraint, 31

S
schema, 40
semantic versioning, 60
sequential scan, 48
service (dependency injection), 29
side effect, 7
singleton, 29
smart constructor, 13
software supply chain, 52
spatial index, 48
SQL injection, 50
SRID, 45
start-up task, 87
state machine, 80
status code, 28
step, 76
stop rule, 76
subnet, 22
system catalogue, 41
systemd, 87

T
test-driven development, 58
Testcontainers, 57
thread, 34
thread pool, 34
throughput, 70
tile, 2
TLS, 53
transaction, 40
type, 12

U
user story, 2

utilisation, 70

V

value type, 13
vector tile, 2
virtual machine, 86

W

Web Mercator, 45

X

xUnit, 56

Z

zoom level, 45
zx, 83